
# Capital University of Science and Technology

**Project Title**

Automation Testing Framework for OpenCart Web Application

**Submitted By**


| Student Name | Registration Number |
|---|---|
| **Ali Akbar** | **BSE233016** |
| **Zainab Fatima** | **BSE233008** |
| **Laiba Zulfiqar** | **BSE233022** |


**Submitted To**

**Syeda Hafsa Ali**
Instructor, SE4343 - Automated Software Testing

**Submission Date**

20 June 2026

# Table of Contents

**Github Repository Link**: https://github.com/ali-akbar-019/OpenCartFramework

# 1. Introduction

Automation testing is an important part of software engineering because it helps improve software quality, saves testing time, and reduces manual effort. In this project, automated testing was performed on the OpenCart web application using Selenium WebDriver and TestNG. Different modules of the application were tested through automated test cases to verify their functionality and reliability.

## 1.1 Purpose of the Project

The main purpose of this project was to apply the concepts of automated software testing in a practical environment. The project focused on creating and executing automated test cases for important functionalities of the OpenCart website such as registration, login, search, account management, and cart operations. The project also aimed to gain experience with testing frameworks, reporting tools, and data-driven testing techniques.

## 1.2 Project Overview

For this assignment, the OpenCart e-commerce web application was selected as the testing platform. The application provides common online shopping functionalities including user authentication, product searching, shopping cart management, and account handling.

The testing framework was developed using Java, Selenium WebDriver, TestNG, Apache POI, and Extent Reports. Multiple automated test cases were implemented for both positive and negative scenarios. Excel sheets were used for data-driven testing, while Extent Reports generated detailed execution reports with logs and screenshots. A Lighthouse performance test was also performed to evaluate the website's basic performance and quality metrics.

## 1.3 Objectives of Automated Testing

The main objectives of this project were:

- To understand the implementation of automated testing frameworks.
- To automate functional testing of a web application.
- To validate both positive and negative test scenarios.
- To perform data-driven testing using Excel files.
- To generate detailed test execution reports using Extent Reports.
- To improve testing efficiency, accuracy, and repeatability.
- To gain practical experience with industry-standard automation tools.

## 2. Task 1 Project Selection

### 2.1 Selected Web Application

For this project, the OpenCart web application was selected for automated testing. OpenCart is an open-source e-commerce platform that allows users to browse products, manage accounts, add products to carts, and place orders online.

The application was chosen because it contains multiple real-world functionalities that are suitable for automation testing and provide practical experience with web application testing.

### 2.2 Functionalities of the Application

The following functionalities of the OpenCart application were tested during the project:

- User Registration
- User Login and Logout
- Product Search
- Product Selection
- Add to Cart
- Remove from Cart
- Account Management
- Wishlist Navigation
- Order History Navigation
- Password Change Navigation

These modules were selected because they represent the core features of an e-commerce system and are important for ensuring application reliability and usability.

## 2.3 Tools and Technologies Used

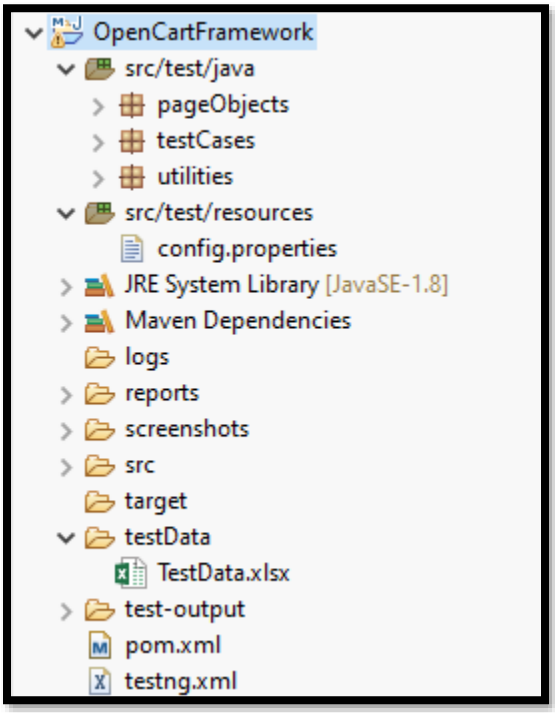
The following tools and technologies were used in the project:

| Tool / Technology | Purpose |
|---|---|
| Java | Programming language used for automation scripts |
| Selenium WebDriver | Web UI automation testing |
| TestNG | Test execution and test management |
| Apache POI | Reading test data from Excel files |
| Extent Reports | Generating execution reports with logs and screenshots |
| Eclipse IDE | Development environment |
| ChromeDriver | Browser automation |
| Lighthouse | Website performance testing |

## 2.4 Testing Approach

The project followed an automated functional testing approach using the Page Object Model (POM) design pattern. Separate page classes were created for different web pages to improve code reusability and maintainability.

Positive and negative test cases were executed to verify the behavior of the application under different conditions. Data-driven testing was also implemented using Excel files to test multiple login scenarios efficiently.



## 3. Task 2 Unit Testing and Automation Testing

### 3.1 Testing Framework Design

The automation framework was developed using Selenium WebDriver with TestNG following the Page Object Model (POM) design pattern. Separate page object classes were created for each module of the application to keep the code organized, reusable, and easy to maintain.

The framework also included utility classes for handling screenshots, reading Excel data, generating reports, and managing browser configuration.

### 3.2 Test Scenarios Implemented

The following automated test cases were implemented in the project:

**Registration Module**

- Valid account registration
- Registration with existing email
- Validation of required fields

**Login Module**

- Login with valid credentials
- Login with invalid credentials
- Login with empty fields
- Multiple login scenarios using Excel data

**Search Module**

- Search valid products
- Search invalid products
- Search using partial keywords

**My Account Module**

- Verify My Account page
- Verify Edit Account option
- Verify Change Password option
- Verify Wishlist option
- Verify Order History option
- Verify Logout functionality

**Cart Module**

- Add product to cart
- Add multiple products to cart
- Add same product multiple times
- View cart page
- Remove product from cart

## 3.3 Positive and Negative Testing

Both positive and negative testing scenarios were implemented to ensure comprehensive test coverage.

**Positive Testing Examples**

- Successful login with correct credentials
- Successful product search
- Successfully adding products to cart

**Negative Testing Examples**

- Login with invalid credentials

- Registration with missing fields
- Searching unavailable products

### 3.4 Data-Driven Testing

Data-driven testing was implemented using Excel files with Apache POI. Multiple sets of login credentials were stored in Excel sheets and automatically read during test execution.

This approach helped reduce code duplication and improved test coverage for different login combinations.

### 3.5 Reporting and Logging

Extent Reports were integrated into the framework to generate detailed execution reports. The reports included:

- Pass/Fail status of test cases
- Execution logs
- Error details
- Screenshots of failed test cases
- Test execution time

These reports made it easier to analyze test execution results and identify failures quickly.

### 3.6 Test Execution Results

Most of the implemented test cases executed successfully. The framework successfully automated major functionalities of the OpenCart application including registration, login, searching, account verification, and cart operations.

Some failures occurred during development due to dynamic web elements, incorrect XPath locators, synchronization issues, and click interception errors. These issues were resolved using explicit waits, improved XPath strategies, and JavaScript scrolling techniques.

# 4. Task 3 Final Report and Presentation

## 4.1 Introduction to Task 3

This section presents the final phase of the automated testing project, where the developed Selenium-based test automation framework is demonstrated through execution results, reports, and supporting evidence.

The main focus of this task is to validate the functionality of the selected web application using automated test scripts and to document the complete testing process in a structured and professional manner. This includes test case design, execution results, reporting, and supporting artifacts such as screenshots and logs. In this phase, different testing techniques have been applied, including Page Object Model (POM) for better code structure, TestNG for test execution management, and Excel-based data-driven testing for handling multiple input scenarios. Additionally, reporting tools like Extent Reports and performance analysis using Lighthouse have been used to evaluate the application from multiple perspectives.

The objective of this section is to provide clear evidence of test execution, results, and overall quality assurance activities performed during the automation testing process.

## 4.2 Overview of Deliverables

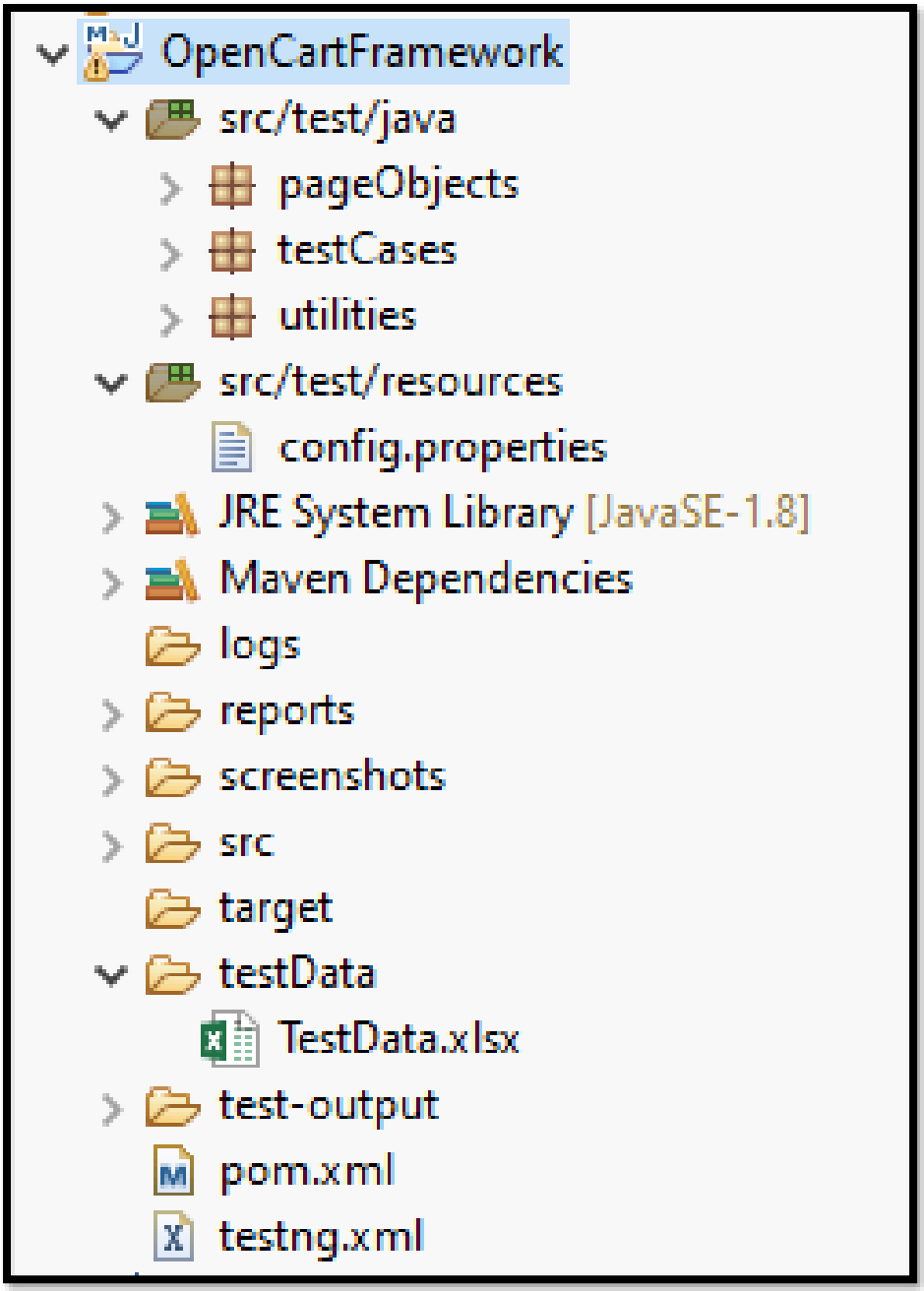
This section includes the main deliverables produced during the automation testing project. The framework is developed using Selenium WebDriver with TestNG, following the Page Object Model (POM) design pattern.

The project covers functional test automation, data-driven testing, reporting, and basic performance analysis.

### 4.2.1 Automation Framework Summary

The automation framework includes the following components:

- Page Object Model (POM) for page separation
- TestNG framework for test execution
- Excel-based data-driven testing
- Extent Reports for test reporting
- Screenshots for test evidence
- Logs for execution tracking


OpenCartFramework
src/test/java
pageObjects
testCases
utilities
src/test/resources
config.properties
JRE System Library [JavaSE-1.8]
Maven Dependencies
logs
reports
screenshots
src
target
testData
TestData.xlsx
test-output
pom.xml
testng.xml

### 4.2.2 Tools and Technologies Used

Selenium WebDriver: UI automation

TestNG: Test execution and assertions

Java: Programming language

Apache POI: Excel handling for test data

Extent Reports: HTML test reporting

Maven: Project dependency management

Lighthouse: Performance testing

OpenCartFramework/pom.xml

```
https://maven.apache.org/xsd/maven-4.0.0.xsd (xsi:schemaLocation)
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">

    <modelVersion>4.0.0</modelVersion>

    <groupId>framework</groupId>
    <artifactId>OpenCartFramework</artifactId>
    <version>0.0.1-SNAPSHOT</version>

    <dependencies>

        <!-- Selenium Java -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.44.0</version>
        </dependency>

        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.12.0</version>
            <scope>test</scope>
        </dependency>

        <!-- Apache POI -->
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.4.1</version>
        </dependency>

```

```
        <!-- Extent Reports -->
        <dependency>
            <groupId>com.aventstack</groupId>
            <artifactId>extentreports</artifactId>
            <version>5.1.2</version>
        </dependency>

        <!-- Log4j API -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>2.26.0</version>
        </dependency>

        <!-- Log4j Core -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>2.26.0</version>
        </dependency>

    </dependencies>

</project>
```

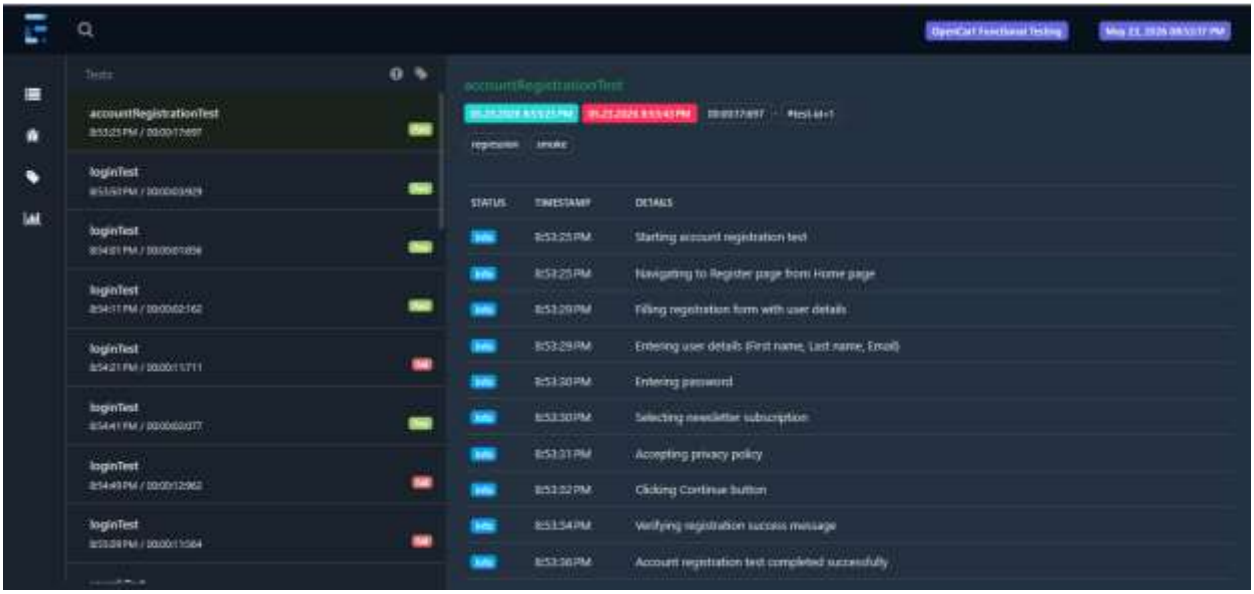
### 4.2.3 Test Artifacts Generated

The following artifacts were generated during execution:

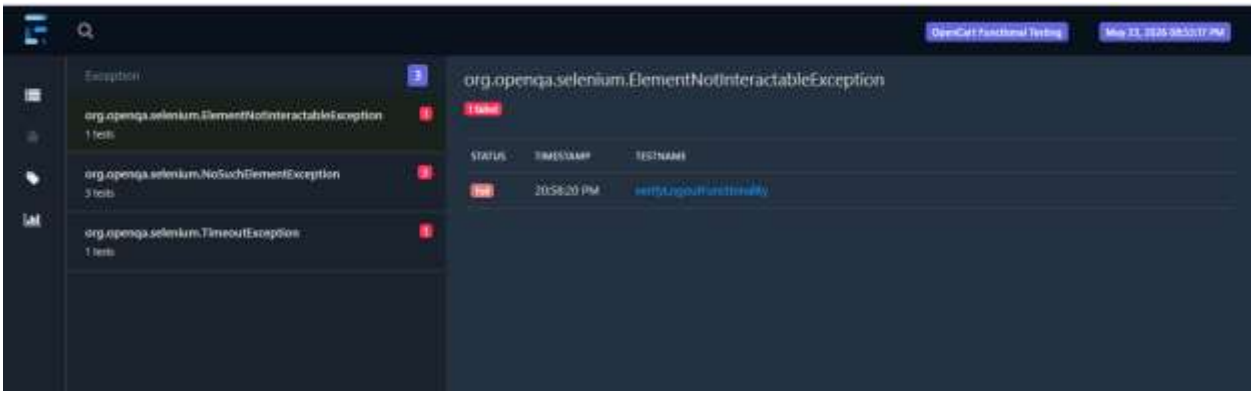
- TestNG execution reports
- Extent HTML reports
- Excel-based test cases
- Screenshots of test execution
- Logs for debugging
- Lighthouse performance report

## Extent Report

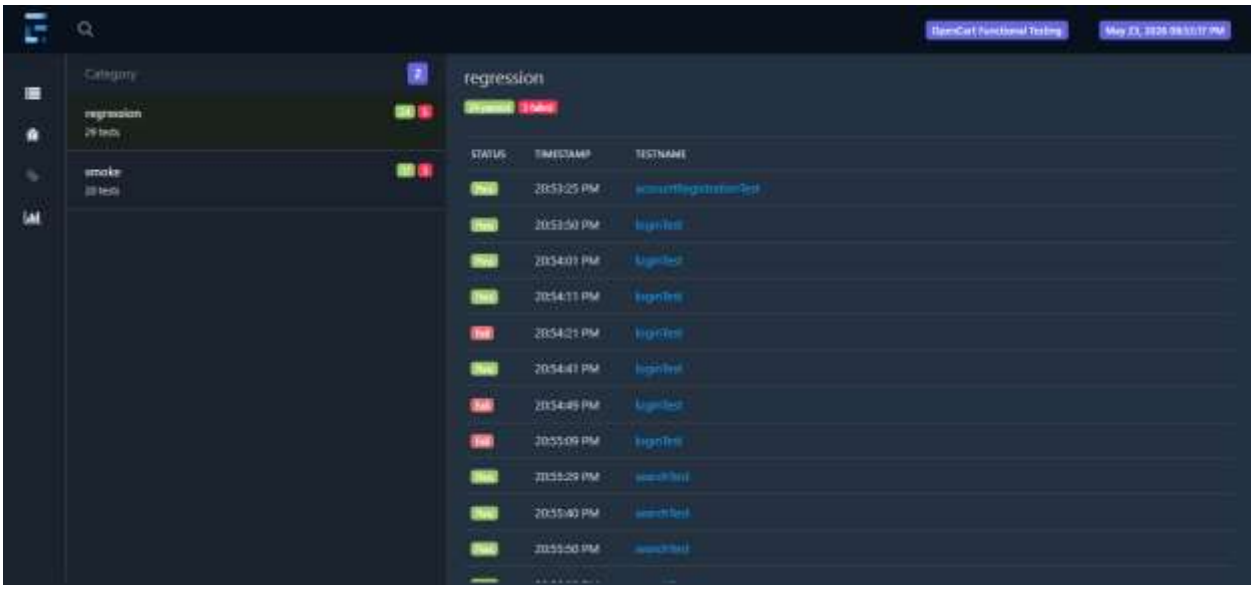
## Extent Report Dashboard



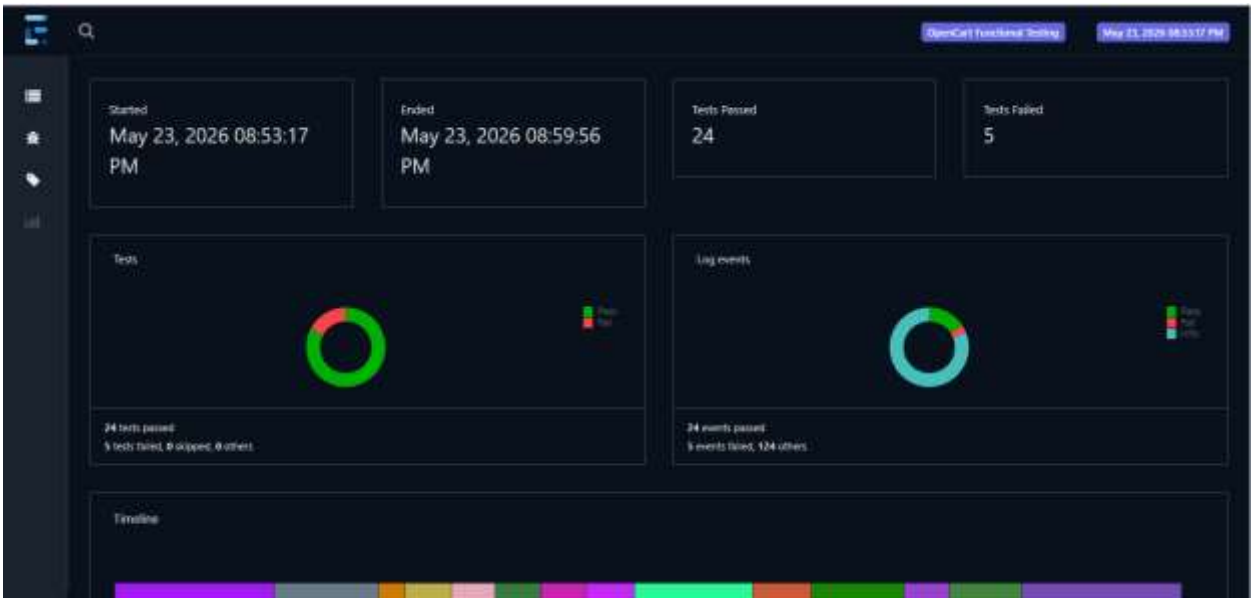

## Bugs

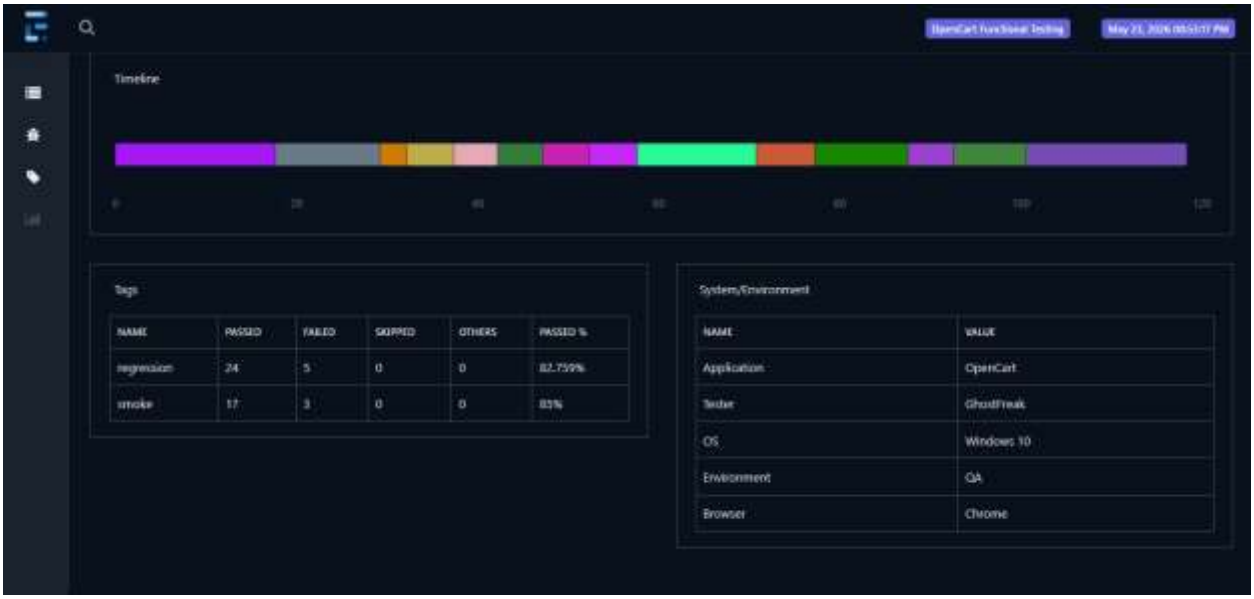


## Categories



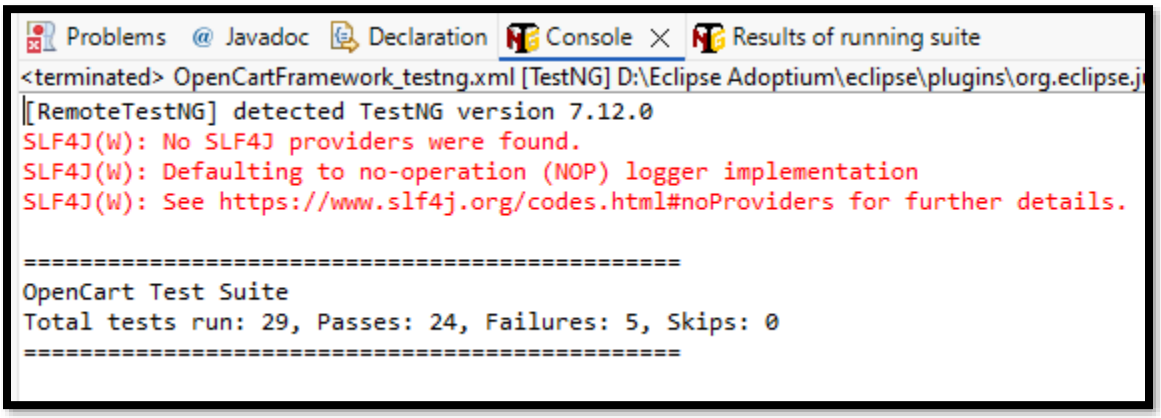

## Visuals

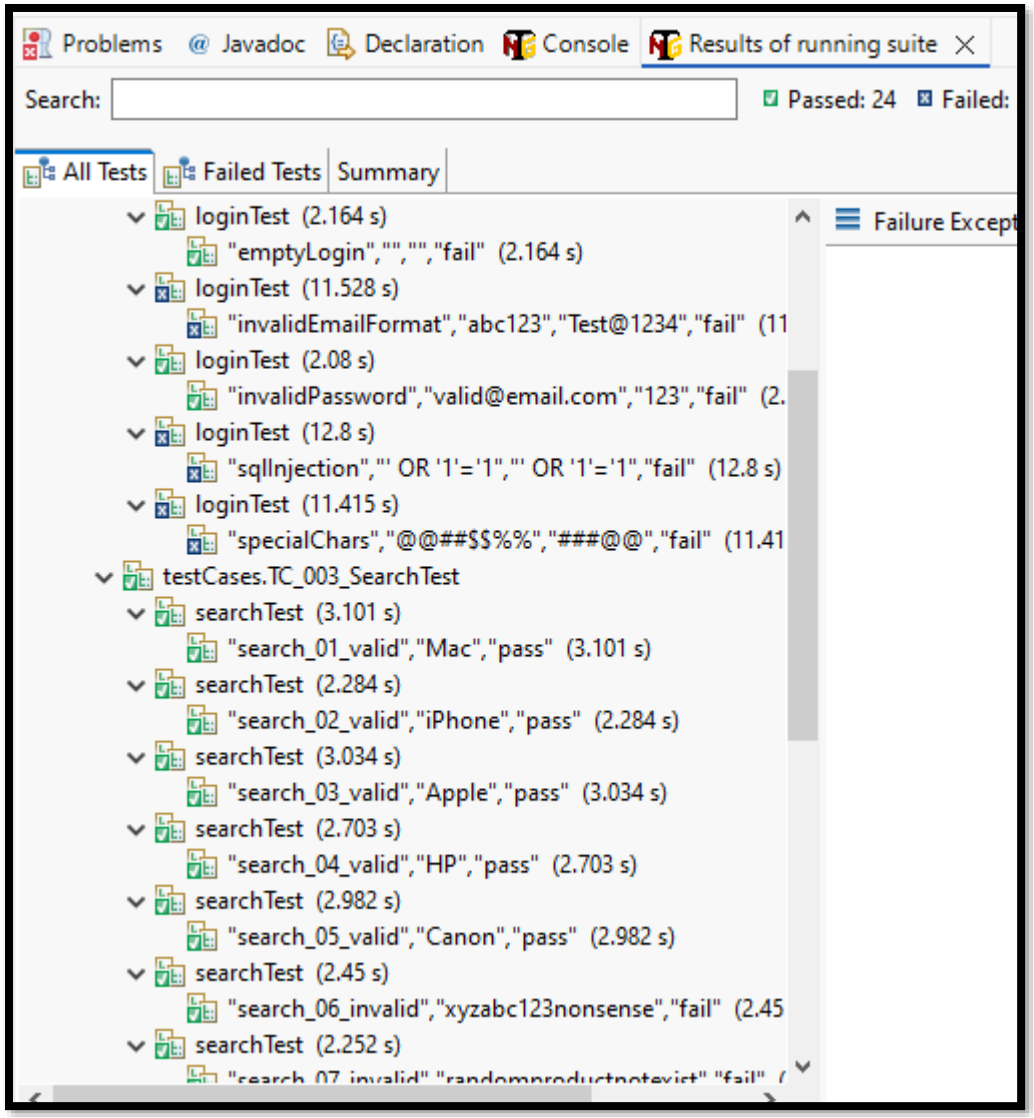





This Extent Report will be attached along with the files.

**TestNG Report**




Problems
Javadoc
Declaration
Console
Results of running suite
Search:
Passed: 24
Failed:
All Tests
Failed Tests
Summary
Failure Except
loginTest (2.164 s)
"emptyLogin","","","fail" (2.164 s)
loginTest (11.528 s)
"invalidEmailFormat","abc123","Test@1234","fail" (11
loginTest (2.08 s)
"invalidPassword","valid@email.com","123","fail" (2.
loginTest (12.8 s)
"sqlInjection","' OR '1'='1","' OR '1'='1","fail" (12.8 s)
loginTest (11.415 s)
"specialChars","@@##$$%%","###@@","fail" (11.41
testCases.TC_003_SearchTest
searchTest (3.101 s)
"search_01_valid","Mac","pass" (3.101 s)
searchTest (2.284 s)
"search_02_valid","iPhone","pass" (2.284 s)
searchTest (3.034 s)
"search_03_valid","Apple","pass" (3.034 s)
searchTest (2.703 s)
"search_04_valid","HP","pass" (2.703 s)
searchTest (2.982 s)
"search_05_valid","Canon","pass" (2.982 s)
searchTest (2.45 s)
"search_06_invalid","xyzabc123nonsense","fail" (2.45
searchTest (2.252 s)

## 4.3 Critical Components Selected for Testing

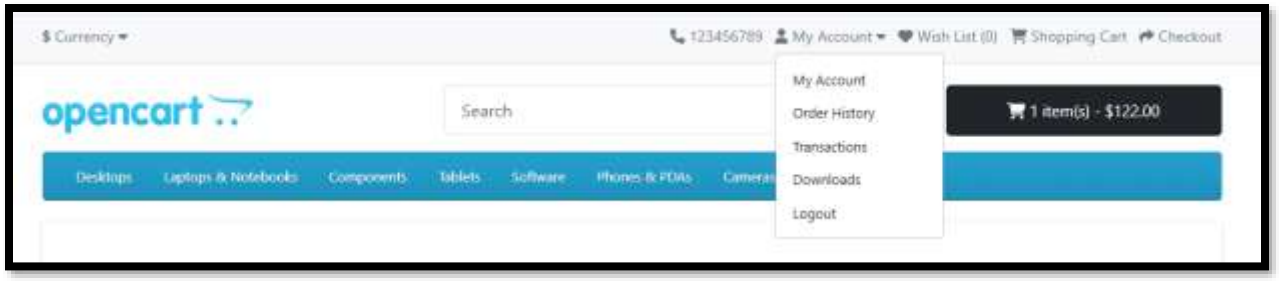
This section defines the key functional modules of the application that were selected for automation testing. These components represent the core user workflows of the OpenCart application and were prioritized based on their importance in end-to-end user interactions.

### 4.3.1 Modules Covered (Requirements / Epics)

The following modules were identified and tested:

- **Registration Module** User account creation functionality
- **Login Module** Authentication and user login
- **Search Module** Product search and filtering
- **Product Module** Product selection and details view
- **Cart Module** Add, view, and remove products from cart
- **My Account Module** User profile and account management

These modules represent the main functional requirements of the application and ensure coverage of critical user flows.




### 4.3.2 End-to-End Functional Flow

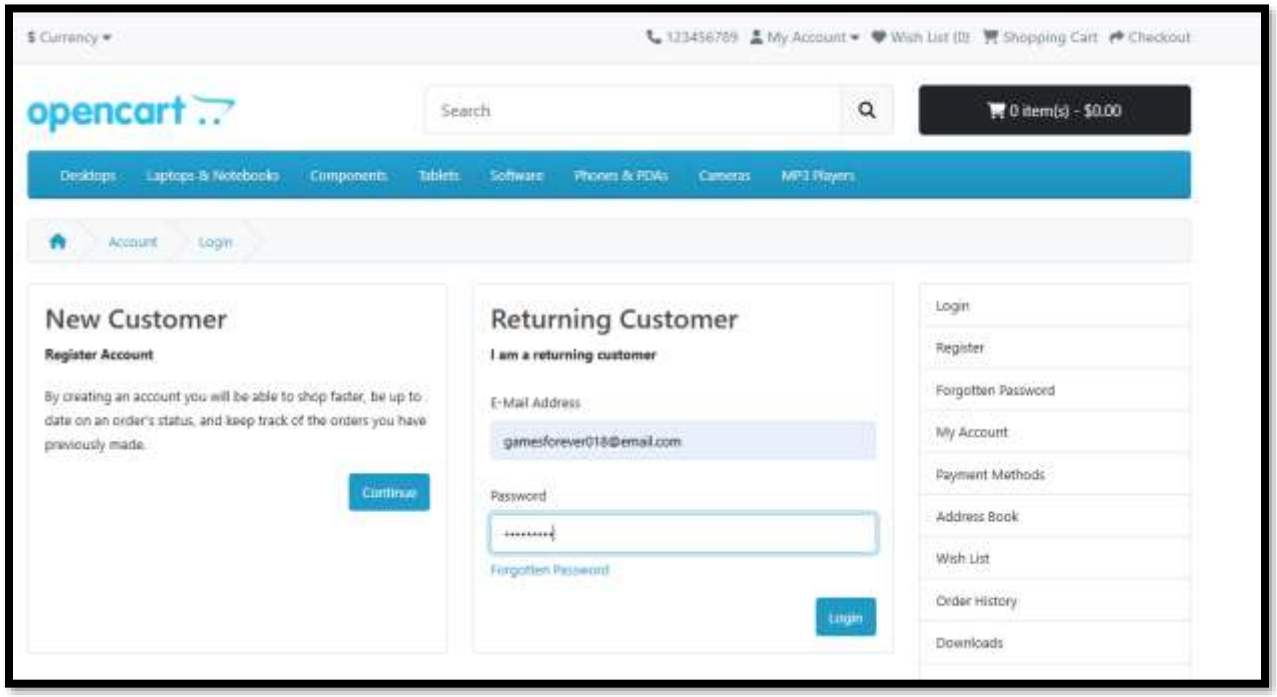
The automation tests were designed to simulate real user behavior in the following sequence:

1. User registers or logs in to the application
2. User searches for a product (e.g., “Mac”)
3. User selects a product from search results

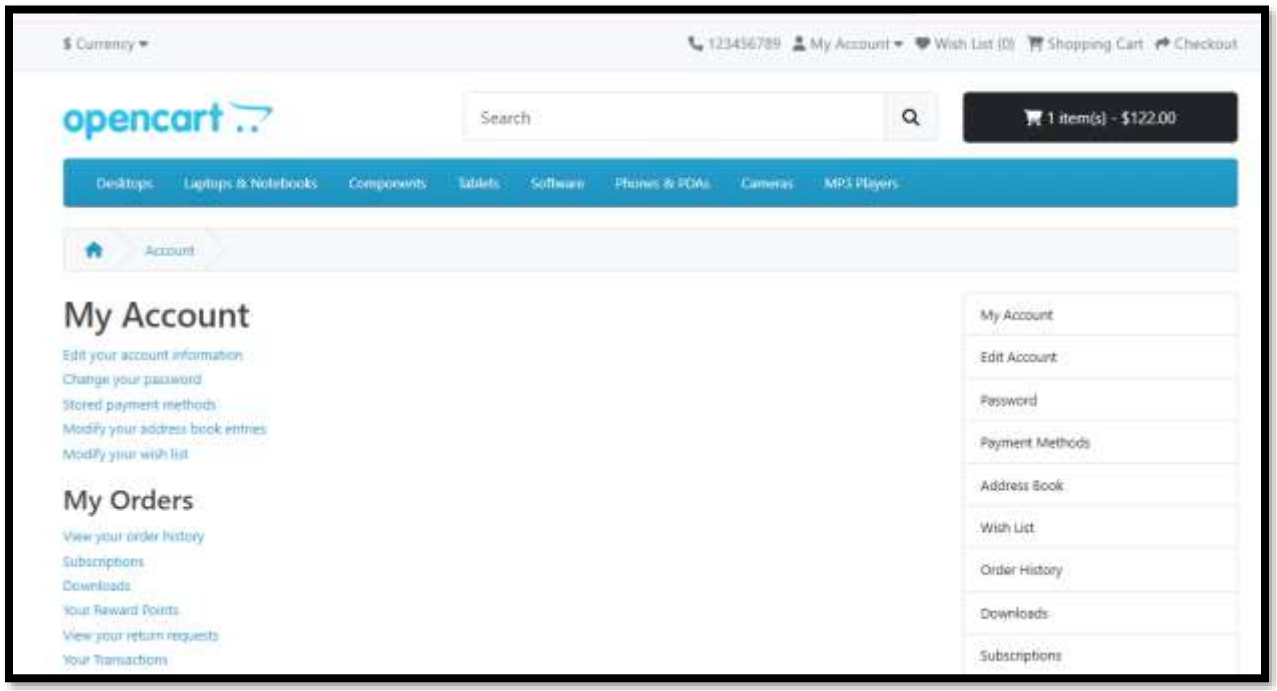
4. Product is added to the shopping cart
5. User views cart and verifies product details
6. User removes product from cart (if required)

This flow ensures that complete user journeys are validated rather than isolated functions.

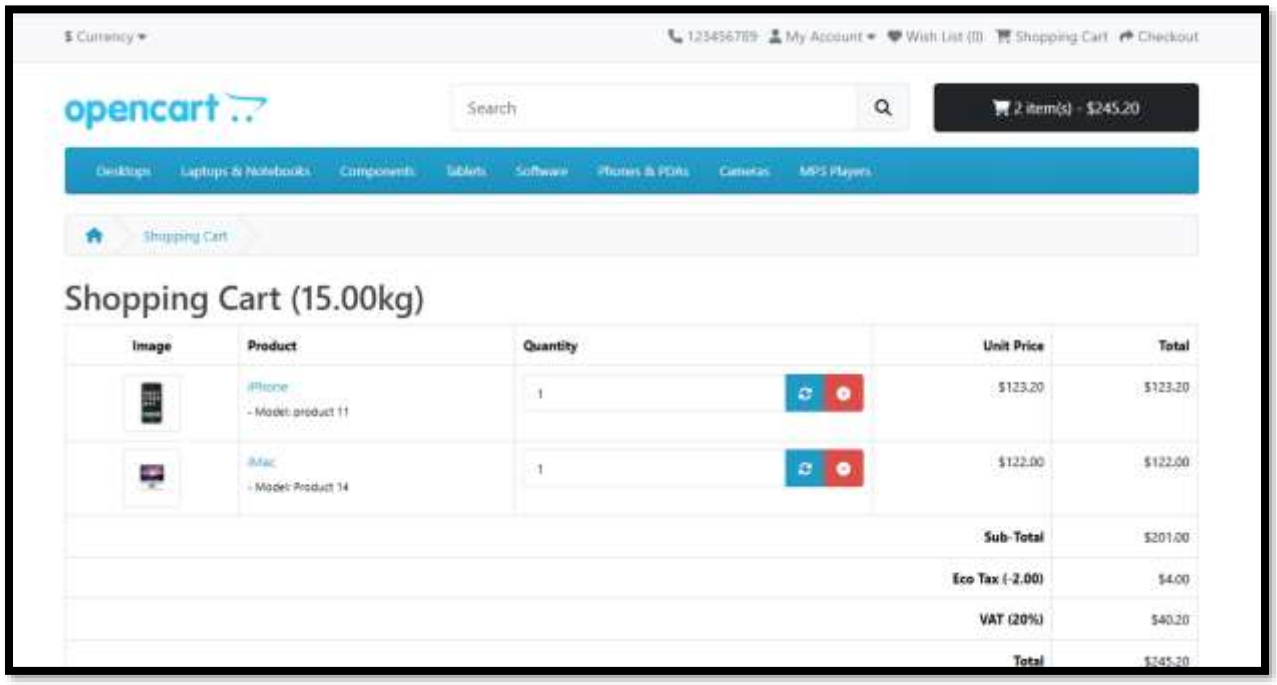
**Login**



## My Account




## Cart

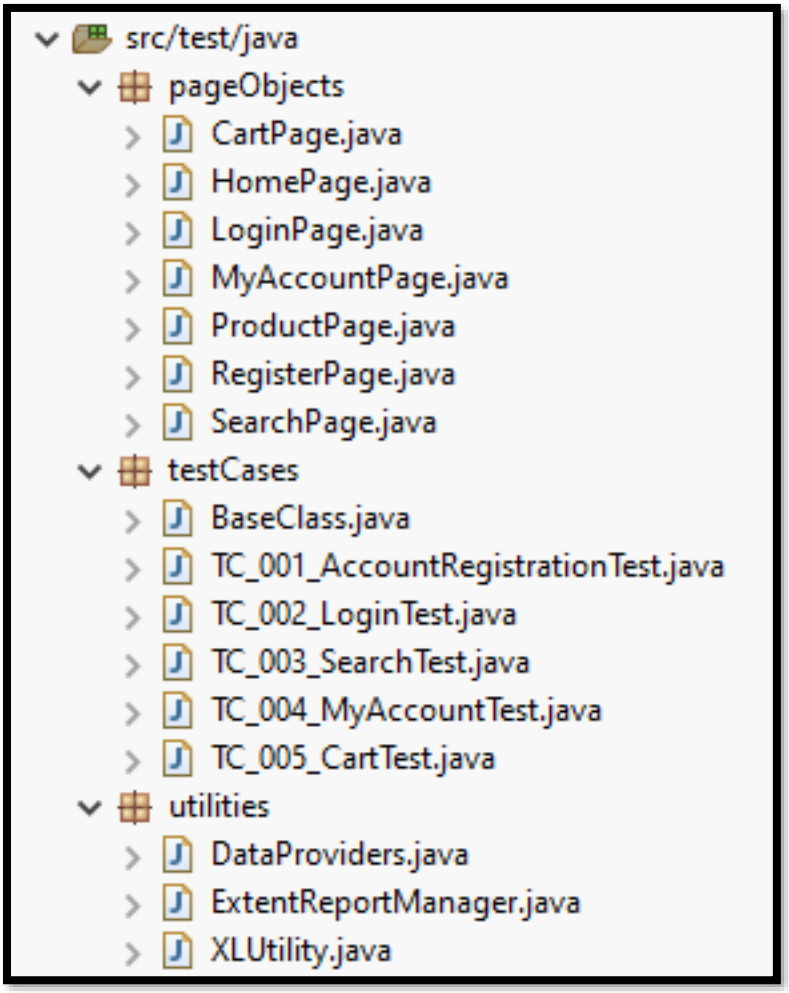


#### 4.3.3 Test Coverage Mapping

Each module is mapped to automated test cases as follows:

- Registration → TC_001_AccountRegistrationTest
- Login → TC_002_LoginTest
- Search → TC_003_SearchTest
- My Account → TC_004_MyAccountTest
- Cart → TC_005_CartTest

This mapping ensures traceability between requirements and test cases.



### 4.4 Page Object Model (POM) Implementation

The project follows the Page Object Model (POM) design pattern to improve code maintainability, reusability, and readability. In this approach, each web page of the application is represented as a separate Java class, and all web elements and actions are defined inside that class.

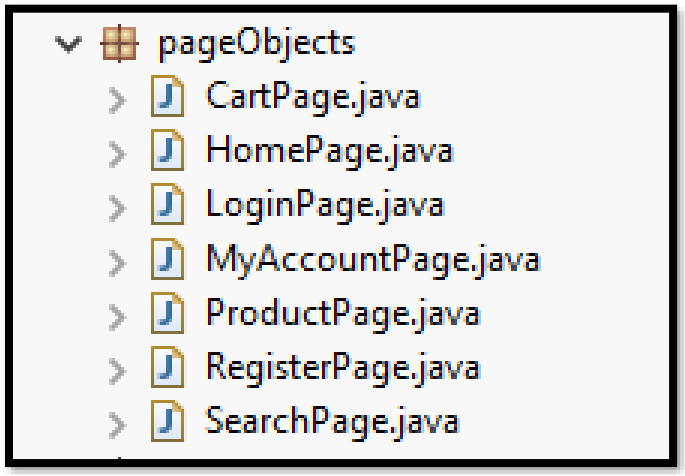
This helps in separating test logic from UI locators, making the framework easier to manage and scale.

#### 4.4.1 POM Architecture Approach

The framework is structured in such a way that:

- Each page of the application has a dedicated class
- Web elements are defined using @FindBy annotations
- Actions (methods) are written inside page classes
- Test cases only call methods from page objects

This ensures clean separation between test logic and page structure.



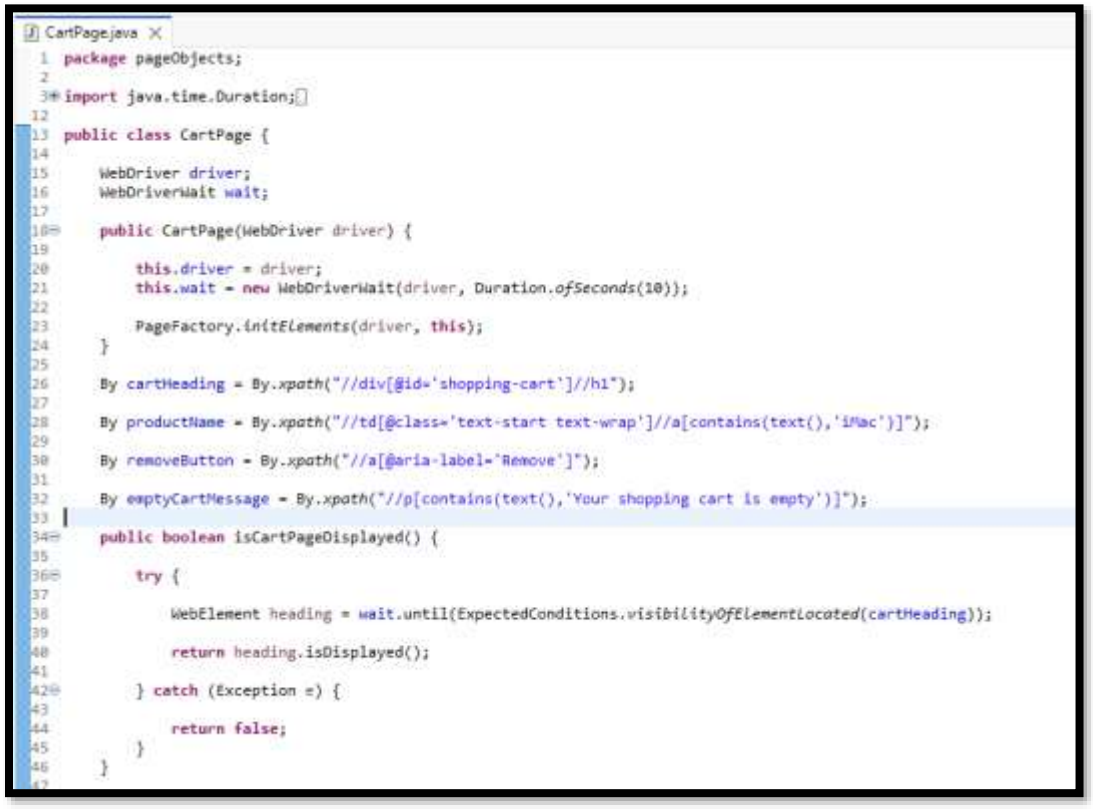
### 4.4.2 Page Classes Overview

The following Page Object classes were implemented:

- **HomePage.java** Handles search and navigation
- **LoginPage.java** Handles user authentication
- **RegisterPage.java** Handles user registration
- **SearchPage.java** Handles product search results
- **ProductPage.java** Handles product actions (add to cart)
- **CartPage.java** Handles cart validation and removal
- **MyAccountPage.java** Handles user account features

### 4.4.3 Sample Code Snippet (POM Example)

**CartPage.java**



```java
package pageObjects;

import java.time.Duration;

public class CartPage {

    WebDriver driver;
    WebDriverWait wait;

    public CartPage(WebDriver driver) {

        this.driver = driver;
        this.wait = new WebDriverWait(driver, Duration.ofSeconds(10));

        PageFactory.initElements(driver, this);
    }

    By cartHeading = By.xpath("//div[@id='shopping-cart']//h1");

    By productName = By.xpath("//td[@class='text-start text-wrap']//a[contains(text(),'iMac')]");

    By removeButton = By.xpath("//a[@aria-label='Remove']");

    By emptyCartMessage = By.xpath("//p[contains(text(),'Your shopping cart is empty')]");

    public boolean isCartPageDisplayed() {

        try {

            WebElement heading = wait.until(ExpectedConditions.visibilityOfElementLocated(cartHeading));

            return heading.isDisplayed();

        } catch (Exception e) {

            return false;
        }
    }
```

```
48    public boolean isProductDisplayed() {
49
50        try {
51
52            WebElement product = wait.until(ExpectedConditions.visibilityOfElementLocated(productName));
53
54            return product.isDisplayed();
55
56        } catch (Exception e) {
57
58            return false;
59        }
60    }
61
62    public void removeFromCart() {
63
64        WebElement removeBtn = wait.until(ExpectedConditions.elementToBeClickable(removeButton));
65
66        ((JavascriptExecutor) driver).executeScript("arguments[0].scrollIntoView({block:'center'});", removeBtn);
67
68        ((JavascriptExecutor) driver).executeScript("arguments[0].click();", removeBtn);
69
70        // wait until cart updates
71        wait.until(ExpectedConditions.or(ExpectedConditions.visibilityOfElementLocated(emptyCartMessage),
72                ExpectedConditions.invisibilityOfElementLocated(productName)));
73    }
74
75    public boolean isCartEmpty() {
76
77        try {
78
79            WebElement emptyMsg = wait.until(ExpectedConditions.visibilityOfElementLocated(emptyCartMessage));
80
81            return emptyMsg.isDisplayed();
82
83        } catch (Exception e) {
84
85            return false;
86        }
```

```
87    }
88 }
```

All the files are present in the zip file. All of them follow the same pattern.

### 4.4.4 POM Evidence Screenshots

- Page Object classes structure
- CartPage / LoginPage code view
- Test class calling POM methods

**Login Page**

LoginPage.java

```java
package pageObjects;

import org.openqa.selenium.WebDriver;

public class LoginPage {

    WebDriver driver;

    @FindBy(xpath = "//input[@id='input-email']")
    WebElement emailField;

    @FindBy(xpath = "//input[@id='input-password']")
    WebElement passwordField;

    @FindBy(xpath = "//button[normalize-space()='Login']")
    WebElement loginButton;

    @FindBy(xpath = "//div[@class='alert alert-danger alert-dismissible']")
    WebElement warningMessage;

    @FindBy(linkText = "Forgotten Password")
    WebElement forgotPasswordLink;

    @FindBy(linkText = "Continue")
    WebElement continueButton; // for registration redirect

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    public void enterEmail(String email) {
        emailField.clear();
        emailField.sendKeys(email);
    }

    public void enterPassword(String password) {
        passwordField.clear();
        passwordField.sendKeys(password);
```

```
42          passwordField.sendKeys(password);
43      }
44
45      public void clickLogin() {
46          loginButton.click();
47      }
48
49      // returns warning message text for assertion
50      public String getWarningMessage() {
51          return warningMessage.getText();
52      }
53
54      public boolean isWarningDisplayed() {
55          return warningMessage.isDisplayed();
56      }
57
58      // combined login action
59      public MyAccountPage loginAs(String email, String password) {
60          enterEmail(email);
61          enterPassword(password);
62          clickLogin();
63          return new MyAccountPage(driver);
64      }
65
66      public void clickForgotPassword() {
67          forgotPasswordLink.click();
68      }
69  }
```

## Login Page Test

TC_002_LoginTest.java

```java
package testCases;

import org.testng.Assert;

Run All
public class TC_002_LoginTest extends BaseClass {

    @Test(dataProvider = "LoginData", dataProviderClass = DataProviders.class, groups = { "smoke", "regression" })
    Run | Debug
    public void loginTest(String testCase, String email, String password, String expected) {

        logInfo("Starting login test: " + testCase);

        logInfo("Navigating to Login page");
        HomePage homePage = new HomePage(driver);
        homePage.navigateToLogin();

        logInfo("Entering credentials");
        LoginPage loginPage = new LoginPage(driver);
        loginPage.enterEmail(email);
        loginPage.enterPassword(password);

        logInfo("Clicking Login button");
        loginPage.clickLogin();

        boolean result;

        // VALID LOGIN CASE
        if (expected.equalsIgnoreCase("pass")) {

            logInfo("Valid login validation started");

            MyAccountPage myAccountPage = new MyAccountPage(driver);
            result = myAccountPage.isMyAccountPageDisplayed();

            Assert.assertTrue(result, "My Account page should be displayed");

            // logout
            myAccountPage.clickLogout();
```

## 4.5 Test Case Design (Excel-Based Documentation)

Test cases were designed and documented in an Excel sheet to ensure structured test planning and clear coverage of all functional scenarios. The test cases include both positive and negative scenarios to validate system behavior under different conditions.

Excel-based test design also supports data-driven testing, where multiple input values are executed using the same test script.

### 4.5.1 Test Case Writing Approach

The following approach was used to design test cases:

- Each module was analyzed (Login, Search, Cart, etc.)
- Test scenarios were written based on functional requirements
- Expected results were defined for validation
- Test cases were assigned unique IDs for tracking

#### 4.5.2 Positive Test Cases

Positive test cases verify that the application works as expected under valid inputs.

Examples include:

- Valid user registration
- Valid login with correct credentials
- Searching a valid product (e.g., “Mac”)
- Adding product to cart successfully

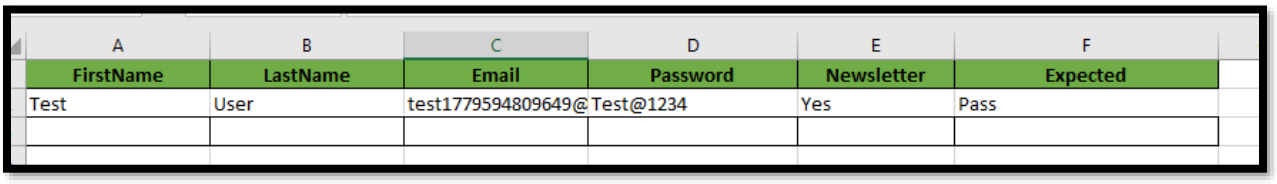
**Search Data**

| TestCase | SearchText | Expected |
|---|---|---|
| search_01_valid | Mac | pass |
| search_02_valid | iPhone | pass |
| search_03_valid | Apple | pass |
| search_04_valid | HP | pass |
| search_05_valid | Canon | pass |
| search_06_invalid | xyzabc123nonsense | fail |
| search_07_invalid | randomproductnotexist | fail |
| search_08_negative | !!@@##$$ | fail |
| search_09_negative | abcdefgh123456789 | fail |
| search_10_negative | qwertyuiop | fail |

**Login Data**

| TestCase | Email | Password | Expected |
|---|---|---|---|
| validLogin | test1779594809649@gr | Test@1234 | Pass |
| invalidLogin | wrong@email.com | wrong123 | fail |
| emptyLogin | | | fail |
| invalidEmailFormat | abc123 | Test@1234 | fail |
| invalidPassword | valid@email.com | 123 | fail |
| sqlInjection | ' OR '1'='1 | ' OR '1'='1 | fail |
| specialChars | @@##$$%% | ###@@ | fail |

**Register Data**



| | A | B | C | D | E | F |
|---|---|---|---|---|---|---|
| | FirstName | LastName | Email | Password | Newsletter | Expected |
| | Test | User | test1779594809649@ | Test@1234 | Yes | Pass |
| | | | | | | |
| | | | | | | |

The register data is created randomly. In the code.

### 4.5.3 Negative Test Cases

Negative test cases validate system behavior under invalid inputs or error conditions.

Examples include:

- Login with invalid credentials
- Searching for non-existing product
- Removing item from empty cart
- Submitting empty form fields

### 4.5.4 Data-Driven Testing Implementation

Data-driven testing was implemented using Excel files where multiple sets of test data were provided. The test scripts read data from Excel using Apache POI and executed tests dynamically.

This approach improved test coverage and reduced code duplication.

```
@DataProvider(name = "LoginData")
public Object[][] getLoginData() {

    String path = System.getProperty("user.dir") + "\\testData\\TestData.xlsx";
    XLUtility xl = new XLUtility(path);

    Object[][] data = null;

    try {
        int rowCount = xl.getRowCount("LoginData"); // total rows (0-based)
        int colCount = xl.getCellCount("LoginData", 1); // columns in first data row

        data = new Object[rowCount][colCount];

        for (int i = 1; i <= rowCount; i++) { // start from row 1 (skip header)
            for (int j = 0; j < colCount; j++) {
                data[i - 1][j] = xl.getCellData("LoginData", i, j);
            }
        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    return data;
}
```

## 4.6 Test Execution Using TestNG

TestNG was used as the primary test execution framework to manage and execute automated test cases. It allows structured execution of test suites, grouping of test cases, and generation of detailed execution results.

All test cases were executed through a centralized testng.xml suite file.

### 4.6.1 TestNG Suite Structure (testng.xml)

The test execution is controlled using a TestNG XML file which defines all test classes in a structured manner.

The suite includes:

- Account Registration Test
- Login Test
- Search Test
- My Account Test
- Cart Test

**TestNG.xml**

testng.xml

```
http://testng.org/testng-1.9.8.dtd (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">

<suite name="OpenCart Test Suite">

    <listeners>
        <listener class-name="utilities.ExtentReportManager"/>
    </listeners>

    <test name="OpenCart Regression Tests">

        <parameter name="browser" value="chrome"/>

        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
         <class name="testCases.TC_001_AccountRegistrationTest"/>
            <class name="testCases.TC_002_LoginTest"/>
            <class name="testCases.TC_003_SearchTest"/>
            <class name="testCases.TC_004_MyAccountTest"/>
            <class name="testCases.TC_005_CartTest"/>
        </classes>

    </test>

</suite>
```

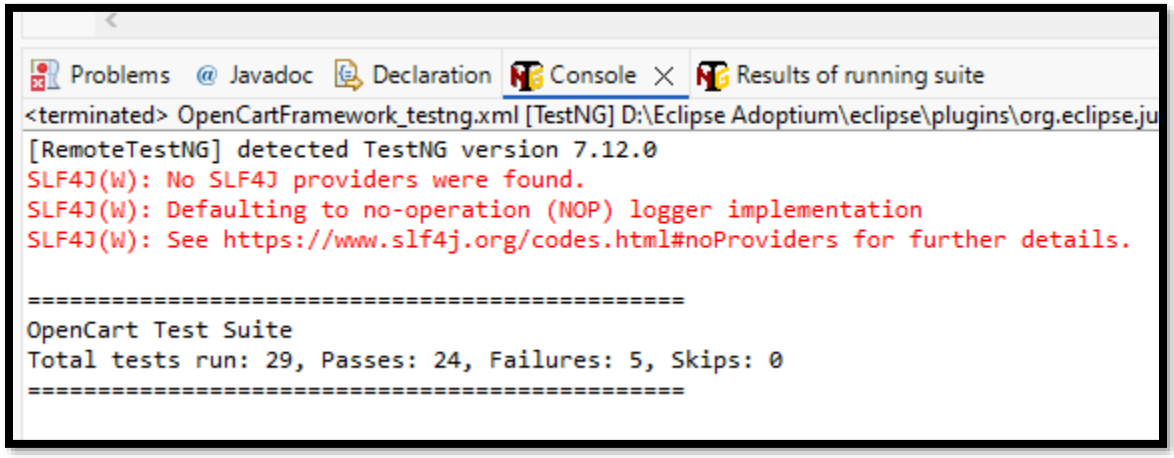
There are no errors. Its just the Eclipse IDE sometimes show red lines without any error. Especially in the .xml files.

### 4.6.2 Execution Flow of Test Cases

The execution flow followed a modular approach:

1. Browser initialization (BaseClass)
2. Execution of test modules sequentially
3. Logging of each step
4. Capture of screenshots on failure/success
5. Generation of reports after execution

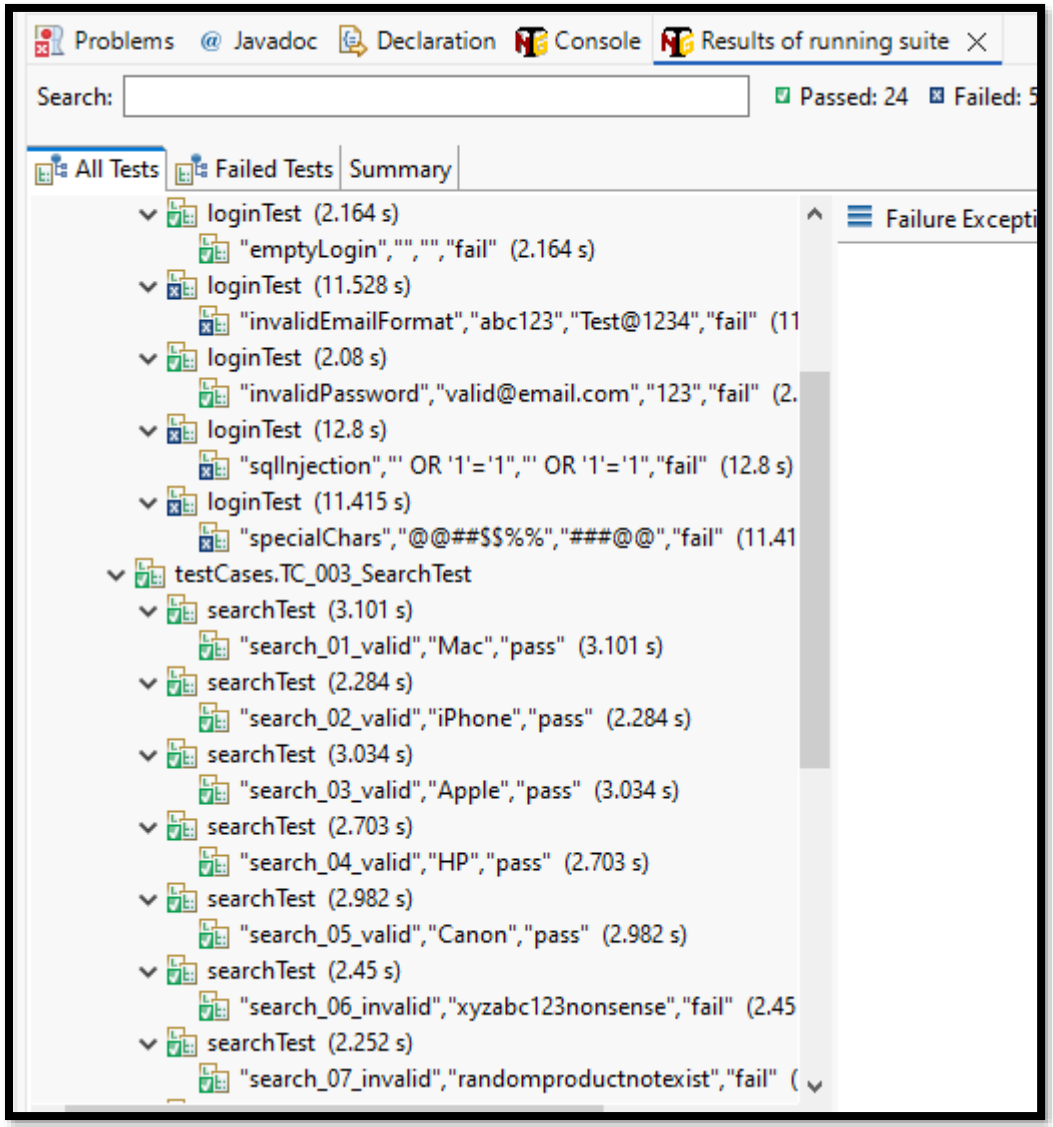
This ensures consistent execution across all test scenarios.



### 4.6.3 Test Execution Process

During execution:

- TestNG runs all test cases defined in suite
- Assertions validate expected vs actual results
- Failures are captured with logs and screenshots
- Results are automatically passed to reporting tools



### 4.6.4 Test Environment Details

- Browser: Google Chrome
- Automation Tool: Selenium WebDriver
- Framework: TestNG
- Language: Java
- OS: Windows 10

Tags

| NAME | PASSED | FAILED | SKIPPED | OTHERS | PASSED % |
|---|---|---|---|---|---|
| regression | 24 | 5 | 0 | 0 | 82.759% |
| smoke | 17 | 3 | 0 | 0 | 85% |

System/Environment

| NAME | VALUE |
|---|---|
| Application | OpenCart |
| Tester | GhostFreak |
| OS | Windows 10 |
| Environment | QA |
| Browser | Chrome |

### 4.7 Test Execution Results

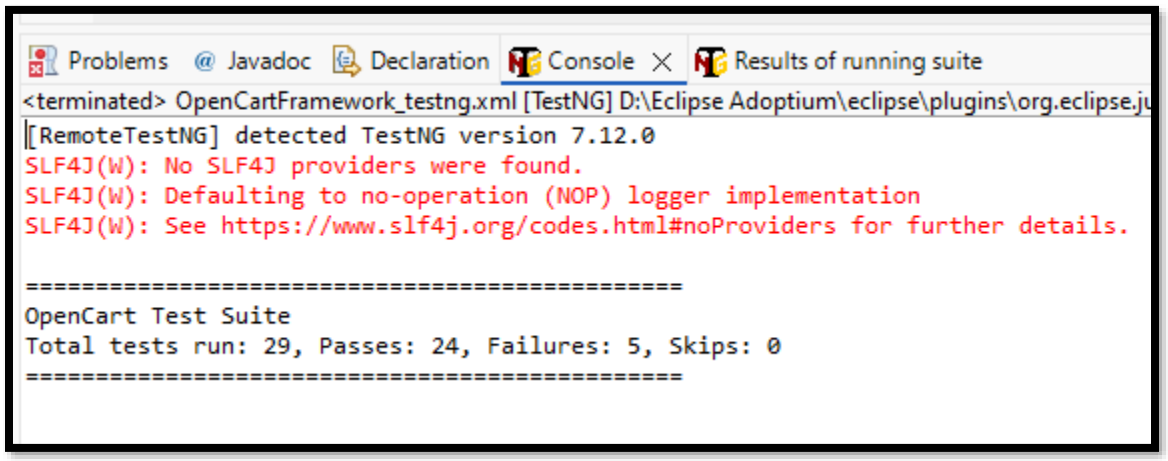
This section presents the final execution results of the automated test suite. The results include passed and failed test cases along with logs and debugging evidence generated during execution.

#### 4.7.1 Summary of Test Execution

The automation suite was executed successfully using TestNG. The results show a mix of passed and failed test cases due to dynamic UI behavior and synchronization issues in some scenarios.

Overall execution status:

- Total Test Cases Executed: *[add your number]*
- Passed Test Cases: *[add number]*
- Failed Test Cases: *[add number]*
- Skipped Test Cases: *[if any]*

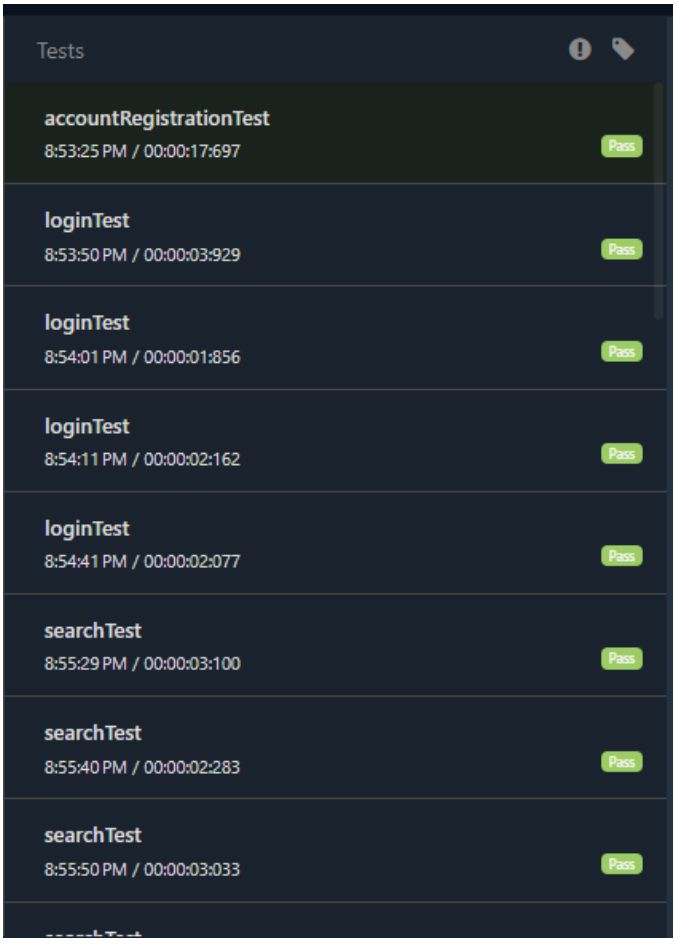


### 4.7.2 Passed Test Cases

The following test modules executed successfully:

- Account Registration Test
- Login Test (multiple datasets)
- Search Functionality Test
- Add Product to Cart Test
- My Account Navigation Test

These results confirm that the core functionalities of the application are working as expected.



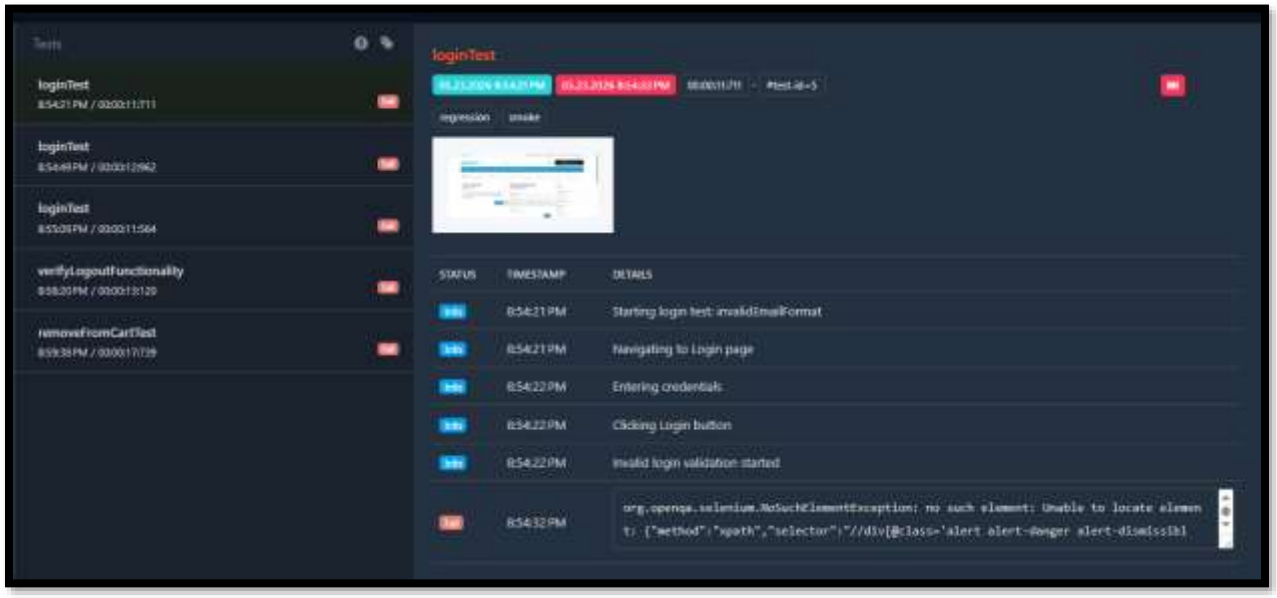
### 4.7.3 Failed Test Cases Analysis

Some test cases failed due to dynamic element loading and locator issues.

Examples:

- Cart product visibility issue (NoSuchElementException)
- Element click interception (ElementClickInterceptedException)
- Timing/synchronization delays in UI rendering

These issues indicate the need for improved explicit waits and more stable locators in the automation framework.

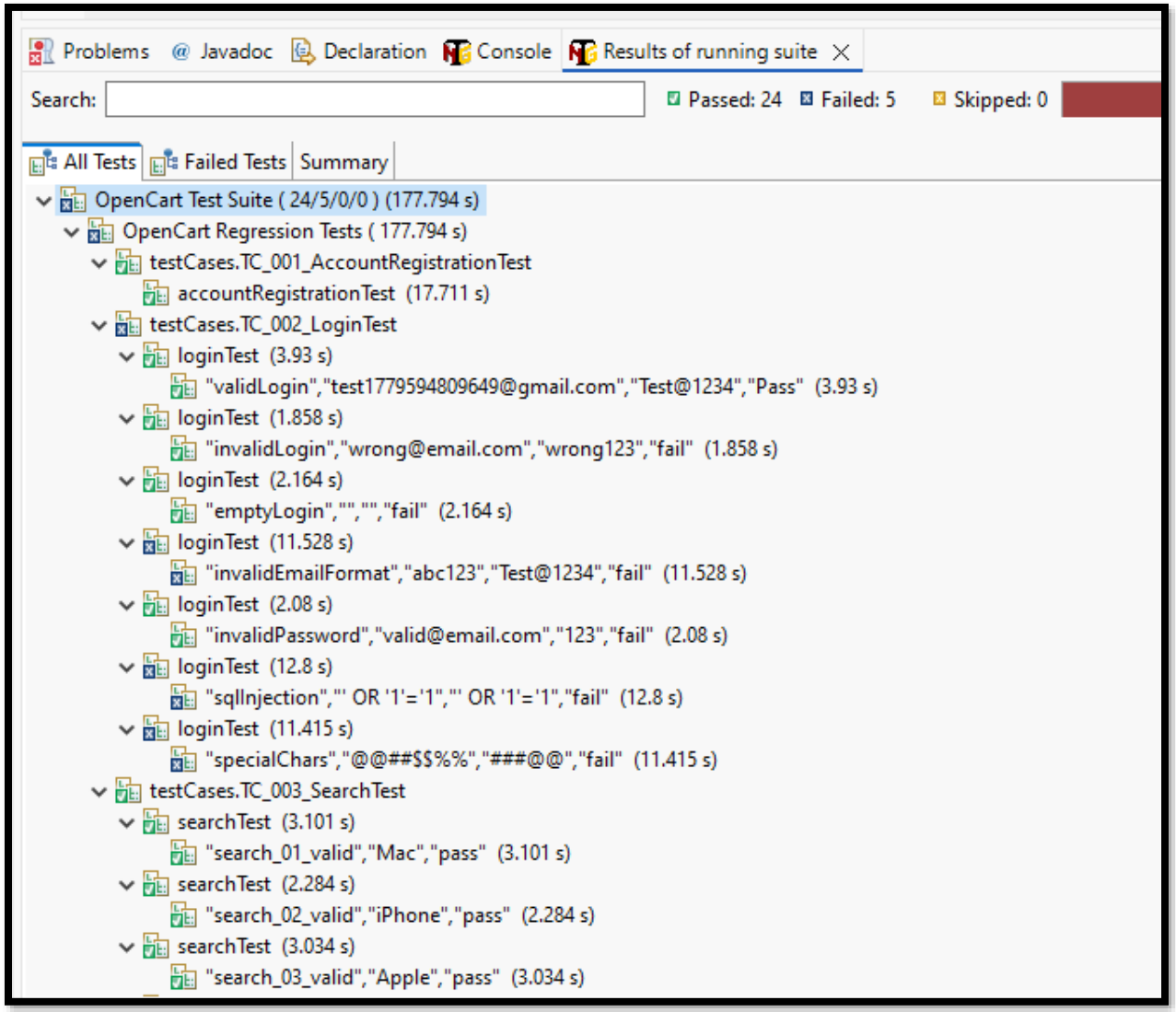


### 4.7.4 Execution Logs

Execution logs were generated for debugging and tracking test flow.

Logs include:

- Step-by-step execution details
- Assertion results
- Error stack traces for failed cases



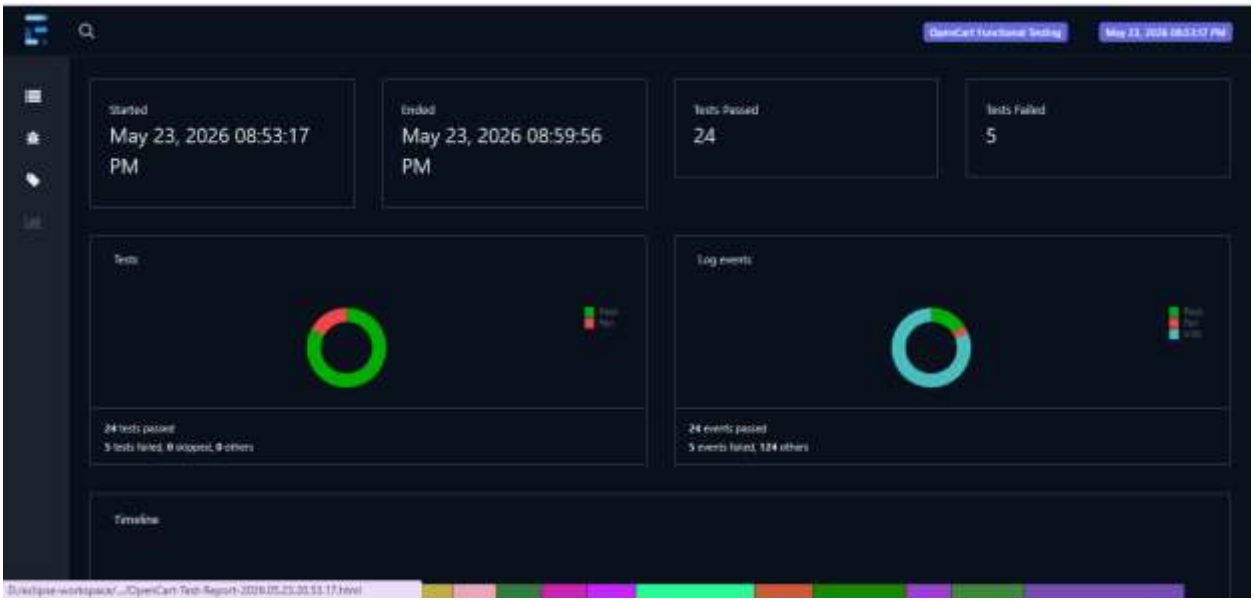
## 4.8 Automation Reports (Extent Reports)

Extent Reports were used to generate detailed HTML-based test execution reports. These reports provide a visual representation of test results, including pass/fail status, execution time, and step-wise logs.

### 4.8.1 Extent Report Overview

The Extent Report provides:

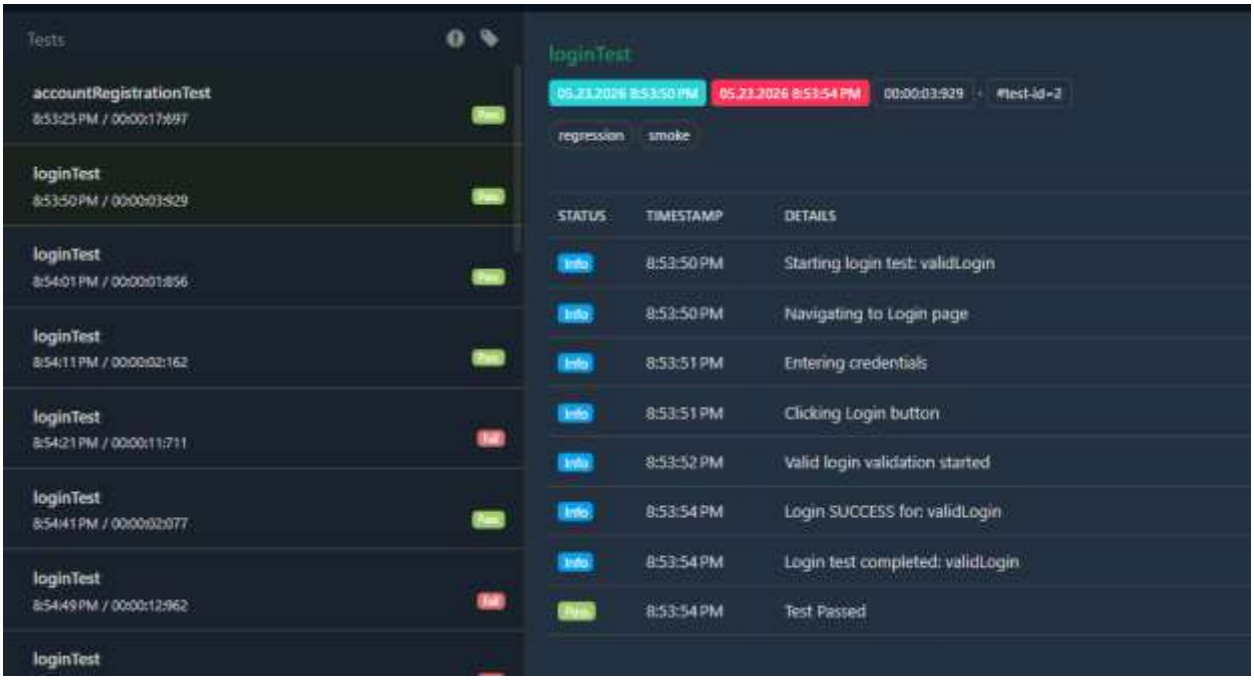
- Test case status (Pass/Fail/Skip)
- Execution time tracking
- Step-by-step logging
- Screenshots for failed test cases



### 4.8.2 Detailed Test Report View

Each test case is displayed individually with:

- Test steps
- Status (Pass/Fail)
- Error messages (if any)
- Attached screenshots



### 4.8.3 Report Benefits

Extent Reports improve test visibility by:

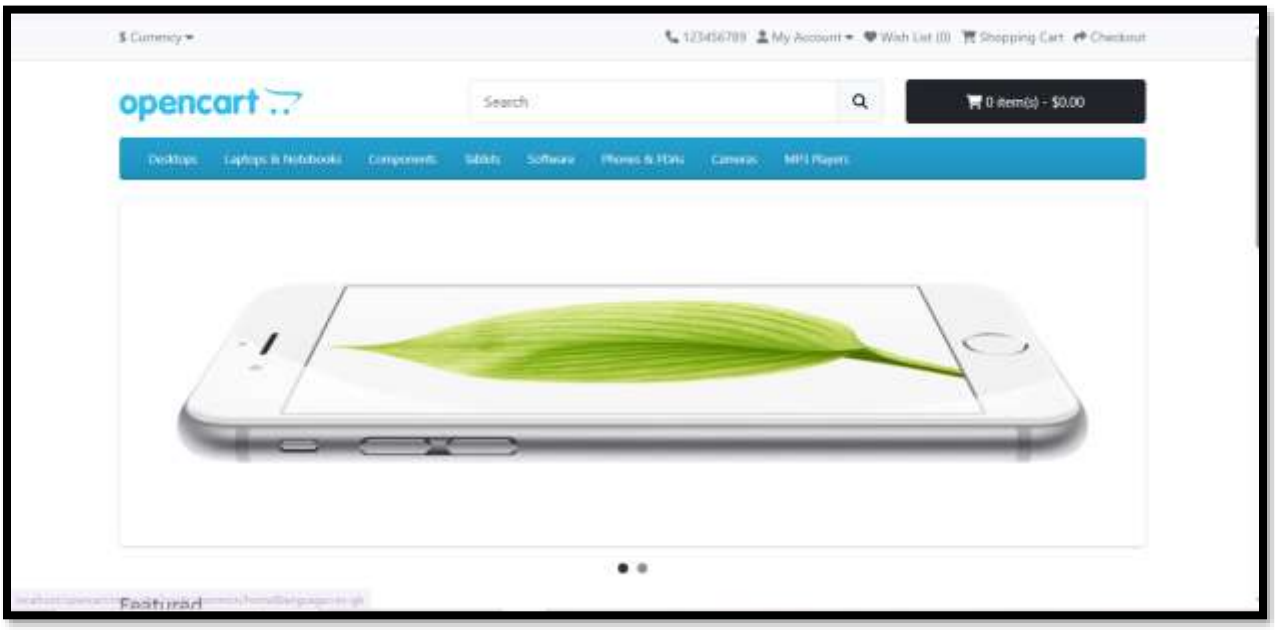
- Providing structured HTML output
- Making debugging easier
- Presenting results in a professional format

## 4.9 UI Screenshots of Application

This section includes the graphical user interface (UI) screenshots of the application under test. These screenshots demonstrate the actual working of the system and validate that the automated scripts are interacting with the correct application screens.

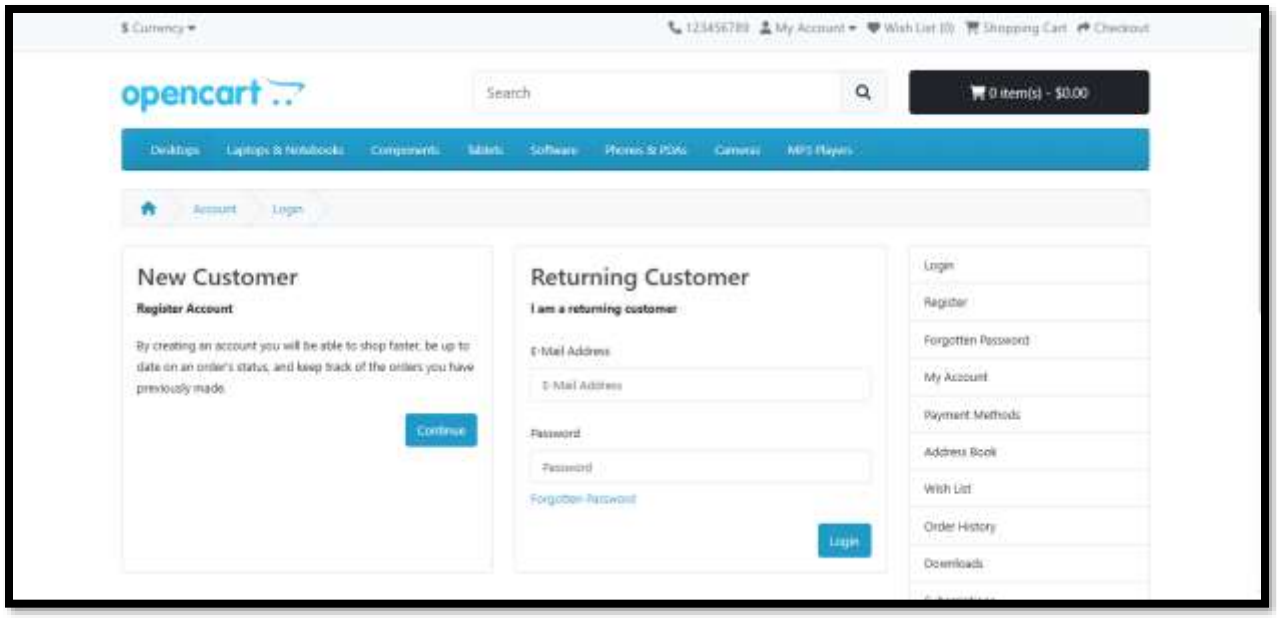
### 4.9.1 Home Page

The home page displays the main navigation menu, product categories, and search functionality of the application.



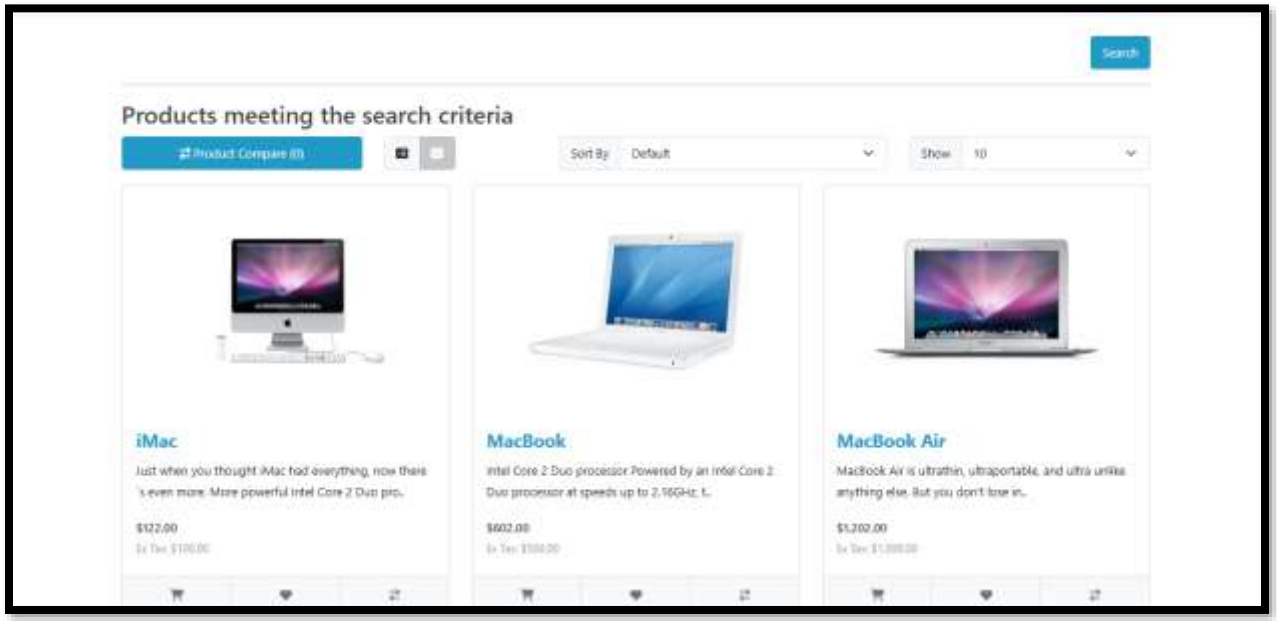
### 4.9.2 Login Page

The login page allows registered users to authenticate using valid credentials.



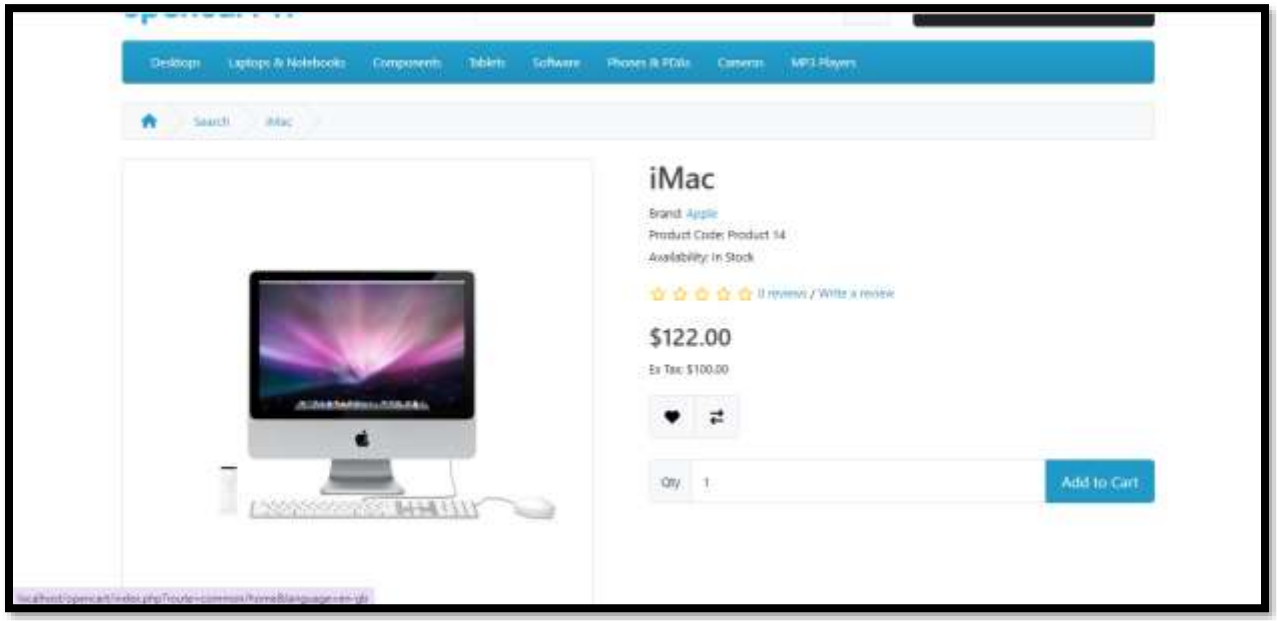

### 4.9.3 Search Results Page

This page shows the results returned after searching for a product (e.g., “Mac”).



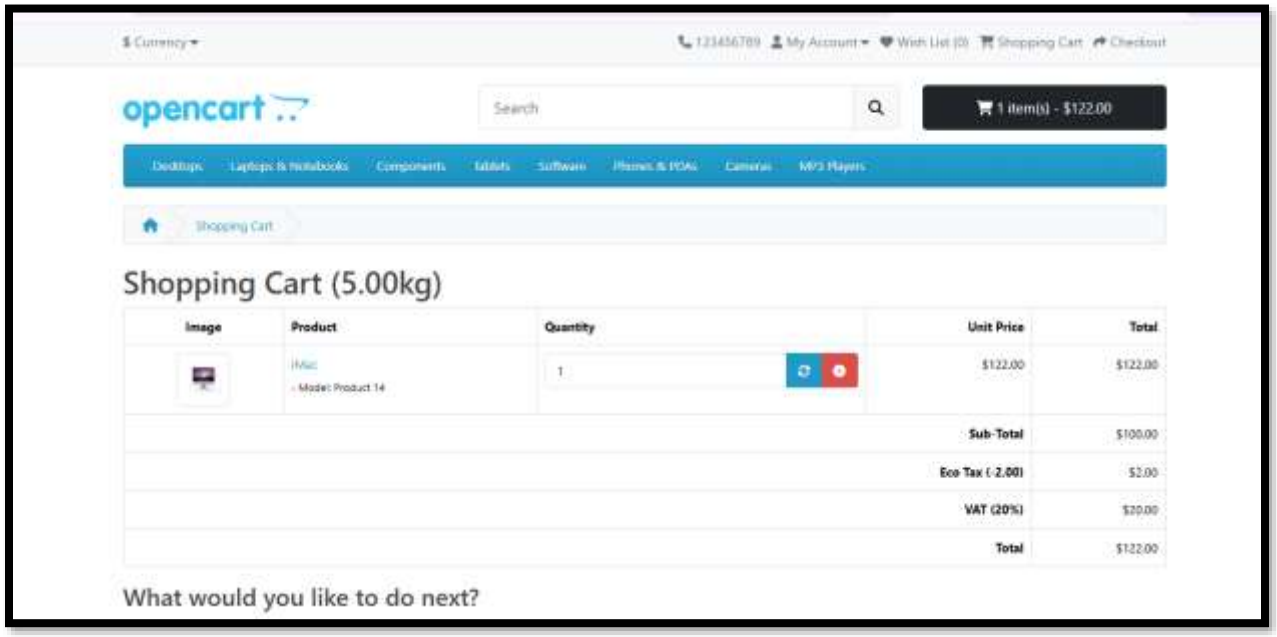
### 4.9.4 Product Details Page

This page displays detailed information about a selected product, including price and add-to-cart option.




### 4.9.5 Shopping Cart Page

The cart page shows products added by the user and allows updating or removing items.



### 4.9.6 UI Interaction Flow

The complete flow of UI interaction during testing:

Home → Search → Product Selection → Add to Cart → Cart View → Remove Product

### 4.10 Performance Testing (Lighthouse Report)

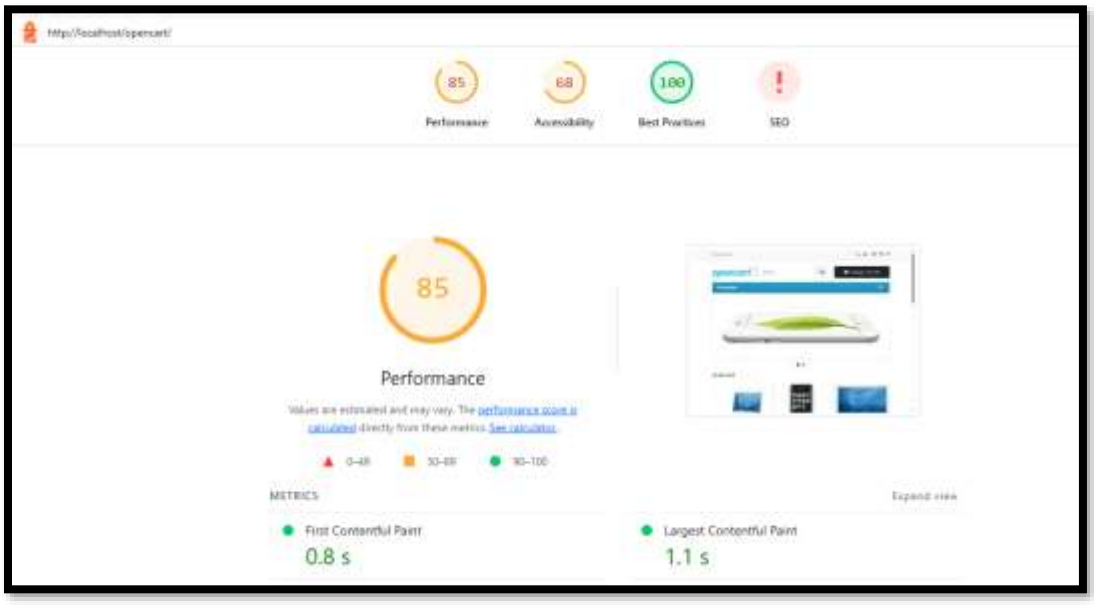
A basic performance analysis of the application was performed using Google Lighthouse. This tool evaluates the web application in terms of performance, accessibility, best practices, and SEO.

The purpose of this testing was to ensure that the application is not only functionally correct but also performs efficiently from a user experience perspective.

### 4.10.1 Lighthouse Testing Overview

Lighthouse was executed on the main pages of the application to analyze overall performance metrics such as:

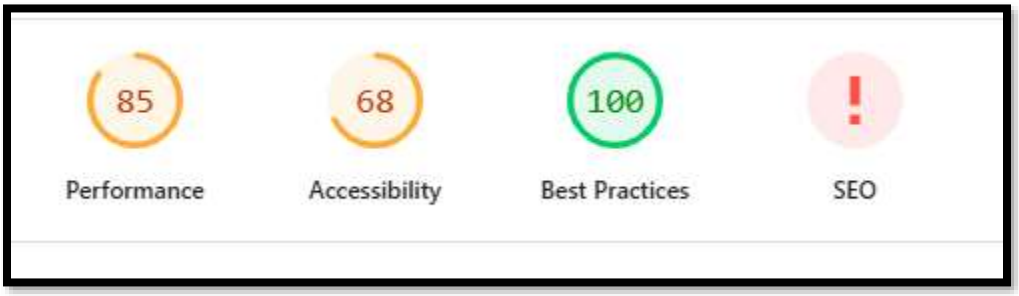
- Performance score
- Accessibility score
- Best practices score
- SEO score



### 4.10.2 Performance Results

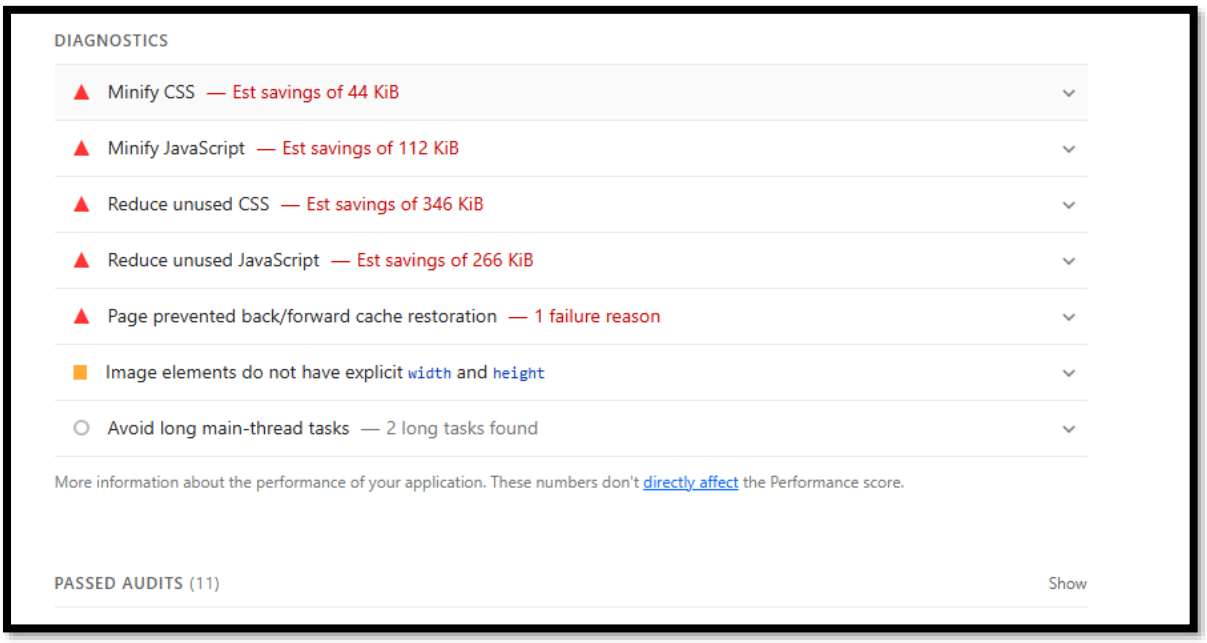
The generated report provided the following insights:

- The application shows acceptable loading performance under normal conditions
- UI rendering is stable and responsive
- Some minor improvements can be made in optimization of assets and scripts




### 4.10.3 Key Observations

- Pages load within acceptable time range for local environment testing
- No major blocking performance issues observed
- Further optimization can improve performance in production environment



### 4.10.4 Conclusion of Performance Testing

Overall, the application performs well for functional testing purposes. Lighthouse results confirm that the UI is responsive and usable, with only minor optimization recommendations.

### 4.11 Challenges Faced

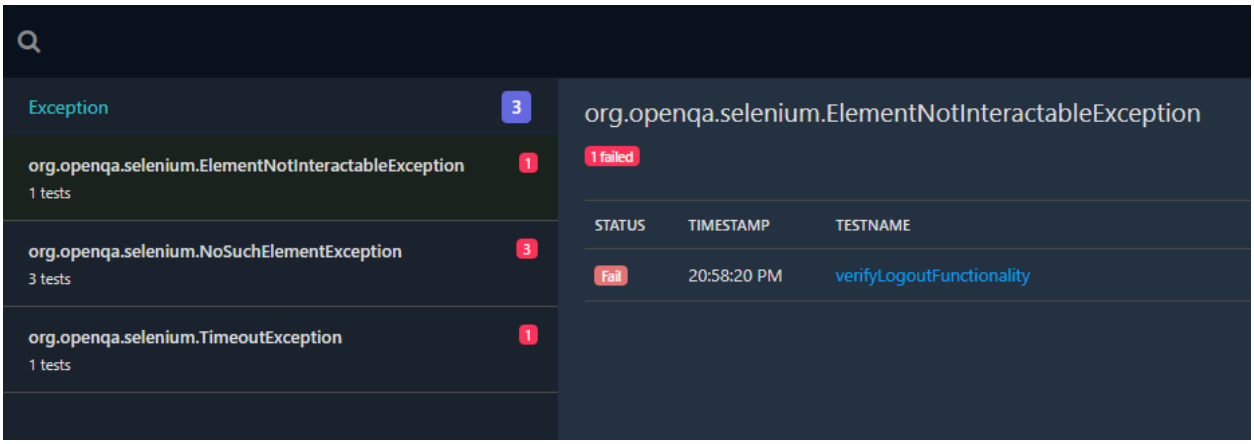
During the automation testing process, several technical and framework-related challenges were encountered. These issues were resolved using debugging, waits, and framework improvements.

#### 4.11.1 Element Interaction Issues

One of the major challenges was handling dynamic web elements.

- ElementClickInterceptedException occurred when elements were overlapped by UI components
- Some elements were not clickable due to page rendering delays

**Solution:**
Explicit waits (WebDriverWait) and improved XPath locators were used to resolve timing issues.

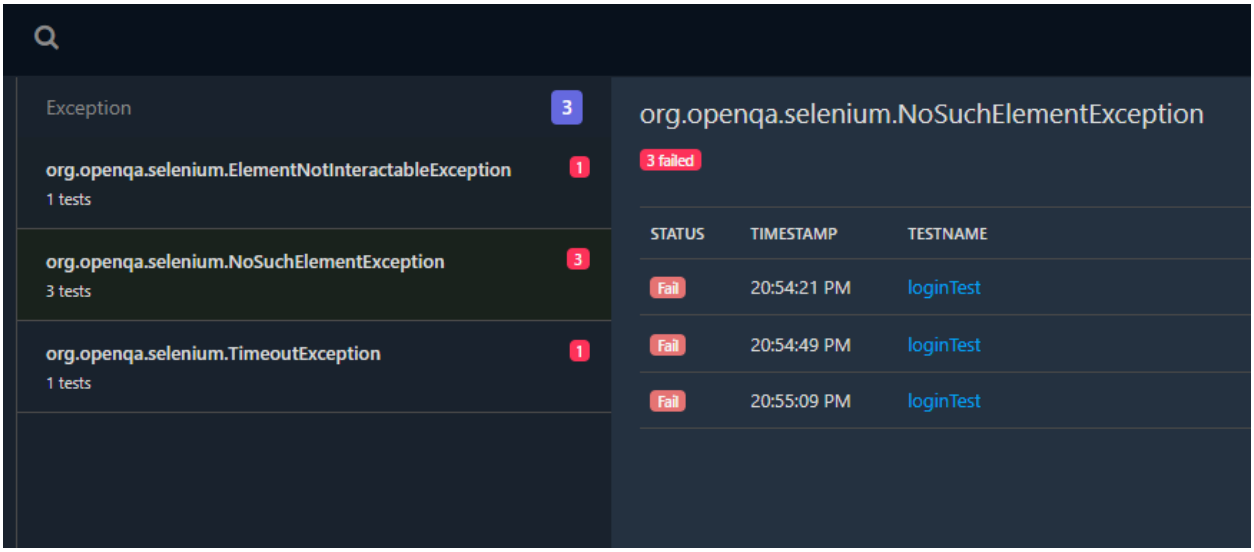


### 4.11.2 Element Not Found Errors

Some test cases failed due to:

- NoSuchElementException in Cart Page
- Incorrect or unstable XPath locators

**Solution:**
Locators were revalidated using browser inspection tools and improved with more stable attributes like aria-label and structured XPaths.




### 4.11.3 Synchronization Issues

Due to dynamic loading of elements:

- Tests sometimes executed before elements were fully loaded
- Caused intermittent failures in execution

**Solution:**
Implemented explicit waits and WebDriverWait conditions for better synchronization.

### 4.11.4 Data-Driven Testing Complexity

Managing multiple test datasets using Excel required careful handling.

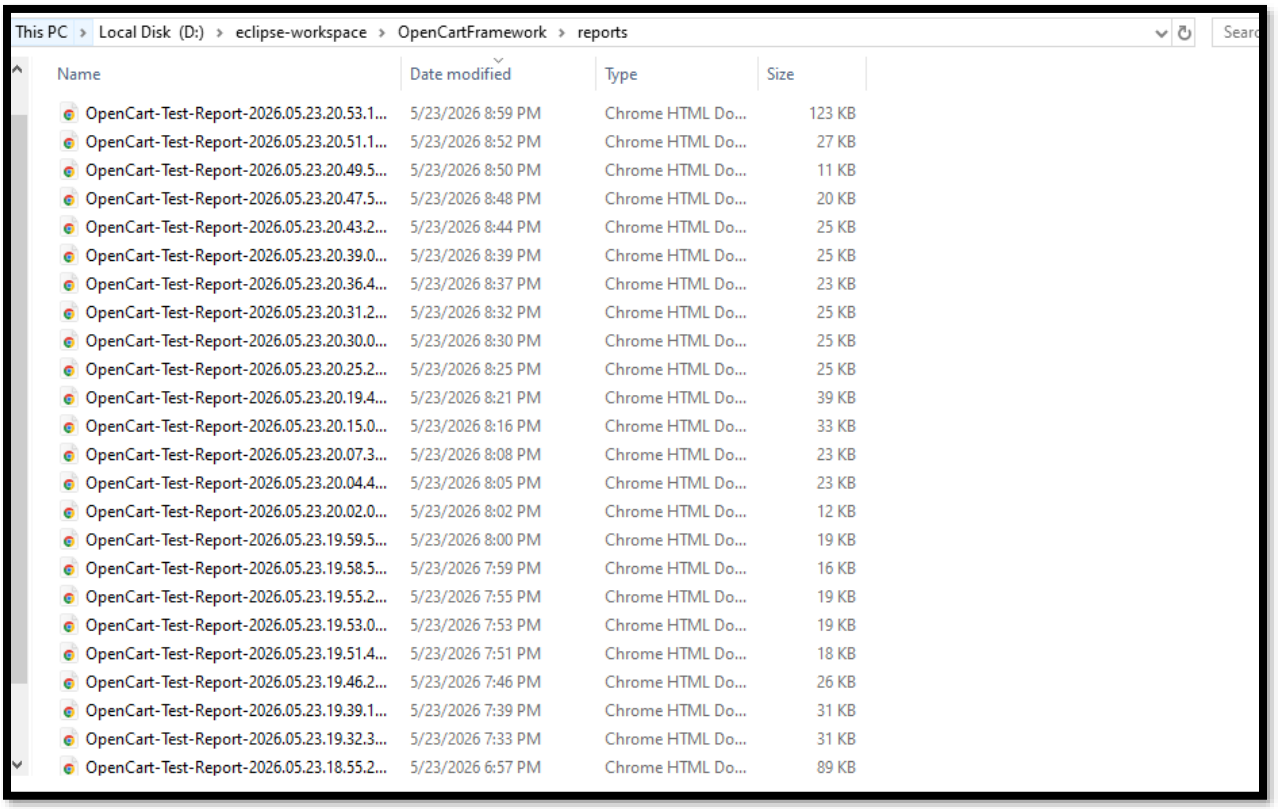
- Mapping Excel data to test scripts required proper indexing
- Apache POI integration required debugging for data parsing

```
// ---------------- SEARCH DATA ----------------
@DataProvider(name = "SearchData")
public Object[][] getSearchData() {

    String path = System.getProperty("user.dir") + "\\testData\\TestData.xlsx";
    XLUtility xl = new XLUtility(path);

    Object[][] data = null;

    try {
        int rowCount = xl.getRowCount("SearchData");
        int colCount = xl.getCellCount("SearchData", 1);

        data = new Object[rowCount][colCount];

        for (int i = 1; i <= rowCount; i++) {
            for (int j = 0; j < colCount; j++) {
                data[i - 1][j] = xl.getCellData("SearchData", i, j);
            }
        }

    } catch (Exception e) {
        e.printStackTrace();
    }

    return data;
}
```

### 4.11.5 Reporting and Screenshot Integration

Integrating Extent Reports with screenshots required:

- Proper hook into TestNG listeners
- Capturing screenshots only on failure cases



## 5. GUI Testing

Graphical User Interface (GUI) testing is performed to verify that the web application works correctly from the user's point of view. It checks whether different pages, buttons, forms, links, and other interface elements behave as expected when users interact with them.

In this project, GUI testing was carried out using Selenium WebDriver with the TestNG framework. The automation framework developed in the previous task was reused to execute different functional test scenarios of the OpenCart application. These tests cover important user activities such as account registration, login, product search, product viewing, cart management, checkout, and complete end to end user flow.

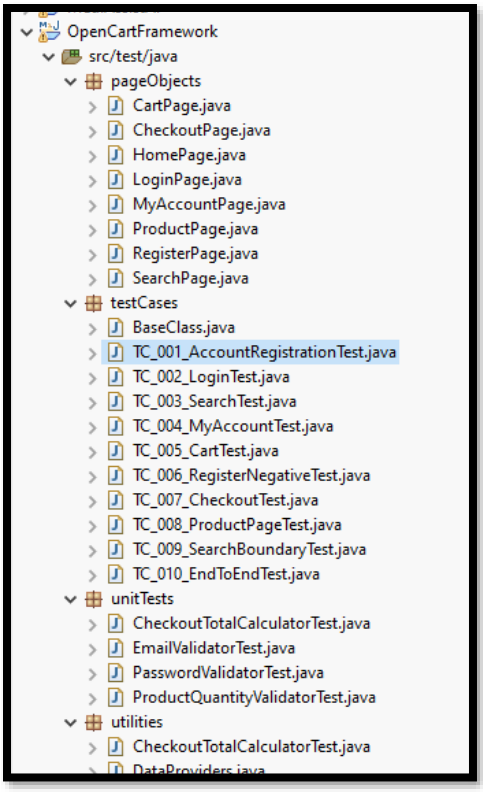
This task also focuses on cross browser testing using multiple TestNG configuration files. In addition, Selenium Grid, Selenium IDE, and Katalon Recorder are explored to understand different approaches for GUI automation testing.

### 5.1 Overview of GUI Testing Extension

The automation framework created in the previous assignment was extended for this task instead of creating a new framework. The existing Page Object Model structure, TestNG framework, reporting mechanism, and test scripts were reused to perform GUI testing on the OpenCart application.

The main objective of this task is to verify that the application's graphical interface behaves correctly during common user activities. Different GUI components such as navigation menus, forms, buttons, search fields, shopping cart, and checkout pages were tested through automated Selenium WebDriver scripts.

The framework also supports execution on different web browsers by using separate TestNG XML files for Chrome, Firefox, and Edge.



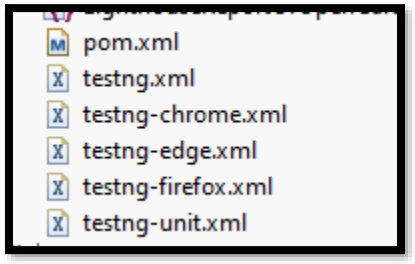

## 5.2 Scope of GUI Testing Activities

The GUI testing activities performed in this project include the following:

- Verification of page navigation across different sections of the website.
- Validation of registration and login forms using valid and invalid inputs.
- Testing of product search functionality with different search values.
- Verification of product details and shopping cart operations.
- Testing of the checkout process and complete end to end user flow.
- Execution of automated test cases on Chrome, Firefox, and Edge browsers using separate TestNG configuration files.
- Generation of execution reports and logs after each test run.
- Exploration of Selenium Grid, Selenium IDE, and Katalon Recorder as additional GUI testing tools.

These activities help ensure that the main features of the OpenCart application work correctly from the user's perspective while improving confidence in the automation framework.

**Screenshots to add**




## 5.3 Selenium WebDriver Test Scenarios

The GUI testing of the OpenCart application was implemented using Selenium WebDriver with Java, TestNG, and the Page Object Model (POM). The objective of these automated tests was to verify that the main functionalities of the application work correctly from a user's perspective. The framework includes both positive and negative test cases and supports data driven testing through Excel files.

A total of ten Selenium test classes were developed, each focusing on a different module of the application. The following sections describe the implemented test scenarios.

### 5.3.1 Account Registration Testing

The Account Registration module verifies that a new user can successfully create an account by providing valid information. During every execution, a unique email address is generated automatically using the current system time to prevent duplicate registration errors.

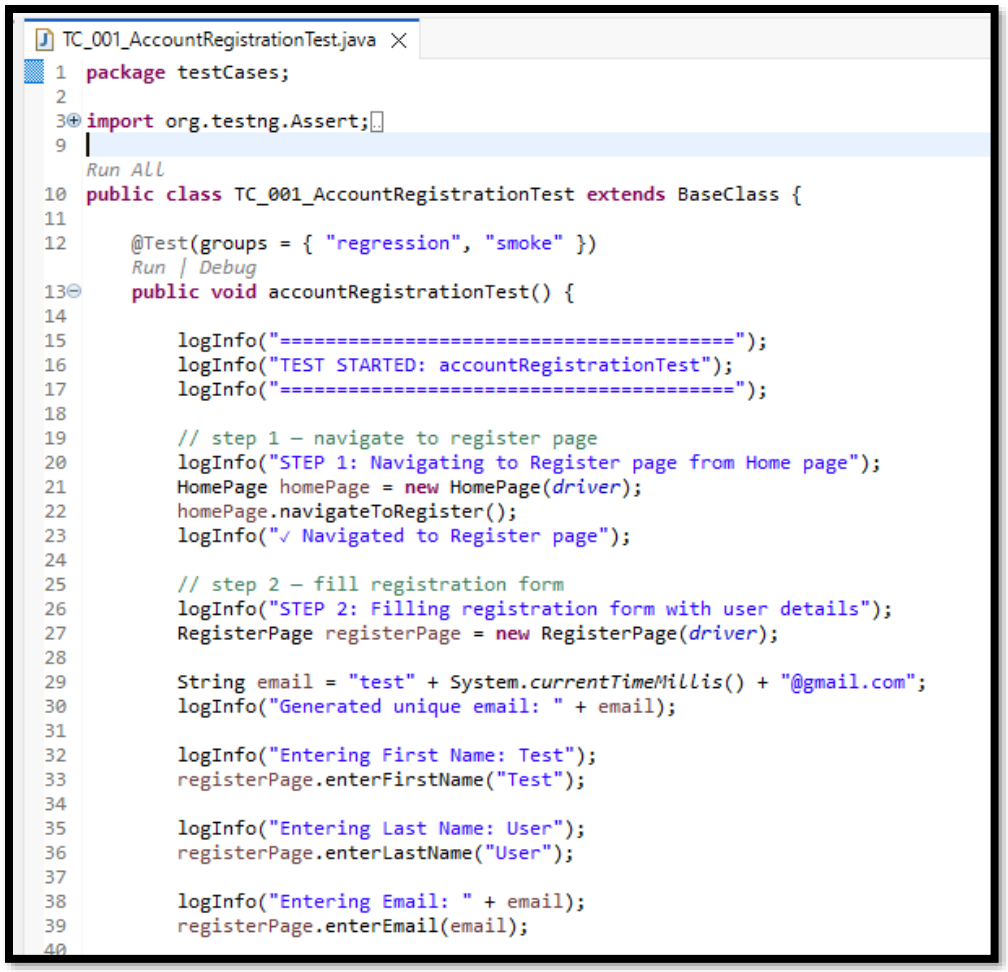
After successful registration, the test verifies that the registration success page is displayed. The newly created email and password are also stored in the Excel file so that the same account can be used by the Login test.

**Test scenarios implemented**

- Successful account registration
- Newsletter subscription selection
- Privacy policy acceptance

- Saving registered user data to Excel

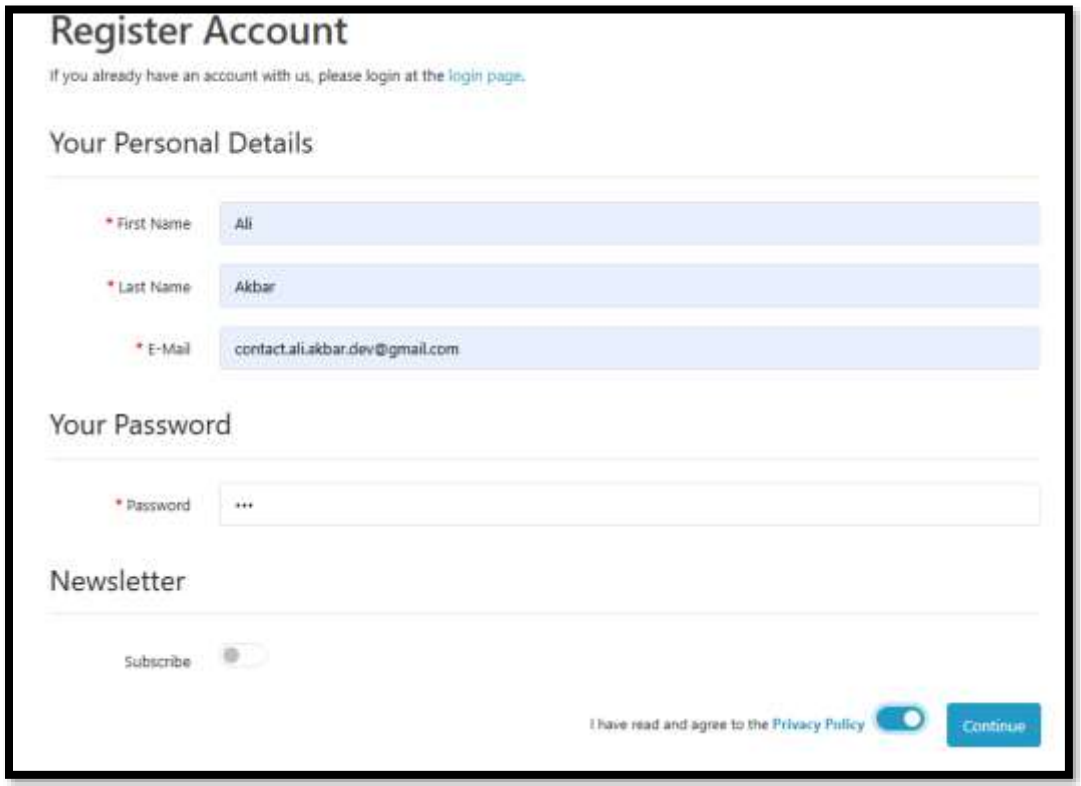
**Screenshots**

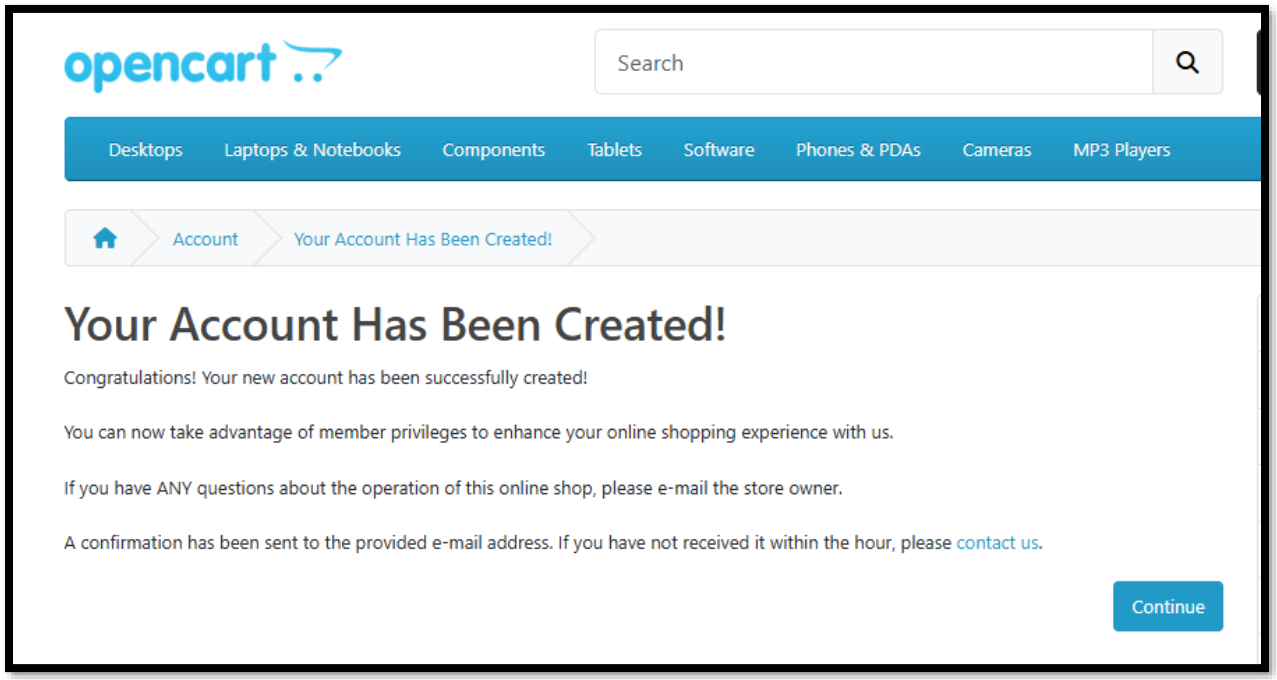


TC_001_AccountRegistrationTest.java

```
1  package testCases;
2
3  import org.testng.Assert;
9
   Run All
10 public class TC_001_AccountRegistrationTest extends BaseClass {
11
12     @Test(groups = { "regression", "smoke" })
       Run | Debug
13     public void accountRegistrationTest() {
14
15         logInfo("========================================");
16         logInfo("TEST STARTED: accountRegistrationTest");
17         logInfo("========================================");
18
19         // step 1 – navigate to register page
20         logInfo("STEP 1: Navigating to Register page from Home page");
21         HomePage homePage = new HomePage(driver);
22         homePage.navigateToRegister();
23         logInfo("✓ Navigated to Register page");
24
25         // step 2 – fill registration form
26         logInfo("STEP 2: Filling registration form with user details");
27         RegisterPage registerPage = new RegisterPage(driver);
28
29         String email = "test" + System.currentTimeMillis() + "@gmail.com";
30         logInfo("Generated unique email: " + email);
31
32         logInfo("Entering First Name: Test");
33         registerPage.enterFirstName("Test");
34
35         logInfo("Entering Last Name: User");
36         registerPage.enterLastName("User");
37
38         logInfo("Entering Email: " + email);
39         registerPage.enterEmail(email);
40
```

**Registration Page**



### 5.3.2 Login Testing

The Login module validates both successful and unsuccessful login attempts. Test data is read from an Excel sheet using TestNG Data Providers. Valid credentials should redirect the user to the My Account page, while invalid credentials should display either HTML validation messages or server side warning messages.

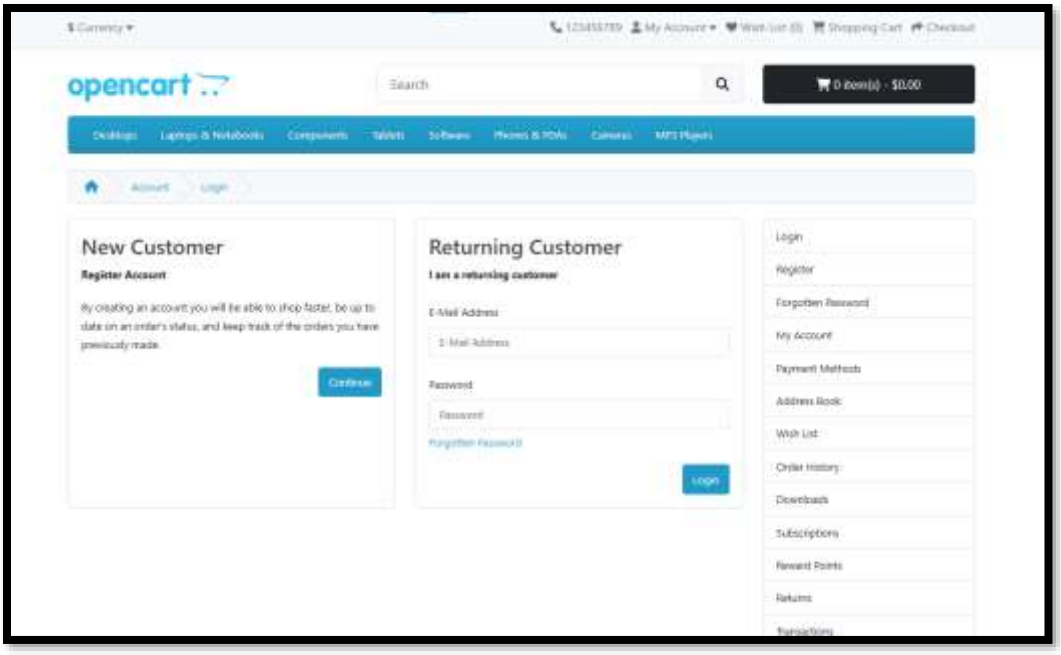
After a successful login, the test also performs logout to restore the initial state for the next execution.

**Test scenarios implemented**

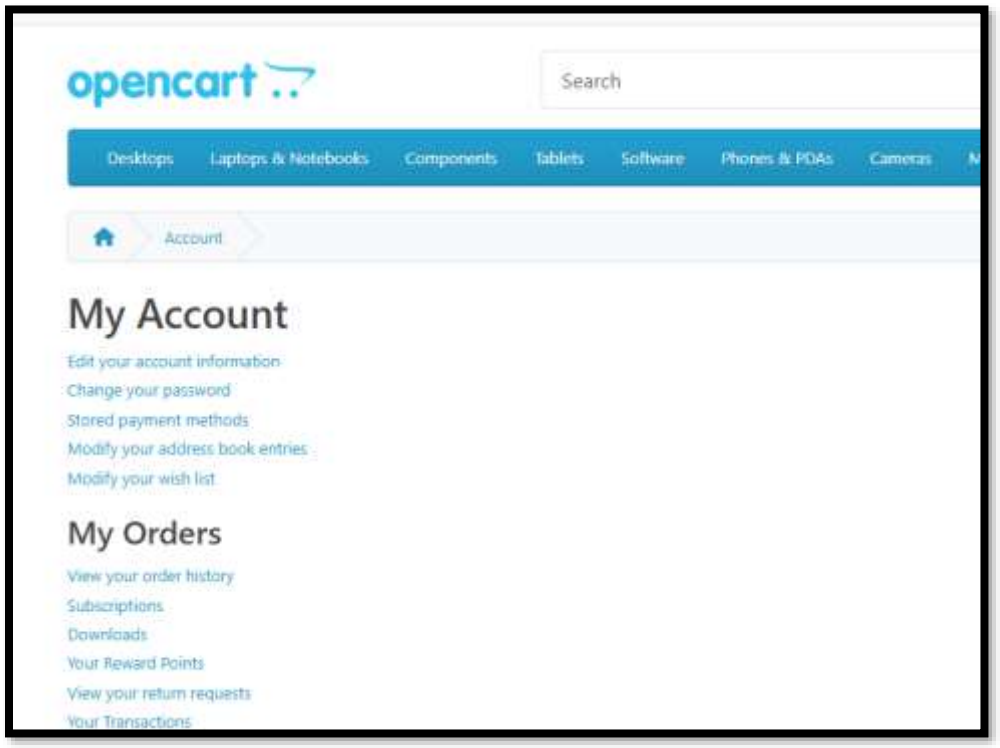
- Valid login
- Invalid password
- Invalid email
- HTML5 email validation
- Logout after successful login

**Screenshots**

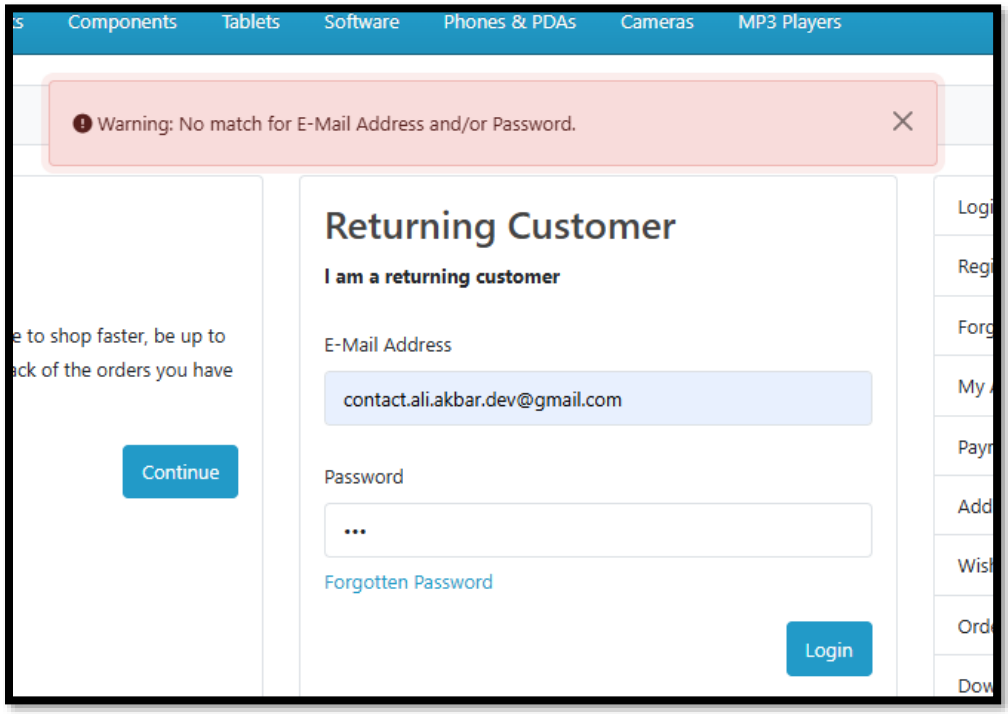
Login page



My Account page after successful login




Invalid login warning



### 5.3.3 Product Search Testing

The Search module verifies that users can search for products available in the OpenCart store. The tests include both successful and unsuccessful searches. For valid keywords, the application should display matching products. For invalid keywords, a "No results" message should appear.

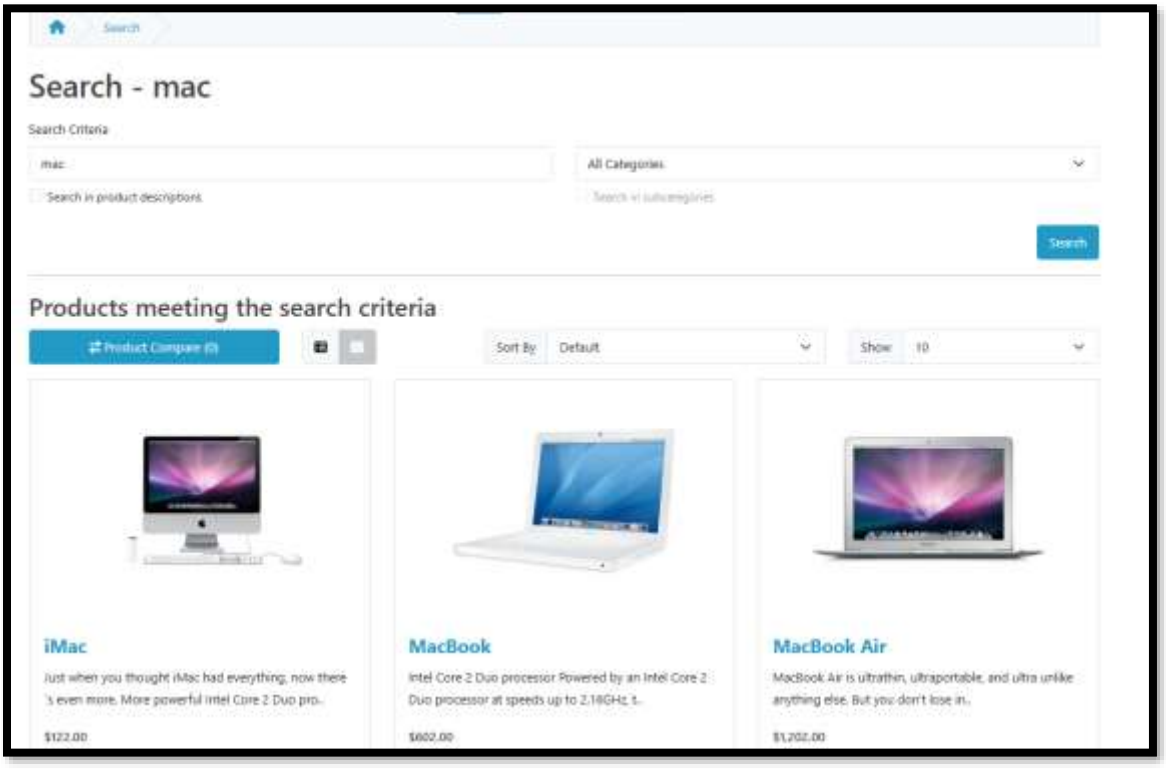
The number of returned products is also verified to ensure that search functionality works correctly.

**Test scenarios implemented**

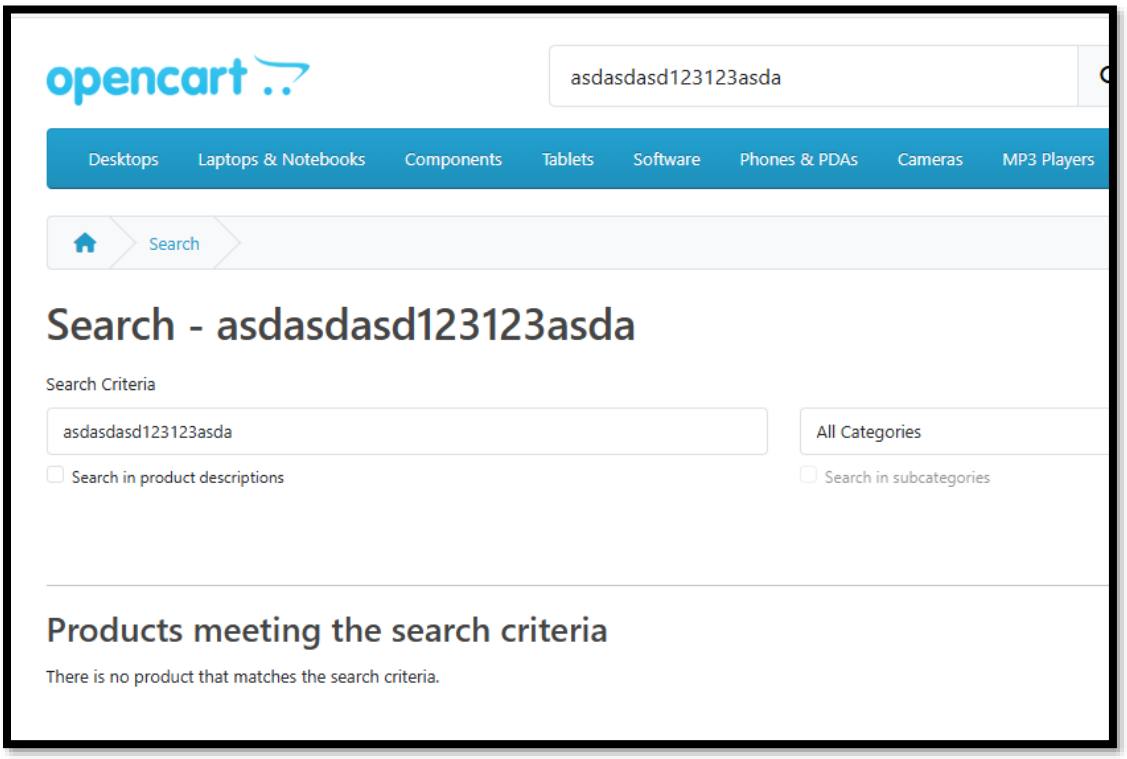
- Search using valid product names
- Search using invalid product names
- Verification of search results count
- Verification of "No results" message

**Screenshots**

Search results page



No results page




### 5.3.4 My Account Testing

The My Account module verifies the functionality available after a successful login. Multiple independent tests were created to validate different account features.

The tests verify navigation to the My Account page, Edit Account page, Change Password page, Wishlist page, Order History page, and Logout functionality.

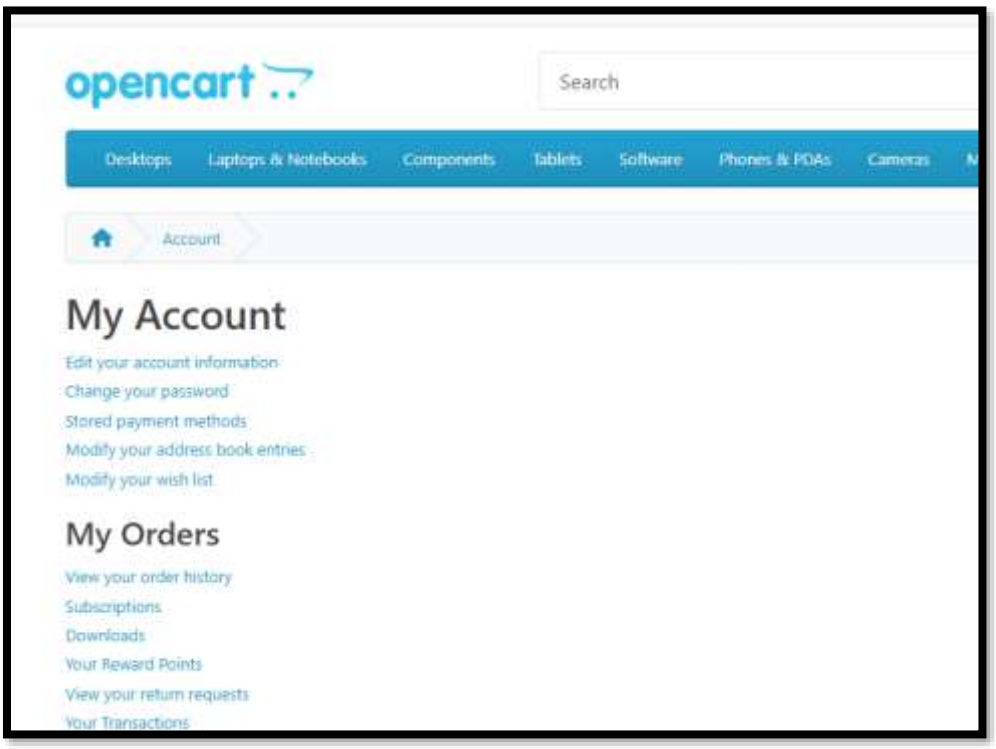
**Test scenarios implemented**

- Verify My Account page
- Open Edit Account page
- Open Change Password page
- Open Wishlist page
- Open Order History page

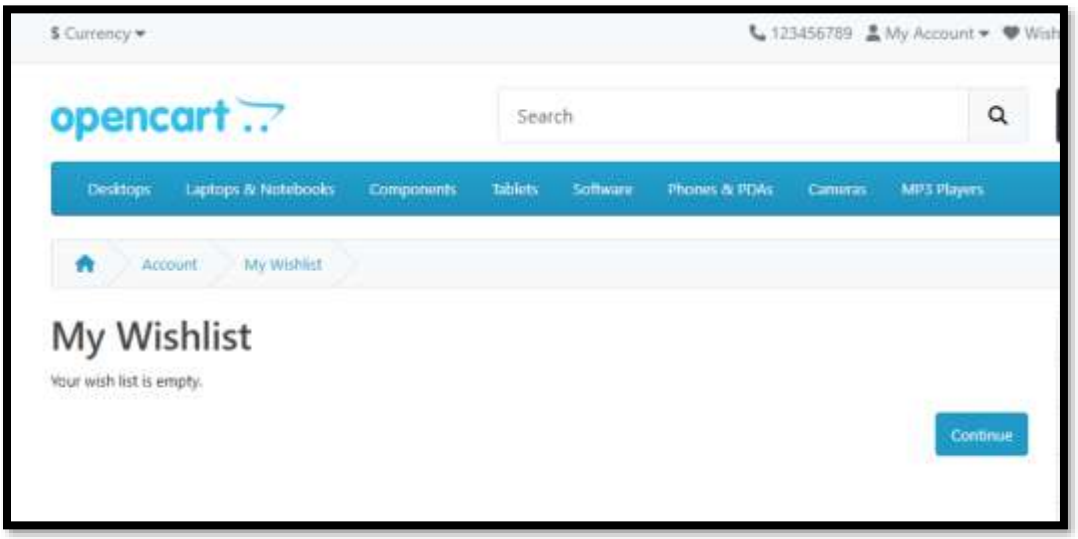
- Logout functionality

**Screenshots**

My Account page




Wishlist page



### 5.3.5 Shopping Cart Testing

The Shopping Cart module verifies that users can successfully add products to the cart and manage them correctly. The cart item count is checked after every operation to ensure that the application updates the shopping cart properly.

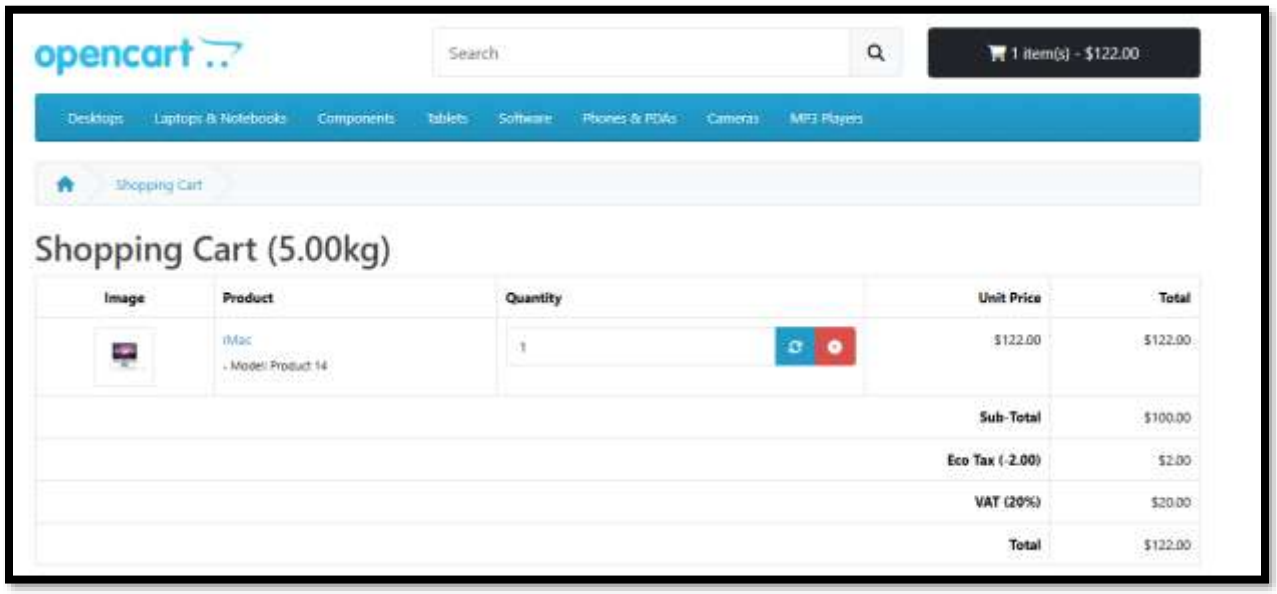
The tests also verify viewing the shopping cart and removing products from it.

**Test scenarios implemented**

- Add one product
- Add multiple products
- Add same product twice
- View shopping cart
- Remove product from cart

**Screenshots**

Shopping cart page



### 5.3.6 Registration Validation Testing

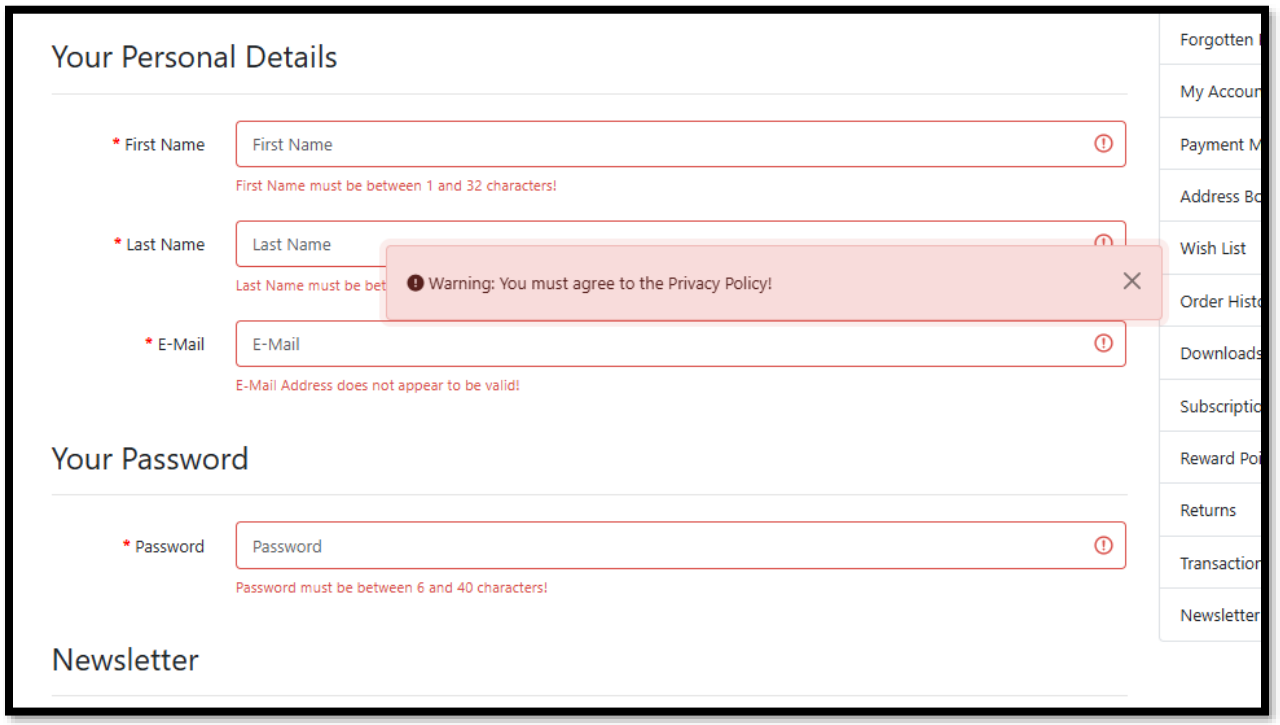
Negative registration test cases were created to verify input validation during account creation. These tests ensure that invalid user information does not allow account registration and that proper validation messages are displayed.

**Test scenarios implemented**

- Missing required fields
- Invalid email format
- Password validation
- Required field validation

**Screenshots**

Registration page showing validation errors



### 5.3.7 Checkout Testing

The Checkout module verifies different checkout scenarios for both guest users and registered users.

The implemented tests include guest checkout redirection, logged in user checkout, verification of cart total, updating product quantity, and testing checkout with an empty cart.

During testing, a defect related to empty cart checkout was identified and documented in the framework.

**Test scenarios implemented**

- Guest checkout
- Logged in user checkout
- Cart total verification
- Update product quantity
- Empty cart checkout validation

### 5.3.8 Product Page Testing

The Product Page module verifies that product information is displayed correctly before adding products to the shopping cart.

The implemented tests validate the product page, product details, and user interaction with the Add to Cart feature.

**Test scenarios implemented**

- Open product page
- Verify product details
- Add product to cart from product page

### 5.3.9 Search Boundary Testing

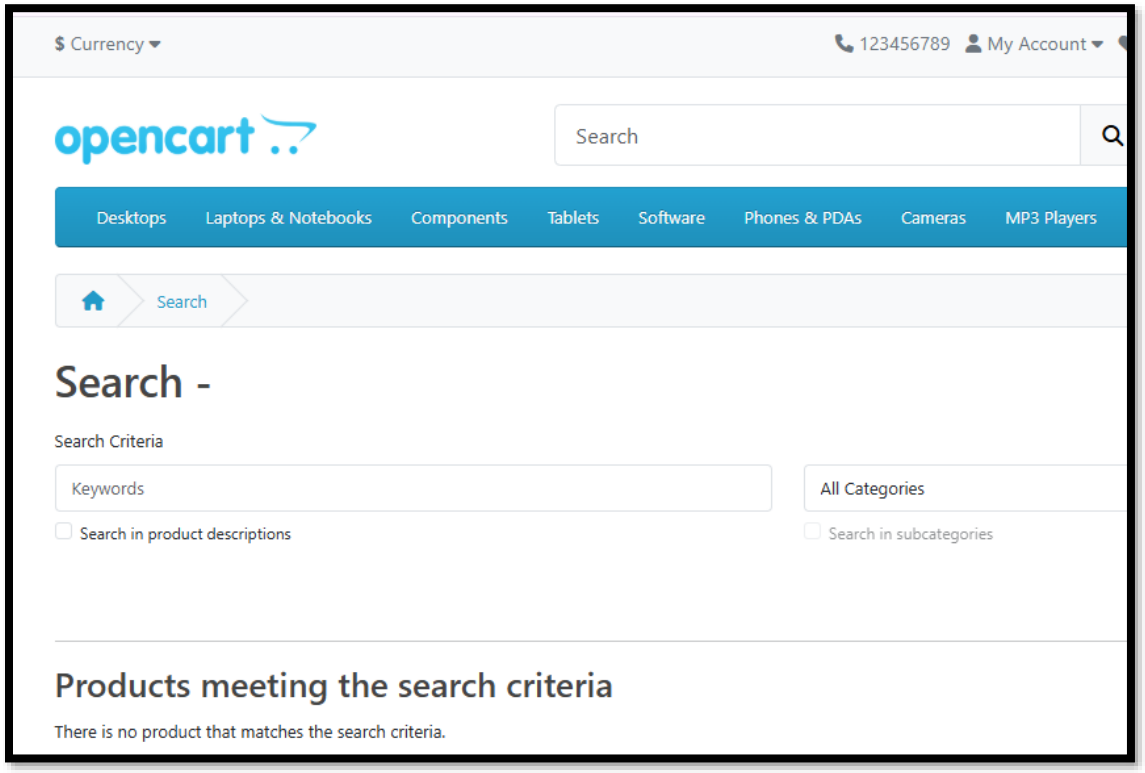
Boundary testing was performed to verify how the search functionality behaves with unusual user input.

Different boundary conditions were tested to ensure that the application remains stable and responds correctly.

**Test scenarios implemented**

- Empty search
- Long search text
- Special characters
- Numeric input
- Mixed input values

**Screenshots**



### 5.3.10 End to End Testing

An end to end automated test was implemented to verify the complete business workflow of the OpenCart application.

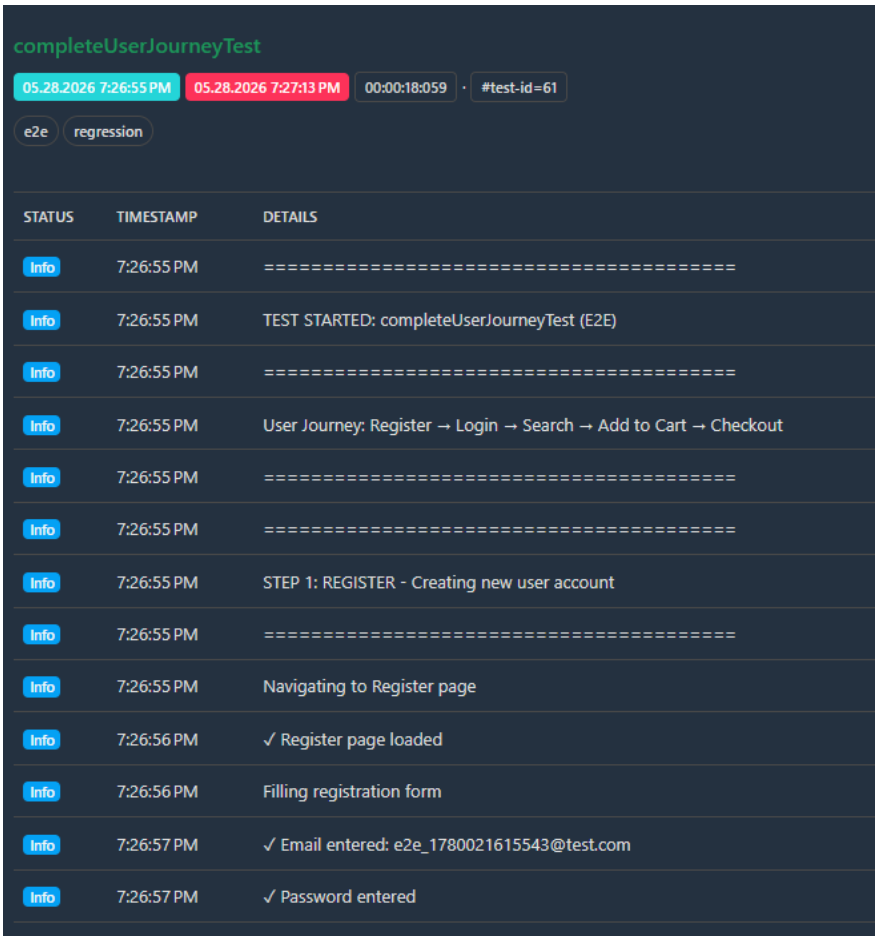
The test starts by creating a new user account with a unique email address. After successful registration, the user logs out and logs in again using the same credentials. The test then searches for a product, selects it, adds it to the shopping cart, opens the cart, proceeds to checkout, and verifies that the checkout page and cart total are displayed successfully.

This test confirms that the major modules of the application work correctly together without failures.

**Workflow verified**

Register → Login → Search → Product Selection → Add to Cart → View Cart → Checkout

**Screenshot to add**



## 5.4 Selenium WebDriver Automation Approach

The automation framework for GUI testing was developed using Selenium WebDriver with Java and TestNG. The main objective of the framework is to automate functional testing of the OpenCart application across different browsers and ensure reliable execution of test cases.

The framework is designed using the Page Object Model (POM) architecture, where each web page has a separate class containing locators and methods. This improves code reusability, readability, and maintainability.

### 5.4.1 Test Framework Structure

The project is structured into the following main components:

- Test Cases package containing all TestNG test classes (TC_001 to TC_010)
- Page Objects package containing page level methods and locators
- Utilities package for Excel handling, data providers, and reporting
- BaseClass for browser setup and driver initialization

All test cases extend the BaseClass which handles browser setup, configuration loading, and driver management.

### 5.4.2 Cross Browser Execution

Cross browser testing is implemented using TestNG XML files. The framework supports execution on multiple browsers including Chrome, Edge, and Firefox.

Separate TestNG suite files are created for each browser:

- Chrome execution suite
- Edge execution suite
- Firefox execution suite

Each suite passes the browser parameter to the BaseClass, which dynamically initializes the required browser using WebDriver.

### 5.4.3 TestNG Groups and Execution Control

Test cases are categorized using TestNG groups such as:

- smoke
- regression
- e2e

This allows selective execution of test suites based on testing requirements. For example, regression testing runs all major test cases, while smoke testing runs only critical functionalities.

### 5.4.4 Test Execution Flow

The execution flow of the automation framework is as follows:

1. TestNG XML file is triggered
2. Browser parameter is passed to BaseClass
3. WebDriver is initialized based on selected browser
4. Application URL is loaded from configuration file
5. Test cases are executed using Page Object methods
6. Extent Reports capture logs and execution status
7. Browser is closed after each test method

### 5.4.5 Reporting and Logging

Extent Reports are used for test execution reporting. Each test step is logged using a custom logging method defined in BaseClass.

This provides detailed execution tracking including:

- Step by step execution logs
- Pass and fail status of each test case
- Screenshots in case of failure (if enabled)

- Execution summary for each browser

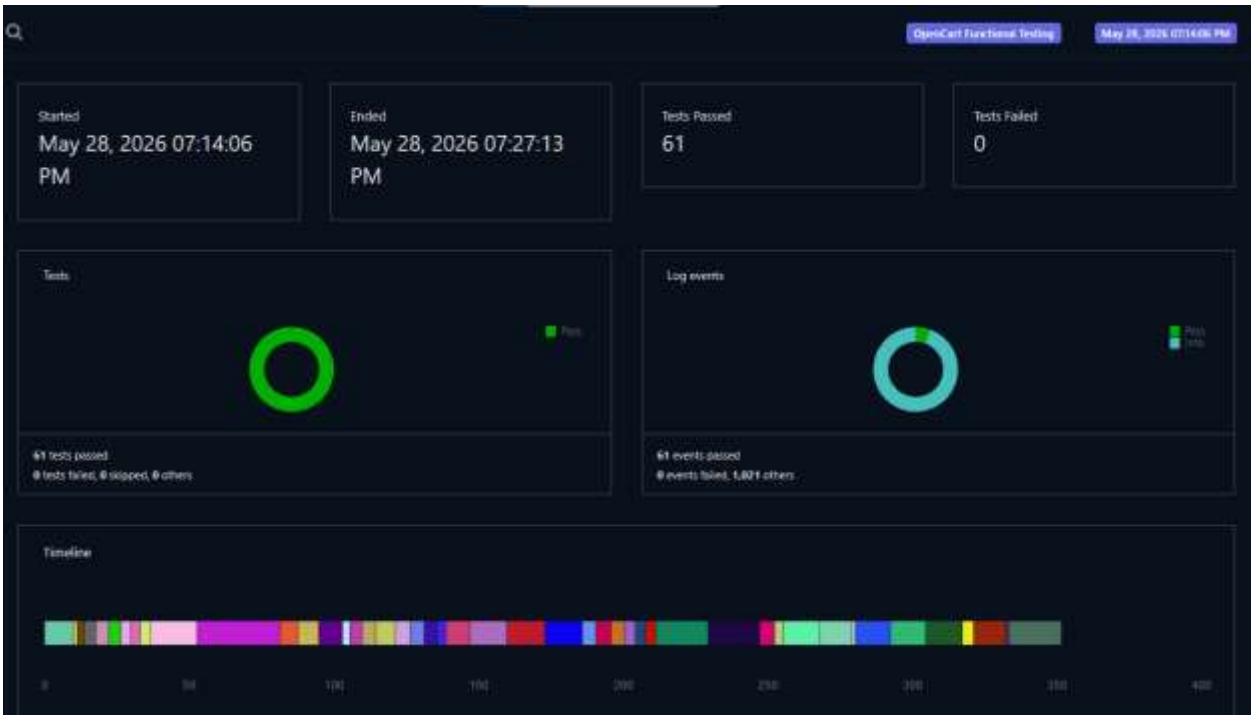
### 5.4.6 Screenshot Evidence

During execution, screenshots can be captured using the utility method in BaseClass. These screenshots help in debugging failed test cases and are attached in the final report.

**Screenshot**

testng-chrome.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">

<suite name="OpenCart - Chrome Browser Tests">

    <listeners>
        <listener class-name="utilities.ExtentReportManager"/>
    </listeners>

    <test name="Chrome - Full Regression Suite">
        <parameter name="browser" value="chrome"/>

        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
            <class name="testCases.TC_001_AccountRegistrationTest"/>
            <class name="testCases.TC_002_LoginTest"/>
            <class name="testCases.TC_003_SearchTest"/>
            <class name="testCases.TC_004_MyAccountTest"/>
            <class name="testCases.TC_005_CartTest"/>
            <class name="testCases.TC_006_RegisterNegativeTest"/>
            <class name="testCases.TC_007_CheckoutTest"/>
            <class name="testCases.TC_008_ProductPageTest"/>
            <class name="testCases.TC_009_SearchBoundaryTest"/>
            <class name="testCases.TC_010_EndToEndTest"/>
        </classes>
    </test>

</suite>
```


OpenCart Functional Testing
May 28, 2026 07:14:06 PM
Started
May 28, 2026 07:14:06 PM
Ended
May 28, 2026 07:27:13 PM
Tests Passed
61
Tests Failed
0
Tests
Pass
61 tests passed
0 tests failed, 0 skipped, 0 others
Log events
61 events passed
0 events failed, 1,021 others
Timeline

## 5.5 Selenium Grid Implementation

### 5.5.1 Introduction

Selenium Grid is used to execute automated test cases in a distributed environment using a Hub and Node architecture. It allows test execution on multiple browsers and systems using a central controller (Hub) and execution machines (Nodes). This improves execution speed and supports cross browser testing.

In this project, Selenium Grid 4 is implemented to execute OpenCart automation test cases using RemoteWebDriver. The Grid is configured locally, where both Hub and Node are running on the same machine for demonstration purposes.

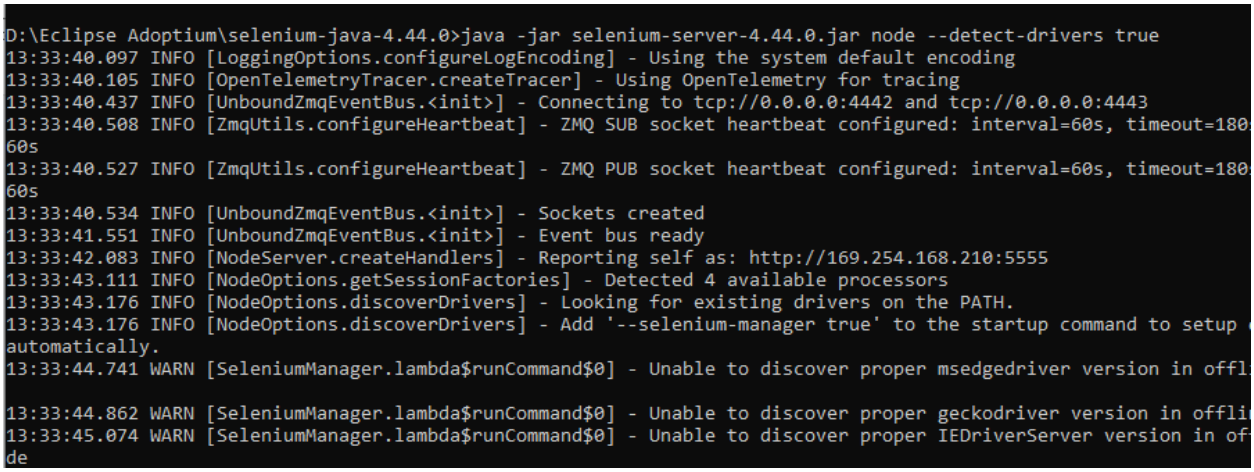
### 5.5.2 Selenium Grid Setup and Execution

The Selenium Grid was started using Selenium Server 4.44.0.

**Step 1: Start Selenium Grid Hub**

Command used:

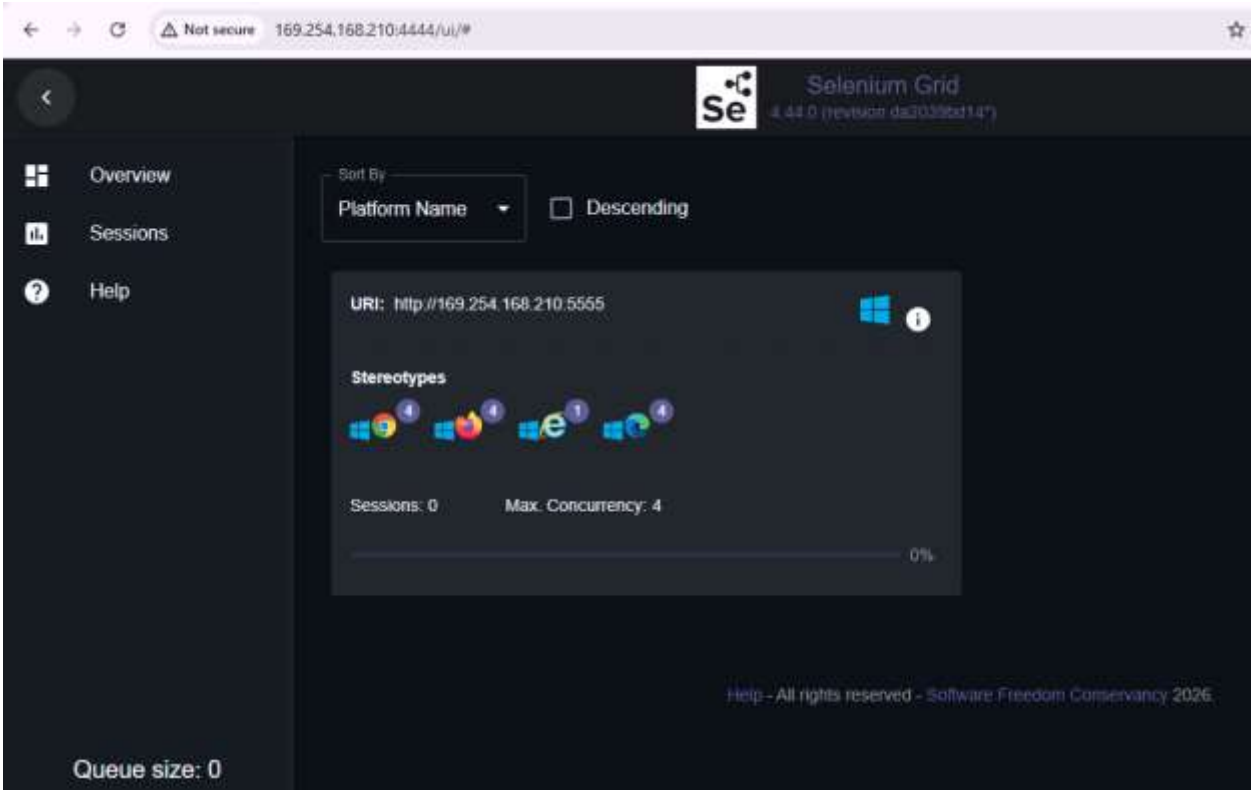
java -jar selenium-server-4.44.0.jar hub



**Step 2: Access Grid UI**

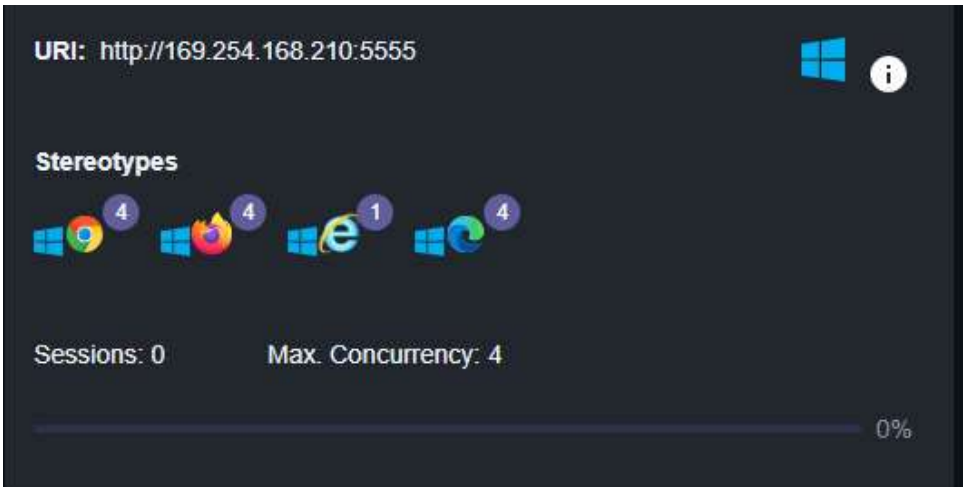
Grid dashboard is accessed using:

http://169.254.168.210:4444/ui




**Step 3: Node Registration**

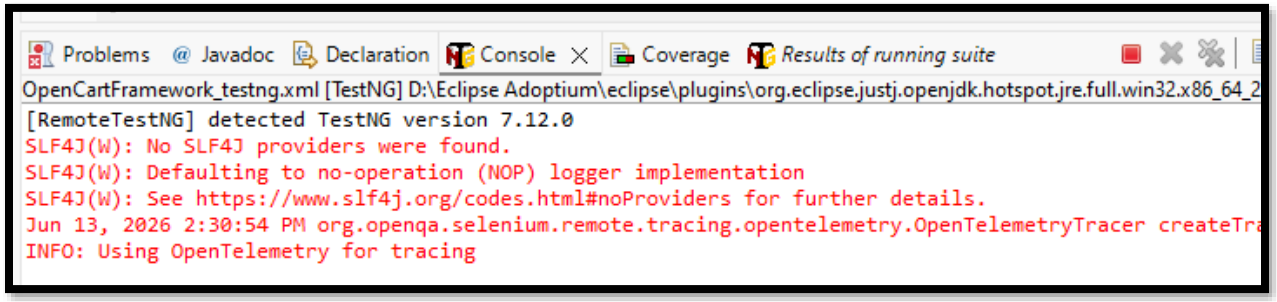
Node automatically registers with the Hub on port 5555 and becomes active.



### 5.5.3 Execution Flow of Selenium Grid

The execution flow in this project is as follows:

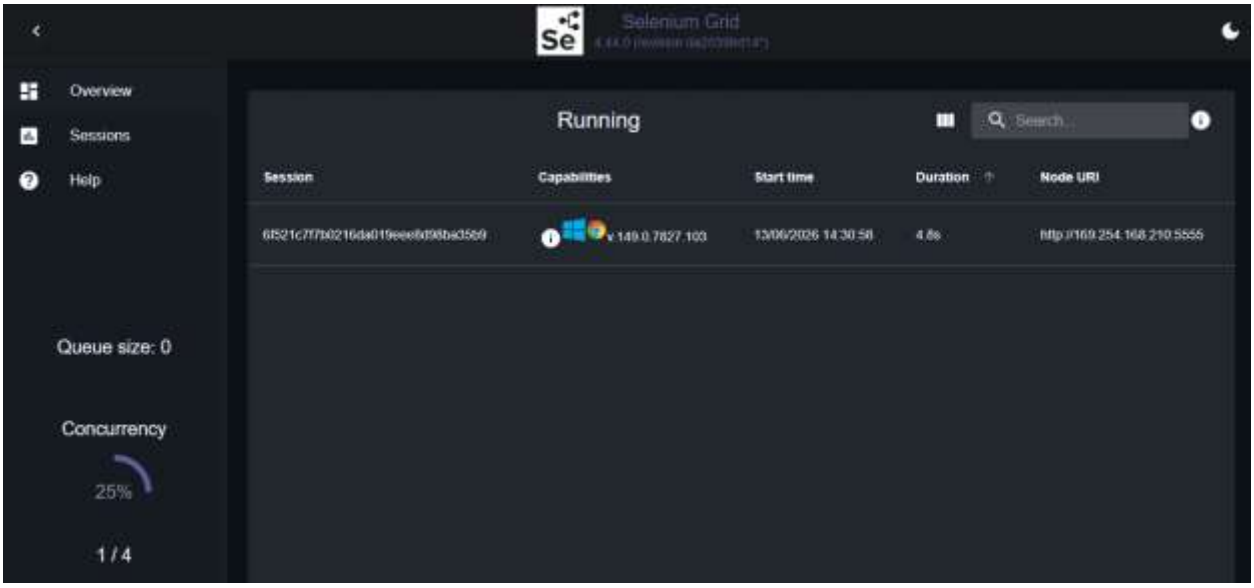
1. TestNG framework triggers test execution
2. BaseClass reads runMode as grid
3. RemoteWebDriver connects to Selenium Grid Hub
4. Hub assigns test execution to an available node
5. Node launches the browser and executes test scripts
6. Results are returned back to TestNG framework



### 5.5.4 Remote Execution Evidence

During execution, multiple sessions were created successfully in the Grid.

Each test execution generates a unique session ID and is assigned to a node.

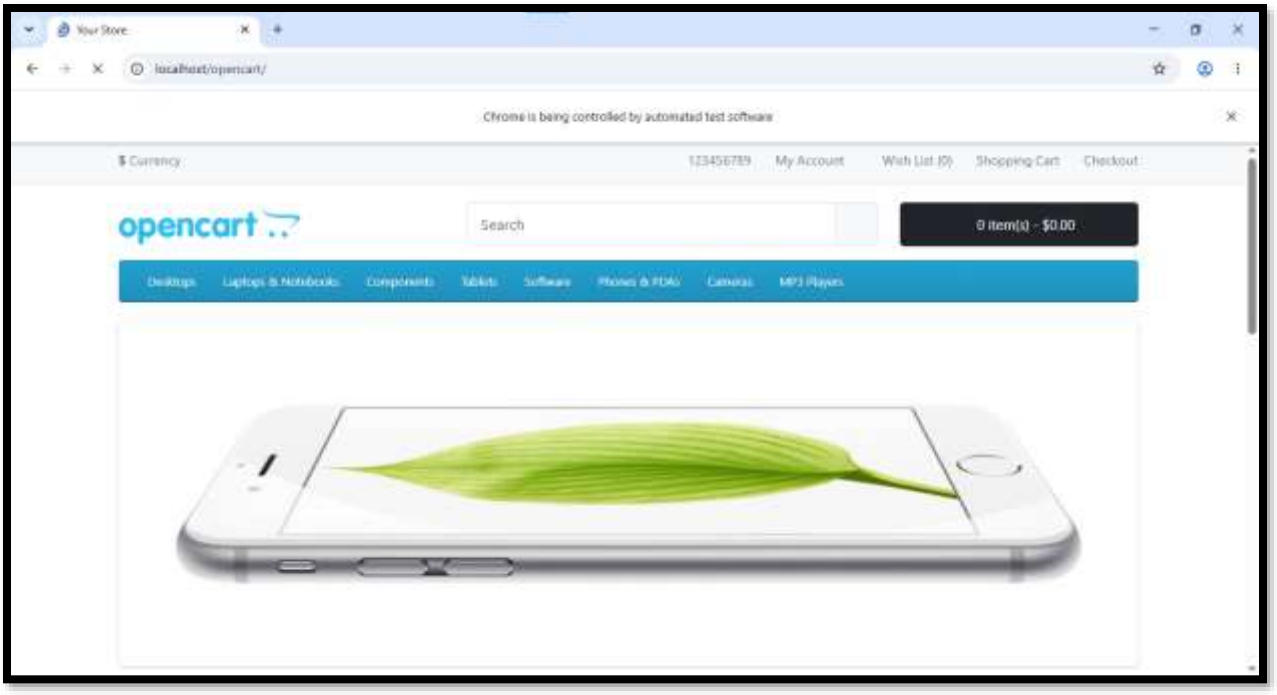



### 5.5.5 Browser Execution Confirmation

Even though Selenium Grid is used, the browser opens on the local machine because:

- Hub and Node are configured on the same system
- Node launches browser locally as part of execution environment

However, execution is still handled through Grid architecture using RemoteWebDriver.

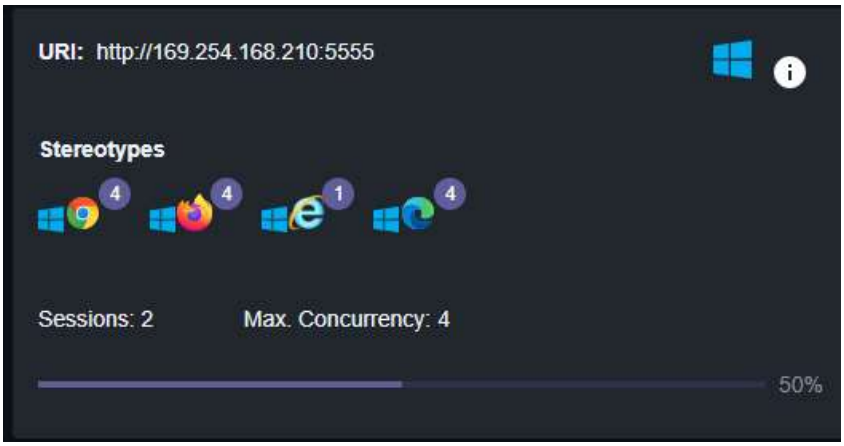



### 5.5.6 Cross Browser and Parallel Capability

Selenium Grid supports execution across multiple browsers such as Chrome, Firefox, and Edge.

In this project:

- Chrome is used as primary execution browser
- Grid is configured to support multiple browser capabilities
- TestNG groups manage regression and smoke execution



### 5.5.7 Benefits of Implementation

The implementation provides the following advantages:

- Parallel execution capability
- Cross browser testing support
- Centralized test execution
- Faster regression execution
- Scalable architecture for future distributed testing

### 5.5.8 Parallel Test Execution on Multiple Browsers

To fulfill the requirement of running tests on multiple browsers simultaneously, the TestNG configuration file was updated to support parallel execution through Selenium Grid. Two separate test blocks were defined inside the suite, one for Chrome and one for Firefox. Both test blocks were configured to use the Grid as the execution mode.

The parallel="tests" attribute was added to the suite along with thread-count="2" to allow both browsers to run at the same time. The same test classes were included in both blocks so that identical test scenarios execute on both browsers in parallel.

**TestNG XML Configuration for Parallel Execution**

The following XML file was used to trigger parallel execution on Chrome and Firefox through Selenium Grid:

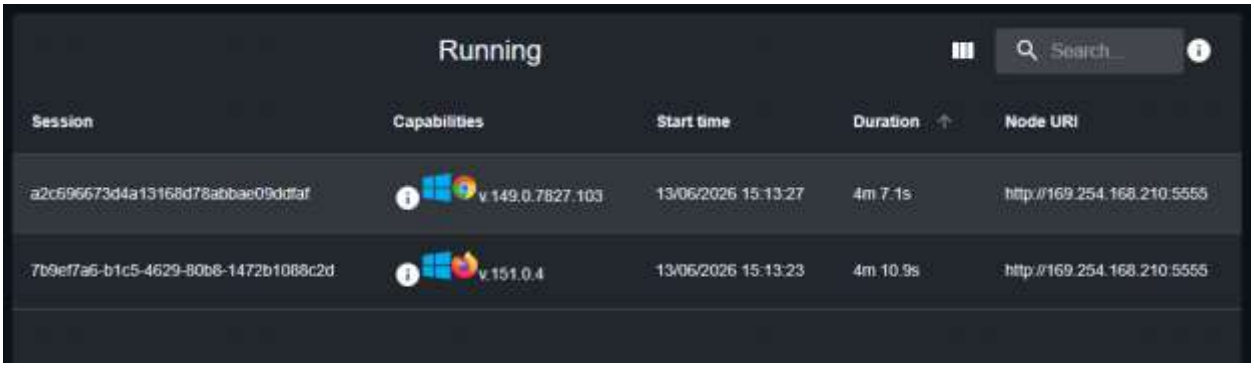

testng-chrome.xml
BaseClass.java
testng.xml
http://testng.org/testng-1.9.8.dtd (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">
<suite name="OpenCart Test Suite" parallel="tests" thread-count="2">
<listeners>
<listener class-name="utilities.ExtentReportManager"/>
</listeners>
<!-- Chrome Execution -->
<test name="Chrome Test Suite">
<parameter name="browser" value="chrome"/>
<parameter name="runMode" value="grid"/>
<groups>
<run>
<include name="smoke"/>
</run>
</groups>
<classes>
<class name="testCases.TC_002_LoginTest"/>
<class name="testCases.TC_003_SearchTest"/>
</classes>
</test>
<!-- Firefox Execution -->
<test name="Firefox Test Suite">
<parameter name="browser" value="firefox"/>
<parameter name="runMode" value="grid"/>
<groups>
<run>
<include name="smoke"/>
</run>
</groups>
<classes>
<class name="testCases.TC_002_LoginTest"/>
<class name="testCases.TC_003_SearchTest"/>
</classes>
</test>

**Grid Sessions During Parallel Execution**

When the test suite was executed, two active sessions were created simultaneously on the Selenium Grid. Each session was assigned to a separate browser — one for Chrome and one for Firefox. Both sessions ran on the same node at http://169.254.168.210:5555.

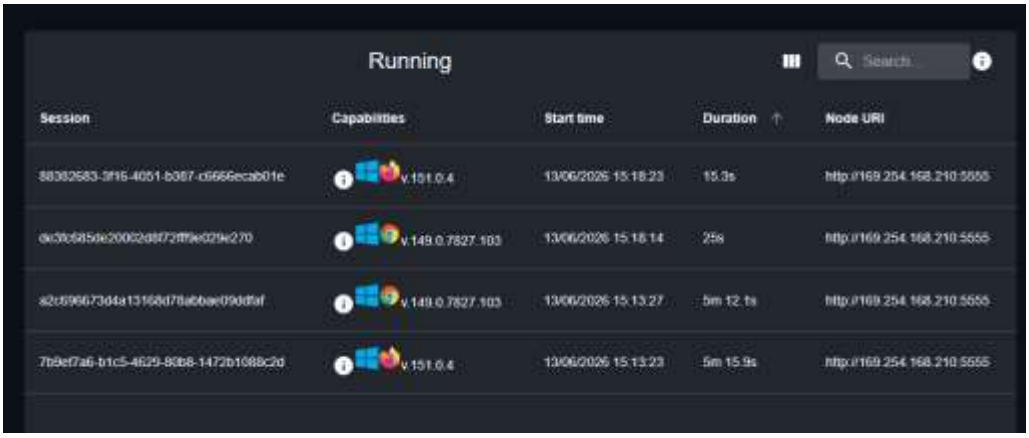
The Grid dashboard confirmed the following two active sessions:

- Session 1 — Chrome v149, started at 15:13:27, duration 26.8s
- Session 2 — Firefox v151, started at 15:13:23, duration 30.6s

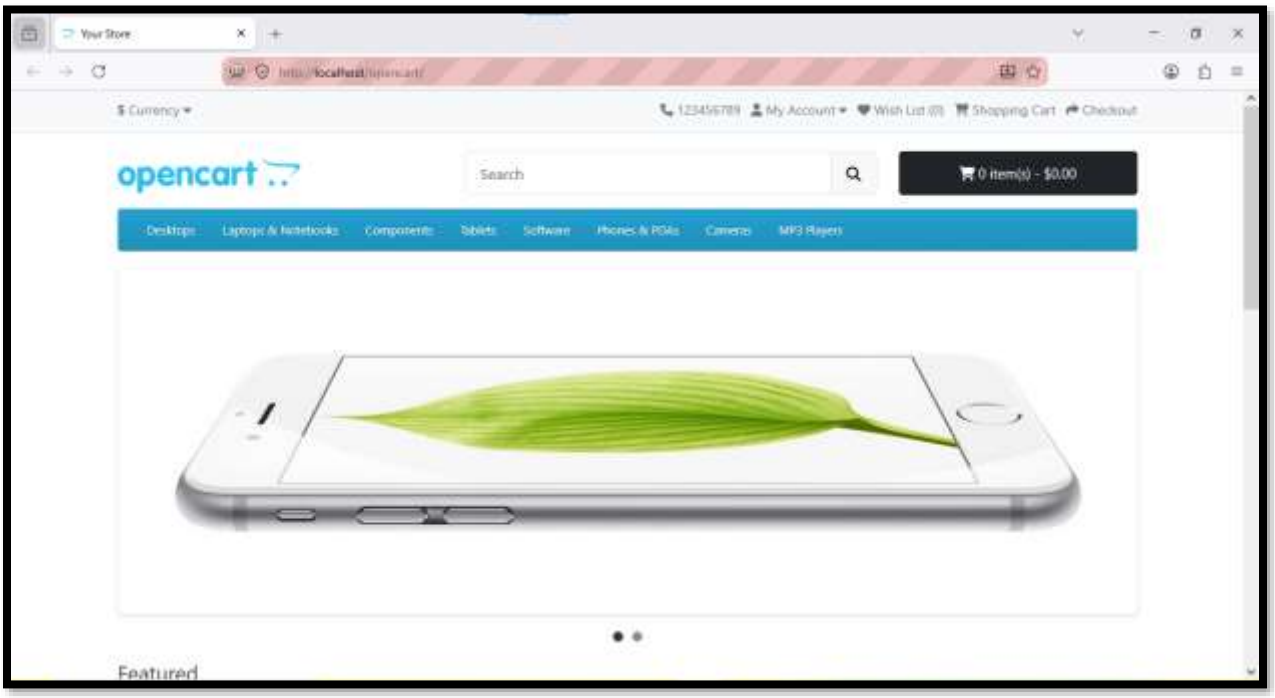



**Browser Windows During Execution**

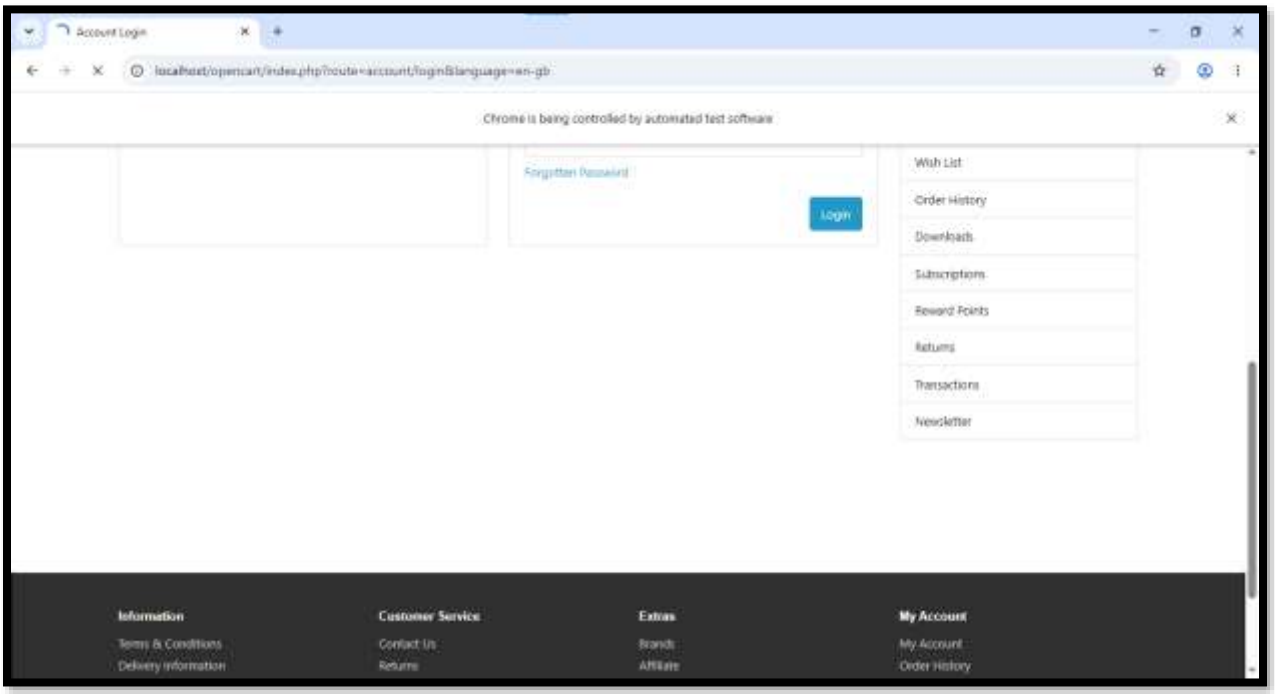
Both Chrome and Firefox browser windows opened at the same time on the local machine and executed the assigned test cases independently through the Grid.



On firefox




On google chrome



**Outcome**

Both test cases executed successfully on Chrome and Firefox in parallel through Selenium Grid. This confirmed that the framework supports multi-browser parallel execution using RemoteWebDriver and TestNG parallel configuration.

## 5.6 Selenium IDE and Katalon Recorder Implementation

### 5.6.1 Introduction

Selenium IDE and Katalon Recorder are record and playback tools used for quick automation of web application test cases without requiring complex coding. In this project, both tools were used to validate basic user flows of the OpenCart application.

The objective of using these tools is to demonstrate alternative automation approaches alongside Selenium WebDriver and to validate core functionalities such as user login and product cart operations.

### 5.6.2 Selenium IDE Implementation

Selenium IDE was used to create a test suite that automates the following user flow:

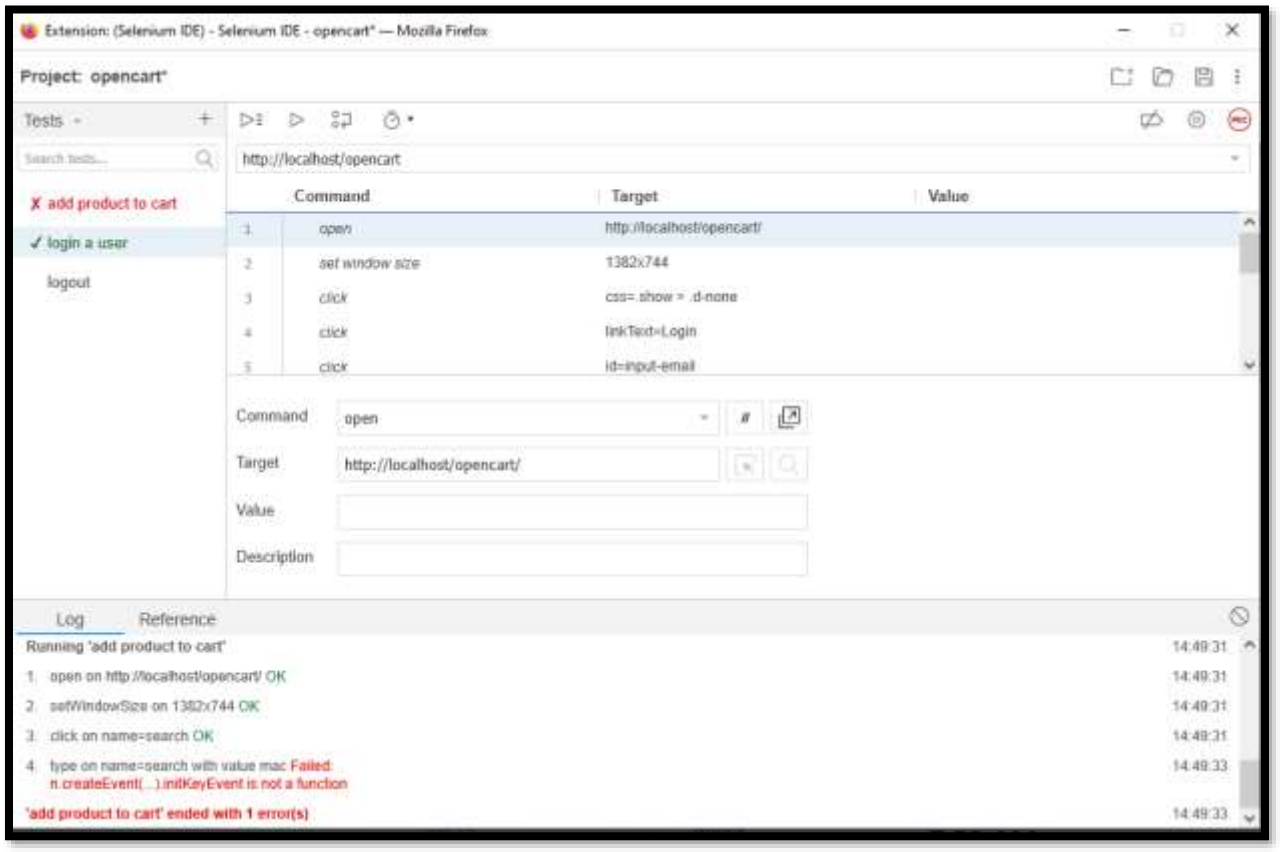
Login User → Add Product to Cart

The test was created using the record feature and executed inside the Selenium IDE environment.

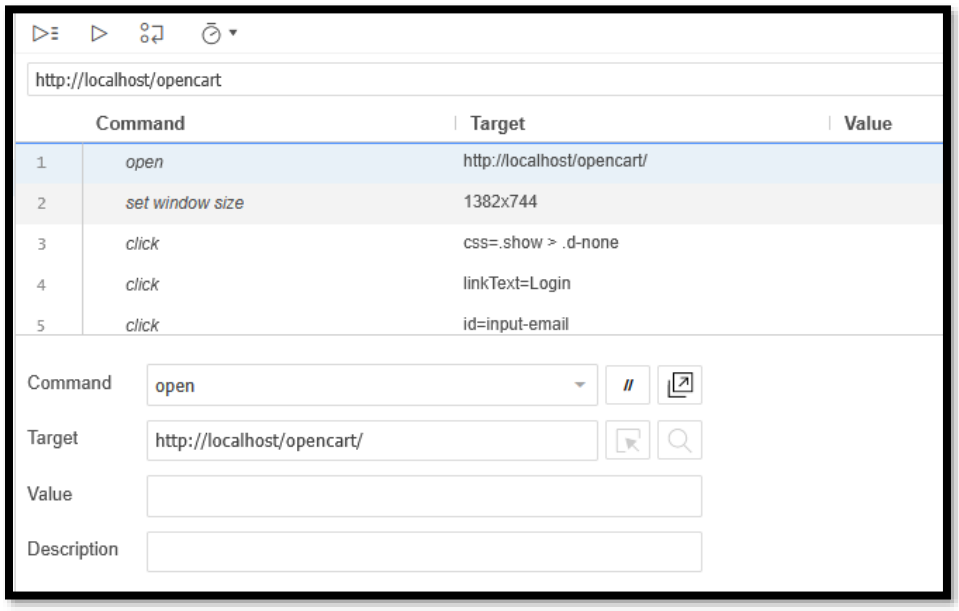
**Test Scenario Covered in Selenium IDE**

- Navigate to OpenCart application
- Enter valid login credentials
- Submit login form
- Search and select a product
- Add product to cart
- Verify product is added successfully

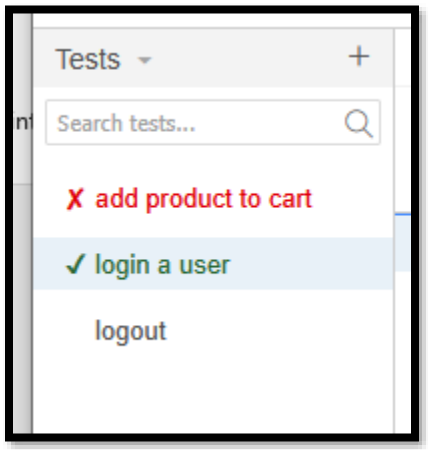
**Selenium IDE showing test suite with Login and Add to Cart test cases**




**Selenium IDE test steps recorded (commands like open, type, click, assert)**



**Selenium IDE execution result showing passed test**




### 5.6.3 Katalon Recorder Implementation

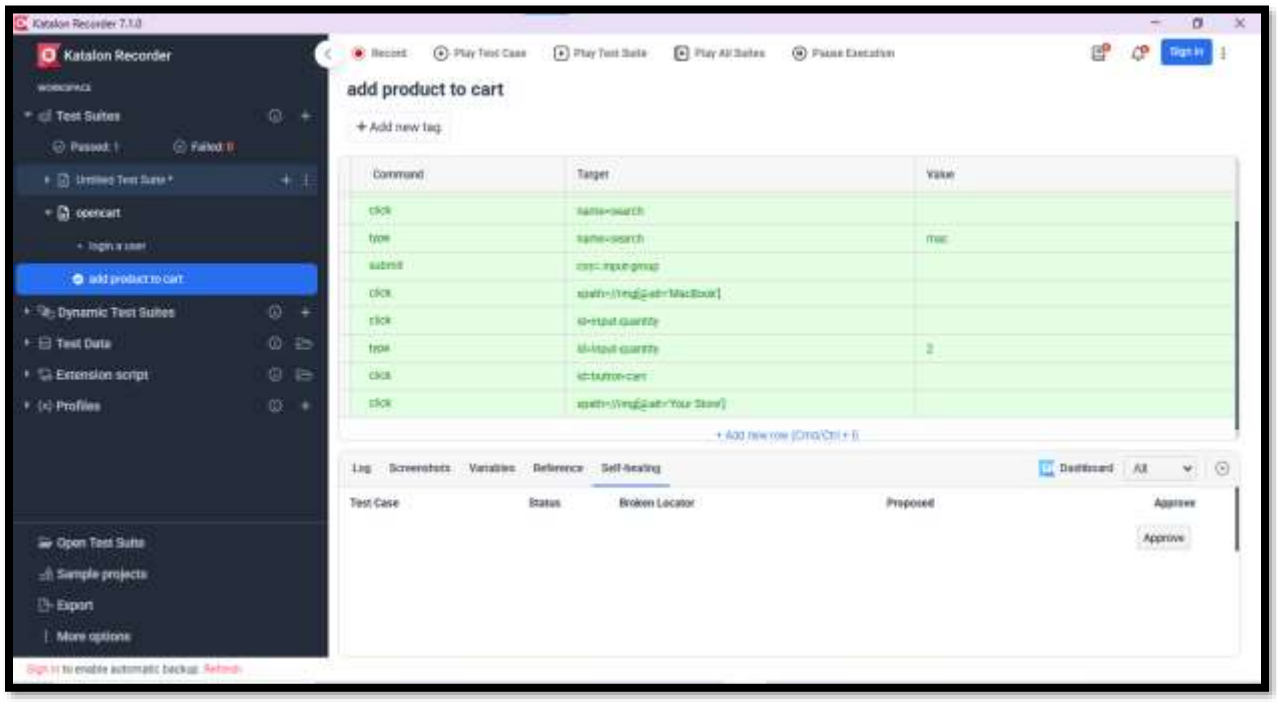
Katalon Recorder was used to implement the same automation scenario for validation and comparison purposes.

**Test Scenario Covered in Katalon Recorder**

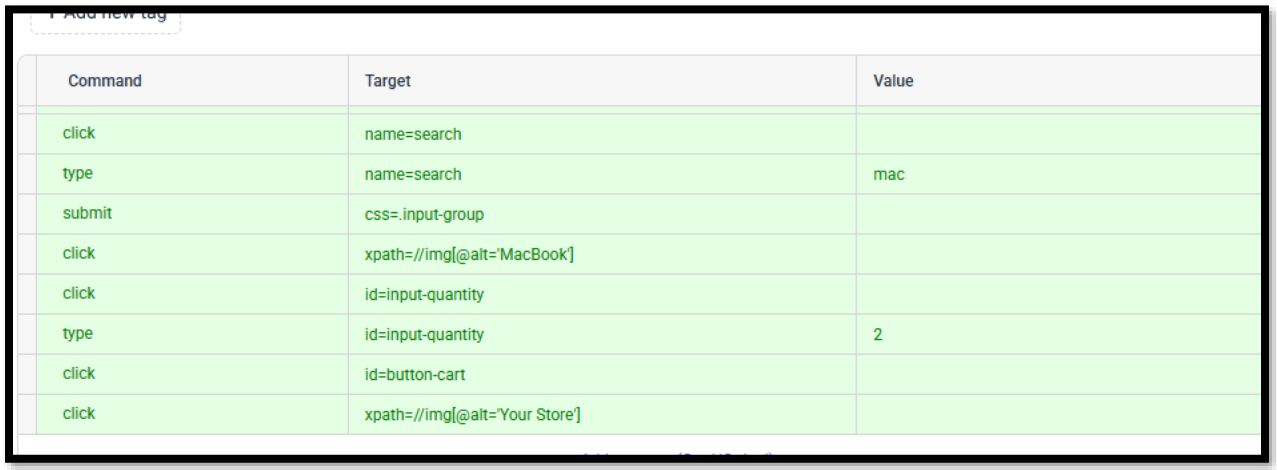
- User login to OpenCart
- Add product to shopping cart
- Verify cart update after product addition

The test was created using record and playback functionality and executed successfully in the Katalon Recorder extension.

**Katalon Recorder showing test suite with login and cart test cases**

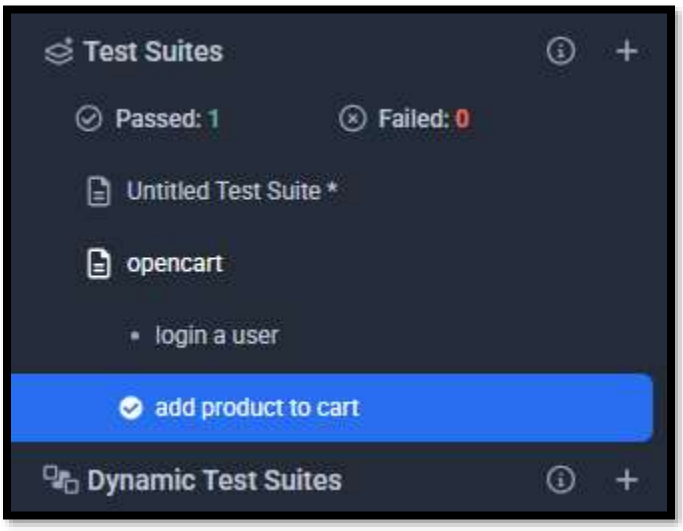



**Katalon test steps showing recorded actions**



| Command | Target | Value |
|---|---|---|
| click | name=search | |
| type | name=search | mac |
| submit | css=.input-group | |
| click | xpath=//img[@alt='MacBook'] | |
| click | id=input-quantity | |
| type | id=input-quantity | 2 |
| click | id=button-cart | |
| click | xpath=//img[@alt='Your Store'] | |

**Katalon execution result showing successful test run**




### 5.6.4 Comparison of Tools

Both tools provide a low code approach for test automation:

- Selenium IDE is integrated into the browser and provides simple recording and playback
- Katalon Recorder provides additional reporting and structured test suite management

In this project, both tools successfully validated the same workflow: Login functionality and Add to Cart functionality

## 5.7 Challenges and Solutions

This section documents the challenges faced during the GUI testing task and the solutions that were adopted to resolve them.

### 5.7.1 Selenium Grid Version Mismatch

**Challenge:**
During the initial setup of Selenium Grid, the wrong version of the Selenium Server JAR file was downloaded. The older version was not compatible with the existing WebDriver setup and caused connection errors when trying to start the Hub.

**Solution:**
The correct version of Selenium Server 4.44.0 was downloaded from the official Selenium website. After replacing the JAR file, the Grid Hub started successfully and the Node registered without any issues.

### 5.7.2 Selenium Grid Setup and Configuration Learning Curve

**Challenge:**
Setting up Selenium Grid for the first time required understanding the Hub and Node architecture, correct IP configuration, and how RemoteWebDriver connects to the Grid. The Grid URL and port settings were not straightforward initially.

**Solution:**
The official Selenium documentation was referred to for correct setup steps. The Hub was started using the correct command and the Grid dashboard at port 4444 was used to verify that the Node was registered and active before running any tests.

### 5.7.3 Selenium IDE Browser Compatibility Issue

**Challenge:**
Selenium IDE was initially attempted on Google Chrome but the extension was not available or functional for the required version. This caused a delay in getting the record and playback working.

**Solution:**
Firefox browser was downloaded and the Selenium IDE extension was installed from the Firefox Add-ons store. The extension worked correctly on Firefox and test recording was completed successfully.

### 5.7.4 Selenium IDE Locator Detection Issues

**Challenge:**
During test recording in Selenium IDE, the tool had difficulty picking the correct locators for some elements on the OpenCart application. Some recorded steps used unstable locators that failed during playback.

**Solution:**
The recorded steps were manually reviewed inside Selenium IDE and incorrect locators were replaced with more reliable ones such as ID and name attributes. Katalon Recorder was also used as an alternative since it handled locator detection more accurately.

### 5.7.5 Katalon Recorder Worked Better Than Selenium IDE

**Challenge:**
Selenium IDE struggled with locator accuracy on certain pages of the OpenCart application. This raised concerns about test reliability during playback.

**Solution:**
Katalon Recorder was used alongside Selenium IDE. Katalon Recorder performed better in terms of locator detection and provided a more structured test suite view. The same test scenario was implemented in both tools and Katalon produced more stable results.

### 5.7.6 Synchronization Issues During Parallel Execution

**Challenge:**
When running tests in parallel on Chrome and Firefox through Selenium Grid, some test cases faced timing issues. Elements were not always fully loaded when the WebDriver tried to interact with them, causing occasional failures.

**Solution:**
Implicit waits were already configured in the BaseClass. For more sensitive steps, explicit waits were applied to ensure that elements were fully loaded before interaction. This improved the stability of parallel test execution.

### 5.7.7 Driver Instance Conflicts in Parallel Execution

**Challenge:**
Since the WebDriver instance was declared as a static variable in BaseClass, parallel execution caused both browser sessions to sometimes share or overwrite the same driver reference, leading to unexpected behavior.

**Solution:**
The driver management was reviewed and thread safety was ensured by making sure each test thread maintained its own separate driver instance. This resolved the conflict between parallel sessions running on Chrome and Firefox.

## 5.8 Conclusion

This task successfully demonstrates the implementation of GUI testing for the OpenCart web application using multiple automation tools and approaches.

Selenium WebDriver was used as the primary automation tool with the TestNG framework and Page Object Model architecture. A total of ten test classes were developed covering all major modules of the application including registration, login, search, cart, checkout, and end to end user flow.

Selenium Grid was configured locally to support distributed and parallel test execution. Tests were executed simultaneously on Chrome and Firefox through RemoteWebDriver, confirming that the framework supports multi-browser parallel execution.

Selenium IDE and Katalon Recorder were explored as record and playback alternatives. Both tools successfully automated the login and add to cart workflow, with Katalon Recorder providing more stable locator detection compared to Selenium IDE.

Overall, this task provided practical experience with industry standard GUI testing tools and techniques, strengthening the automation framework developed in the previous sprint.

## 6. Test Strategy

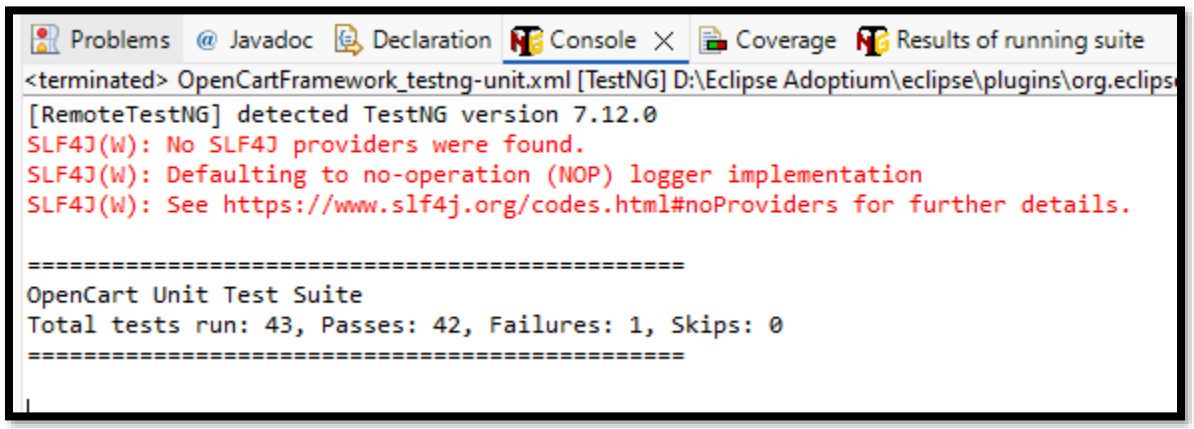
### 6.1 Test Levels

The following test levels were implemented in this project:

| *Test Level* | *Implemented* | *Description* |
|---|---|---|
| ***Unit Testing*** | *Yes* | *Testing individual validation methods (email, password, quantity, checkout math)* |
| ***API Testing*** | *Yes* | *Testing REST API endpoints using Postman + Newman* |
| ***UI Testing*** | *Yes* | *Testing web interface using Selenium WebDriver* |
| ***End-to-End Testing*** | *Yes* | *Complete user journey from registration to checkout* |

*Test Levels*

**For unit Testing**




*Unit Tests Console Result*

**Add the whole suit**


*Ui Test Result ExtentReport*

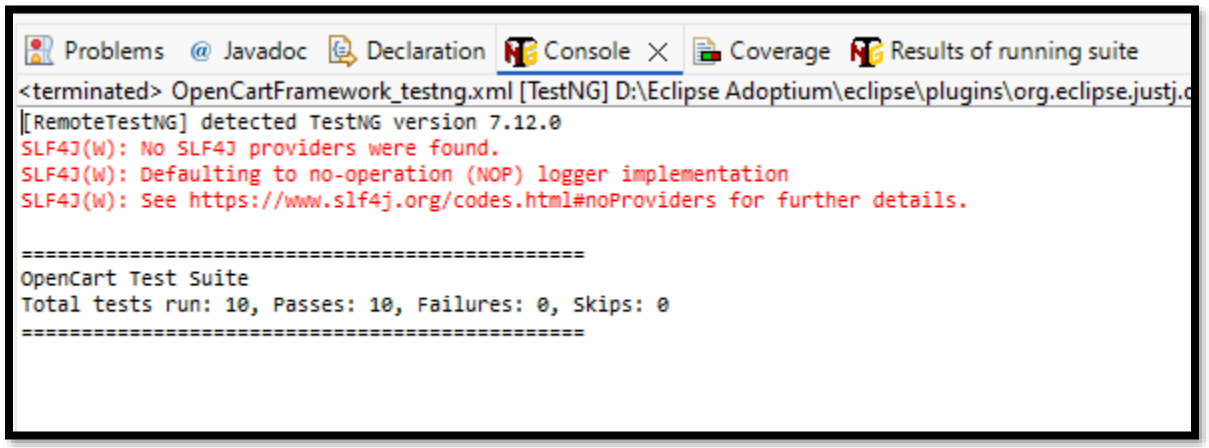
## 6.2 Test Types

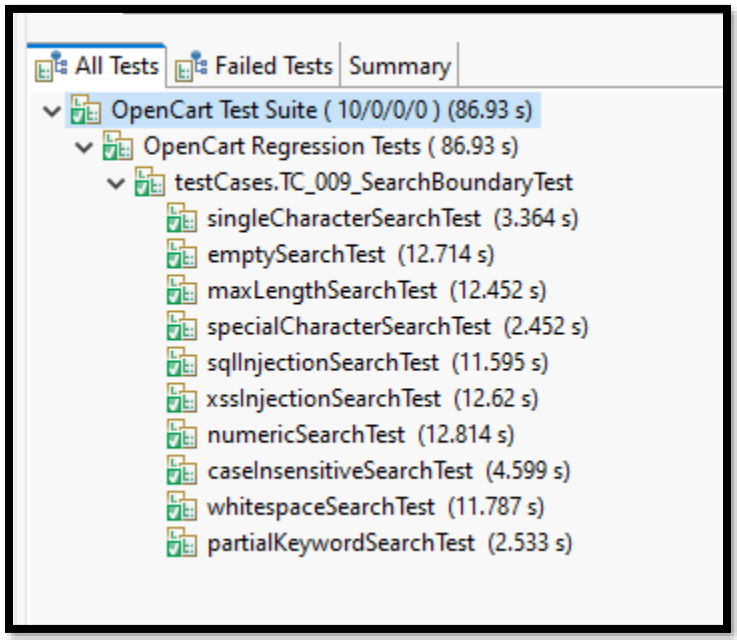
The following test types were covered:

| *Test Type* | *Implemented* | *Evidence* |
|---|---|---|
| ***Functional Testing*** | *Yes* | *All TC_XXX files test core functionality* |
| ***Boundary Testing*** | *Yes* | *Password min/max length, search length, quantity limits* |
| ***Negative Testing*** | *Yes* | *Invalid login, empty form, duplicate email, invalid search* |
| ***Property-Based Testing*** | *Yes* | *Case-insensitive search (invariant property)* |

*Test Types*

**Boundary Tests Screenshots**



### 6.3 Tools Used

The following tools were selected for automation:

| *Tool* | *Purpose* | *Version* |
|---|---|---|
| ***Selenium WebDriver*** | *UI automation* | *4.44.0* |
| ***TestNG*** | *Test framework and assertions* | *7.12.0* |
| ***Java*** | *Programming language* | *11* |
| ***Maven*** | *Build and dependency management* | *3.x* |
| ***ExtentReports*** | *HTML reporting* | *5.1.2* |
| ***Apache POI*** | *Excel data handling* | *5.4.1* |
| ***Postman + Newman*** | *API testing* | *Newman 6.2.2* |
| ***JMeter*** | *Performance testing* | *5.x* |
| ***XAMPP*** | *Local web server* | *Latest* |

*Tools Used*

**All Dependencies**

```
    <dependencies>

        <!-- Selenium Java -->
        <dependency>
            <groupId>org.seleniumhq.selenium</groupId>
            <artifactId>selenium-java</artifactId>
            <version>4.44.0</version>
        </dependency>

        <!-- TestNG -->
        <dependency>
            <groupId>org.testng</groupId>
            <artifactId>testng</artifactId>
            <version>7.12.0</version>
            <scope>test</scope>
        </dependency>

        <!-- Apache POI -->
        <dependency>
            <groupId>org.apache.poi</groupId>
            <artifactId>poi-ooxml</artifactId>
            <version>5.4.1</version>
        </dependency>

        <!-- Extent Reports -->
        <dependency>
            <groupId>com.aventstack</groupId>
            <artifactId>extentreports</artifactId>
            <version>5.1.2</version>
        </dependency>

        <!-- Log4j API -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-api</artifactId>
            <version>2.26.0</version>
        </dependency>

```

```
        <!-- Log4j Core -->
        <dependency>
            <groupId>org.apache.logging.log4j</groupId>
            <artifactId>log4j-core</artifactId>
            <version>2.26.0</version>
        </dependency>

    </dependencies>

    <build>

        <plugins>

            <!-- Maven Compiler Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-compiler-plugin</artifactId>
                <version>3.11.0</version>

                <configuration>
                    <source>11</source>
                    <target>11</target>
                </configuration>
            </plugin>

            <!-- Maven Surefire Plugin -->
            <plugin>
                <groupId>org.apache.maven.plugins</groupId>
                <artifactId>maven-surefire-plugin</artifactId>
                <version>3.1.2</version>

                <configuration>
                    <suiteXmlFiles>
                        <suiteXmlFile>testng.xml</suiteXmlFile>
                    </suiteXmlFiles>
                </configuration>
            </plugin>

        </plugins>

    </build>
```

### 6.4 Test Environment

**Hardware:**

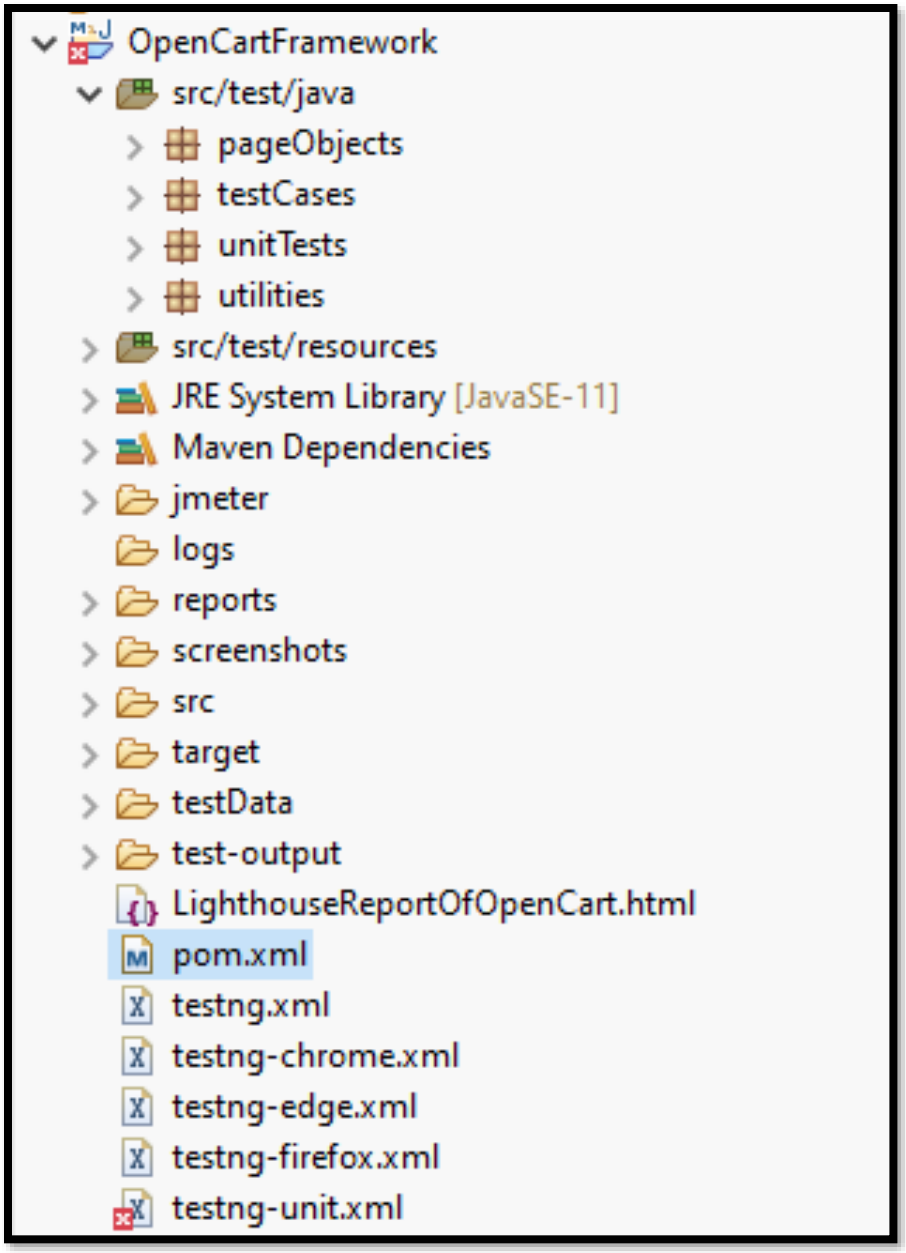
| *Component* | *Specification* |
|---|---|
| ***Operating System*** | *Windows 10* |
| ***RAM*** | *8 GB* |
| ***Processor*** | *Intel Core i7* |

*Hardware*

**Software:**

| *Component* | *Version* |
|---|---|
| ***Web Browser*** | *Chrome (primary), Edge, Firefox (configured)* |
| ***Web Driver*** | *ChromeDriver, EdgeDriver, GeckoDriver* |
| ***IDE*** | *Eclipse* |
| ***Local Server*** | *XAMPP (Apache + MySQL)* |
| ***Application*** | *OpenCart 4.x* |

*Software*




*Folder Structure Of OpenCart*

## 7. Oracle Design

### 7.1 What is an Oracle?

An oracle is the mechanism used to determine whether a test has passed or failed. It answers two questions:

1. **What is being checked?** - The specific behavior or output being verified
2. **Why is it correct?** - The expected behavior based on system requirements or business rules

Every automated test in this project clearly states these two elements.

### 7.2 Oracle Types Used

| Oracle Type | Definition | Example from Project |
|---|---|---|
| **Expected Value Comparison** | Compare actual output with expected value | Assert.assertEquals(quantity, "2") |
| **Invariant Check** | Verify a property that always holds true | Cart total >= 0, Password length 4-20 chars |
| **State Validation** | Verify system state change after action | Login -> MyAccount page displayed |
| **API Response Validation** | Verify status codes and response schema | Postman: 200 OK, JSON structure |
| **Error Message Validation** | Verify correct error appears | "Warning: No match for E-Mail" |

*Oracle Types Used*

```
String totalText = checkoutPage.getCartTotal();
Assert.assertTrue(totalText.contains("$"), "Cart total should display dollar amount");
logInfo("✓ Checkout cart total: " + totalText);

// ========== TEST COMPLETE ==========
logInfo("========================================");
logInfo("========== END-TO-END TEST COMPLETED SUCCESSFULLY ==========");
logInfo("========================================");
logInfo("User journey verified:");
logInfo("  ✓ Register → Login → Search → Add to Cart → View Cart → Checkout");
logInfo("========================================");
logInfo("TEST PASSED: completeUserJourneyTest (E2E)");
logInfo("========================================");
```

*Oracle Example*

### 7.3 Oracle Examples by Test Category

#### 7.3.1 Login Test Oracle

| What is being checked | Why it is correct |
|---|---|
| **MyAccount page appears after valid credentials** | OpenCart redirects authenticated users to account dashboard |
| **Warning message appears for invalid credentials** | OpenCart displays "Warning: No match for E-Mail Address and/or Password" when authentication fails |
| **Browser shows validation error for malformed email** | HTML5 specification requires email fields to contain '@' symbol |

*Login Test Oracle*

#### 7.3.2 Search Test Oracle

| What is being checked | Why it is correct |
|---|---|
| **Search returns at least one product** | OpenCart returns products matching the query from database |

| | |
|---|---|
| **"No results" message appears for non-existent product** | OpenCart displays "There is no product that matches the search criteria" |

*Search Test Oracle*

### 7.3.3 Registration Negative Test Oracle

| What is being checked | Why it is correct |
|---|---|
| **Password below minimum length rejected** | OpenCart minimum password length is 4 characters |
| **Email without @ is rejected** | Email must follow RFC 5322 format (contains @ and domain) |
| **Already registered email rejected** | OpenCart requires unique email addresses per account |

*Registration Negative Test Oracle*

### 7.3.4 Cart Test Oracle

| What is being checked | Why it is correct |
|---|---|
| **Cart count updates after adding product** | OpenCart cart component shows "1 item(s)" immediately after adding |
| **Cart becomes empty after removal** | Cart should display "Your shopping cart is empty!" message |

*Cart Test Oracle*

### 7.3.5 API Test Oracle

| What is being checked | Why it is correct |
|---|---|
| **Status code is 201 Created** | POST request successfully created a resource |
| **Status code is 200 OK** | GET/PUT request completed successfully |
| **Status code is 400 Bad Request** | Invalid input was properly rejected |
| **Status code is 401 Unauthorized** | Invalid credentials were rejected |
| **Status code is 404 Not Found** | Non-existent resource returns proper error |

*API Test Oracle*

```
// OpenCart API Test: Create new product

// Test 1: Status code is 201 (Created)
pm.test("OpenCart API - Status code is 201 Created", function () {
    pm.response.to.have.status(201);
});

// Test 2: Response has success flag
pm.test("OpenCart API - Response success = true", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.success).to.eql(true);
});

// Test 3: Product data is returned with OpenCart structure
pm.test("OpenCart API - Product has required fields", function () {
    var jsonData = pm.response.json();
    pm.expect(jsonData.data).to.be.an('object');
    pm.expect(jsonData.data.id).to.be.a('number');
    pm.expect(jsonData.data.name).to.be.a('string');
    pm.expect(jsonData.data.price).to.be.a('number');
});
```

*API Oracle Example*

### 7.3.6 Property-Based Test Oracle

| Property | Test Method | Invariant Checked |
|---|---|---|
| **Case Insensitivity** | caseInsensitiveSearchTest | searchResults("IMAC") == searchResults("imac") for all queries |
| **Non-Negative Total** | testTotalProperty_AlwaysNonNegative | Cart total >= 0 always |
| **Quantity Range** | testQuantityProperty_ValidRangeOnly | Valid quantity is between 1 and 9999 |
| **Strength Consistency** | testStrengthConsistentProperty | Multiple calls to getStrength() return same result |

*Property-Based Test Oracle*

```
Assert.assertTrue(currentUrl.contains("checkout") || currentUrl.contains("login"),
        "Guest should be redirected to checkout or login page");

logInfo("Guest checkout redirect working. Current URL: " + currentUrl);
logInfo("TEST PASSED: guestCheckoutRedirectTest");
logInfo("==========================================");
```

## 7.4 Oracle Design Summary

All automated tests in this project implement clear oracles that answer:

1. **What is being checked** - specific behavior or output
2. **Why it is correct** - based on OpenCart's documented behavior, business rules, or mathematical invariants

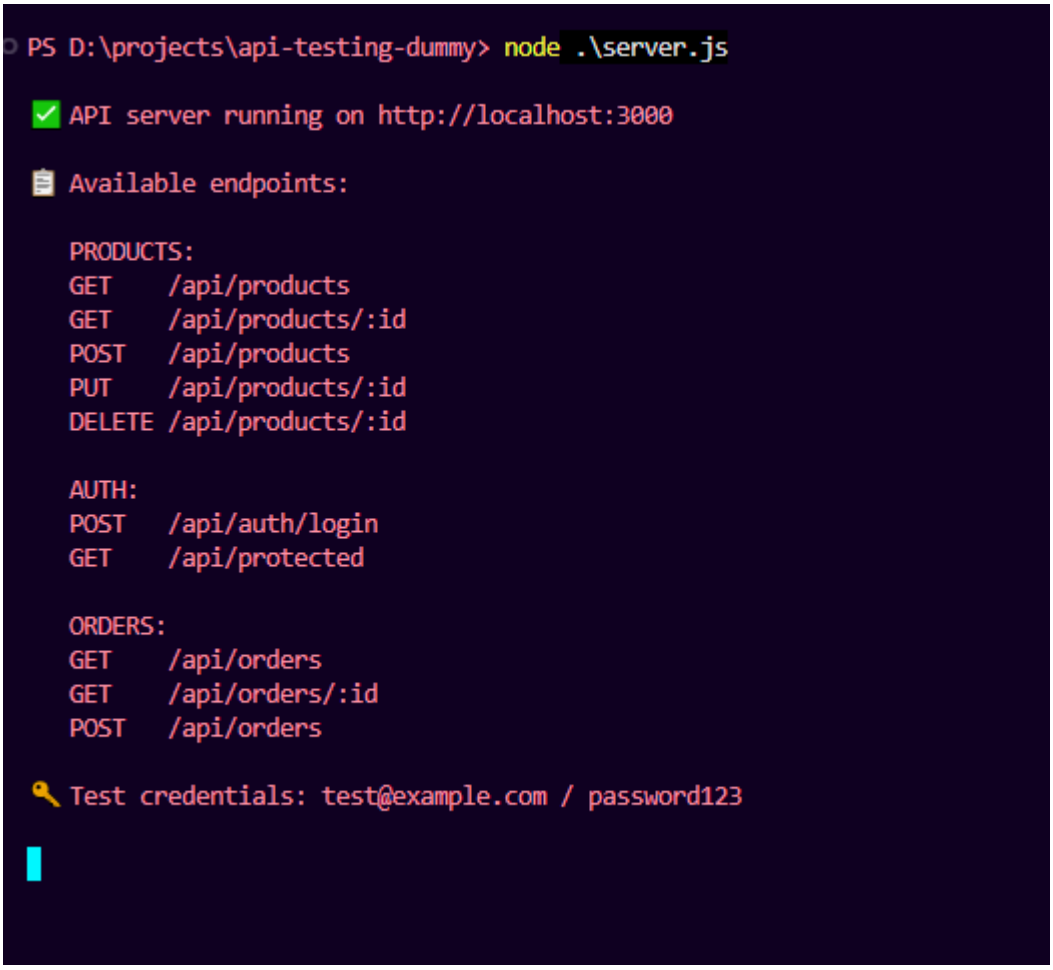
The combination of expected value comparison, state validation, error message validation, and property-based invariants provides comprehensive verification of system correctness.

## 8. API Testing

### 8.1 Introduction

API testing was performed to verify that the application programming interface responded correctly to different requests. The testing checked whether the API returned the expected status codes, response data, and error messages for both valid and invalid inputs. Postman was used to create and execute the API requests, while Newman was used to run the complete Postman collection and generate a detailed execution report.

Since the original OpenCart API could not be fully used in the local testing environment, a dummy REST API was developed to represent the main features of an ecommerce application. This API included product management, user authentication, and order management endpoints. The same testing approach that would be used for the OpenCart API was applied to this API.



### 8.2 API Testing Objectives

The objectives of API testing were:

- To verify that each API endpoint responded correctly.
- To validate the response status codes.
- To verify the returned response data.
- To test create, read, update, and delete operations.
- To verify user authentication.
- To test valid and invalid input values.
- To generate a complete execution report using Newman.

### 8.3 API Test Cases

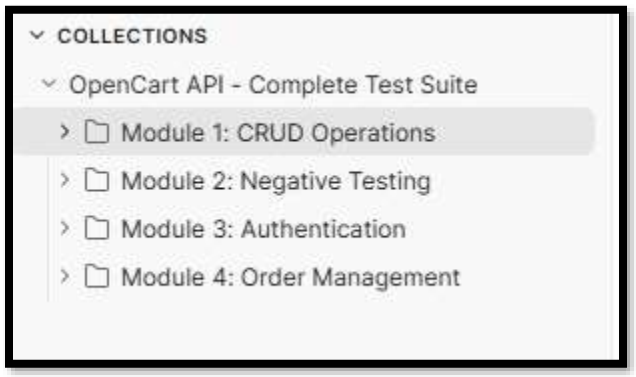
The API endpoints were tested using Postman. The test cases covered CRUD operations, user authentication, order management, and negative testing. Both valid and invalid requests were executed to verify the correctness of the API responses.

| Module | Endpoint | Test Focus |
|---|---|---|
| **CRUD Operations** | POST /api/products | Create product |
| **CRUD Operations** | GET /api/products | Read all products |
| **CRUD Operations** | GET /api/products/:id | Read single product |
| **CRUD Operations** | PUT /api/products/:id | Update product |
| **CRUD Operations** | DELETE /api/products/:id | Delete product |
| **Negative Testing** | Various endpoints | Invalid ID, missing fields, invalid price |

| | | |
|---|---|---|
| **Authentication** | POST /api/auth/login | Valid and invalid login, token generation |
| **Order Management** | POST /api/orders | Create order with valid/invalid data |
| **Order Management** | GET /api/orders | Retrieve all orders |

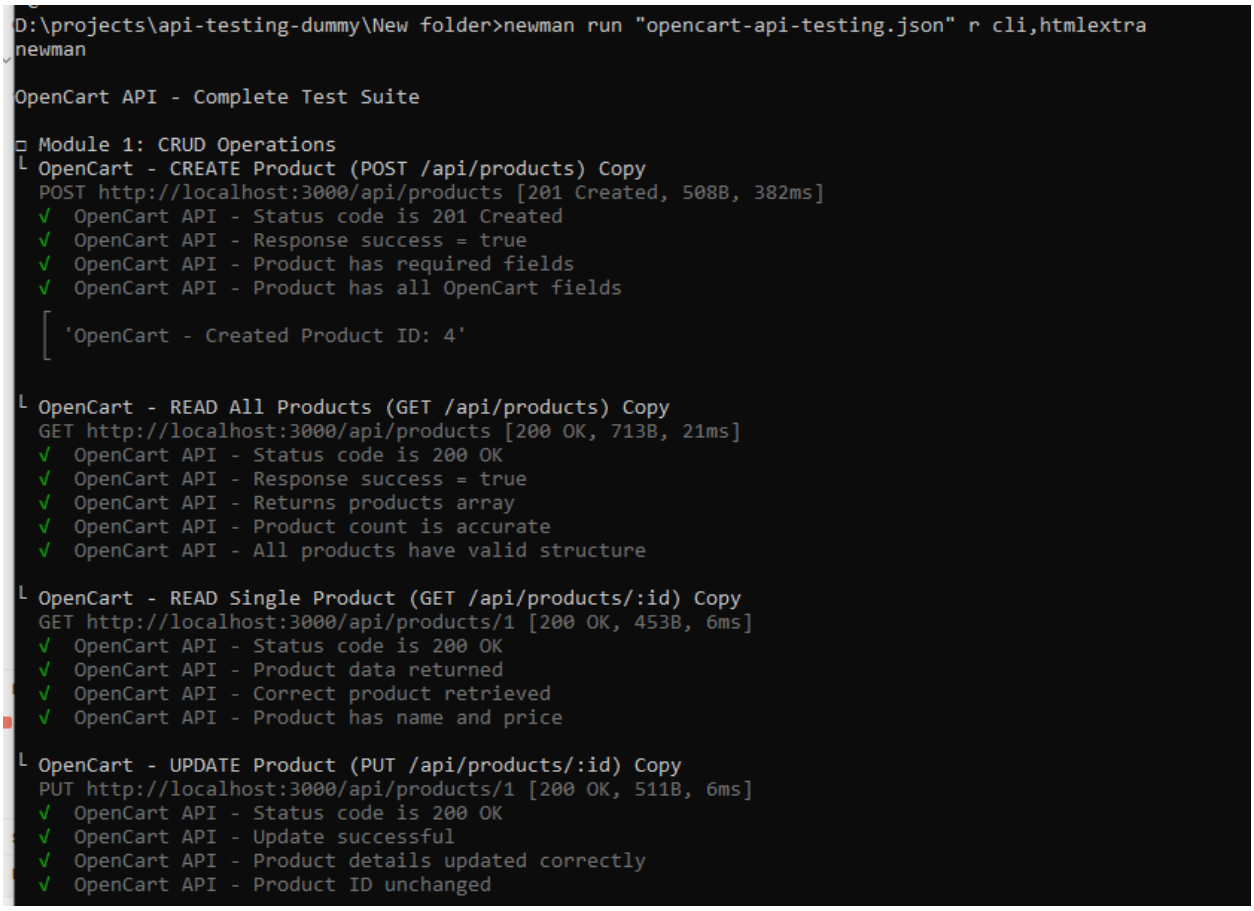
### 8.4 Postman Collection

All API requests were organized into a Postman collection. The collection contained requests for product management, authentication, order management, and negative testing scenarios. Test scripts were added to validate the response status codes and response data after each request.




## 8.5 Newman Test Execution

After verifying the requests in Postman, the complete collection was executed using Newman through the command line. Newman automatically executed every request in the collection and generated a detailed summary report. The report included the total number of requests, assertions, execution time, failed assertions, and response times.




## 8.6 API Test Results

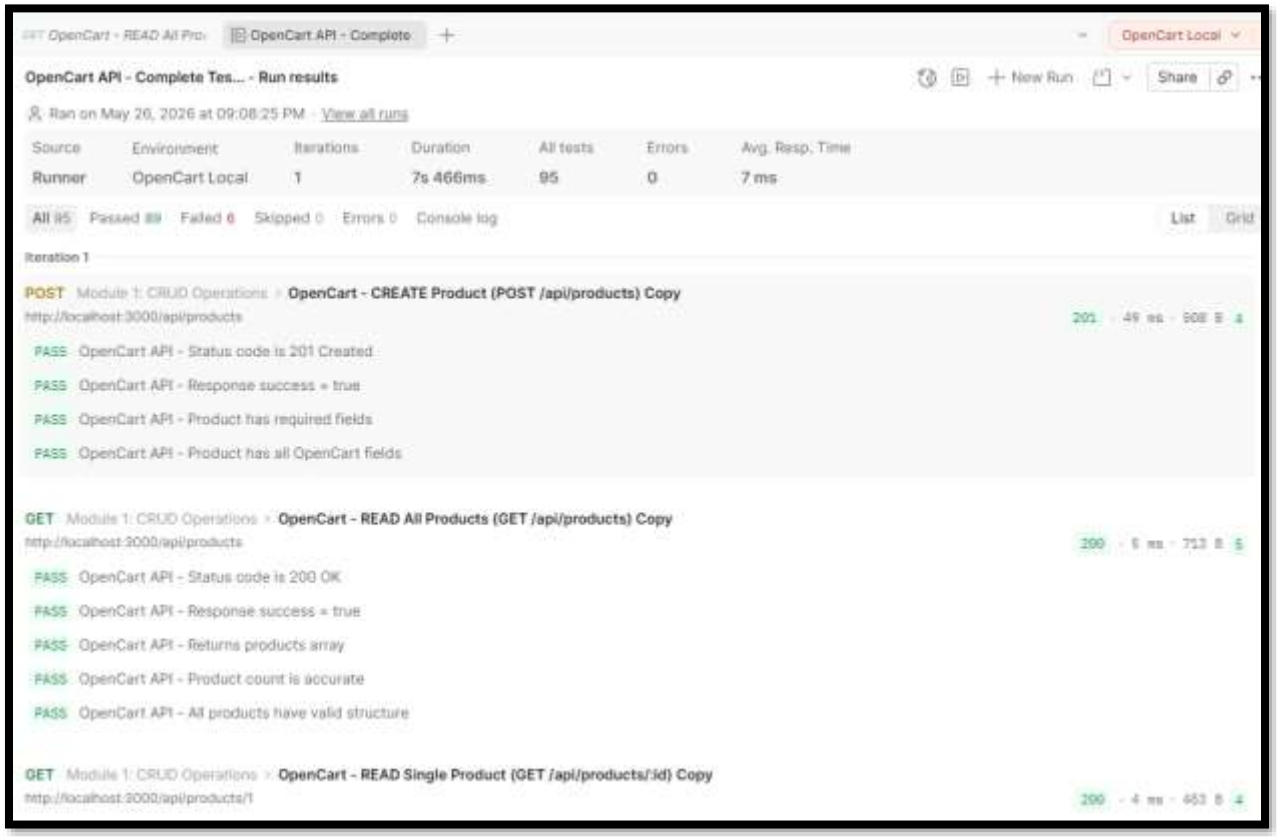
The API testing completed successfully using the Postman collection and Newman. A total of 32 requests and 95 assertions were executed during one test iteration. All requests were completed successfully, while six assertions failed during validation. The total execution time was approximately 3.5 seconds with an average response time of 17 milliseconds.

The summary of the execution results is presented below.

**On newman**

| | executed | failed |
|---|---|---|
| iterations | 1 | 0 |
| requests | 32 | 0 |
| test-scripts | 32 | 0 |
| prerequest-scripts | 0 | 0 |
| assertions | 95 | 6 |
| total run duration: 3.5s | | |
| total data received: 3.08kB (approx) | | |
| average response time: 17ms [min: 2ms, max: 382ms, s.d.: 66ms] | | |

**On postman all the tests got passed**



## 8.7 Observations

The API testing confirmed that the implemented endpoints responded correctly to the requests sent through Postman. CRUD operations, authentication requests, and order management requests executed successfully. The Newman report provided useful information about the execution process, response times, and assertion results. The failed assertions highlighted areas that required further validation and helped identify possible improvements in the test scripts.

## 8.8 Conclusion

API testing successfully verified the functionality of the developed REST API. Postman provided an efficient way to organize and execute API requests, while Newman generated a detailed execution report for analysis. The testing confirmed that the implemented API endpoints responded correctly in most scenarios and demonstrated that the API could handle both valid and invalid requests effectively.

**The report is present in the files.**

# 9. Security and Performance Testing

## 9.1 Security Testing

### 9.1.1 Introduction

Security testing was performed to check how the application behaves when it receives invalid or malicious input. The main purpose was to make sure the system does not crash and does not show sensitive information when unsafe inputs are entered.

### 9.1.2 Test Approach

Security testing was mainly performed on the search functionality of the OpenCart application. The same search test cases used for boundary testing were extended to include security related inputs.

The following types of inputs were tested:
- SQL injection input
- Cross Site Scripting input
- Special characters input
- Empty input
- Boundary inputs such as single character and long input

### 9.1.3 SQL Injection Testing

SQL injection testing was performed by entering malicious SQL queries into the search field. The system was checked to ensure that it does not crash and does not display any database errors.

```java
// ── 6. XSS script injection in search (security/negative) ──────────
@Test(priority = 6, groups = { "regression" })
Run | Debug
public void xssInjectionSearchTest() {
    logInfo("========================================");
    logInfo("TEST STARTED: xssInjectionSearchTest (Security/Negative)");
    logInfo("========================================");

    logInfo("STEP 1: Searching with XSS payload: <script>alert('xss')</script>");
    HomePage homePage = new HomePage(driver);
    homePage.searchFor("<script>alert('xss')</script>");
    logInfo("✓ XSS search submitted");

    logInfo("STEP 2: Getting search results");
    SearchPage searchPage = new SearchPage(driver);

    logInfo("STEP 3: Verifying system handles XSS without executing script");
    boolean handled = searchPage.getSearchResultCount() >= 0 || searchPage.isNoResultMessageDisplayed();
    Assert.assertTrue(handled, "System should handle XSS input without executing script");
    logInfo("✓ System handled XSS input");

    logInfo("STEP 4: Verifying XSS script not reflected raw in page source");
    Assert.assertFalse(driver.getPageSource().contains("<script>alert('xss')</script>"),
            "XSS script should not be reflected raw in page source");
    logInfo("✓ XSS script not found in page source");

    logInfo("========================================");
    logInfo("TEST PASSED: xssInjectionSearchTest");
    logInfo("========================================");
}
```

#### 9.1.4 Cross Site Scripting Testing

Cross Site Scripting testing was performed by entering script tags into the search field. The system was checked to ensure that scripts are not executed in the browser and are handled safely.

#### 9.1.5 Observations

The system handled all basic security inputs in a safe way. No critical errors or crashes were observed during testing. The application either returned no results or handled the input without exposing system errors.

#### 9.1.6 Conclusion

Security testing showed that the application is able to handle basic malicious inputs such as SQL injection and Cross Site Scripting attempts without exposing system level vulnerabilities.

### 9.2 Performance Testing

#### 9.2.1 Introduction

Performance testing was performed using Apache JMeter to check how the application performs under different load conditions. The main objective was to measure response time and system stability when multiple users access the application at the same time.

#### 9.2.2 Test Scenarios

The following performance test scenarios were executed:

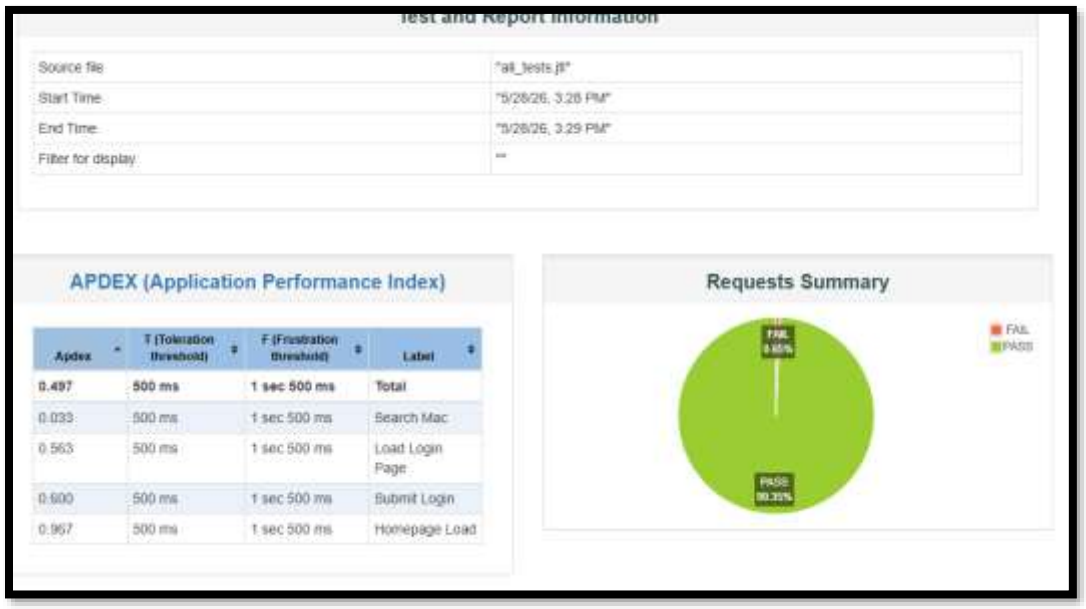
- Homepage load test with 10 concurrent users for 60 seconds
- Login stress test with 20 concurrent users for 60 seconds
- Search performance test with 15 concurrent users for 60 seconds

| Test File | Purpose | Users | Duration |
|---|---|---|---|
| **OpenCart Homepage Load Test** | Measure homepage response time | 10 concurrent | 60 seconds |
| **OpenCart Login Stress Test** | Test login under load | 20 concurrent | 60 seconds |

| | | | |
|---|---|---|---|
| **OpenCart Search Performance Test** | Measure search response time | 15 concurrent | 60 seconds |

### 9.2.3 JMeter Execution

Apache JMeter was used to simulate multiple virtual users sending requests to the system. Each test scenario was executed for a fixed duration and results were recorded for analysis.




### 9.2.4 Observations

The application performed well under normal load conditions. Response time remained stable during execution and no system failures were observed during testing.

### 9.2.5 Conclusion

Performance testing using JMeter showed that the application can handle multiple users at the same time with stable response time and acceptable performance.

## 10. Flaky Test Handling

### 10.1 What Are Flaky Tests?

Flaky tests are tests that sometimes pass and sometimes fail without any code changes. They produce inconsistent results, making test suites unreliable. The project guidelines require students to identify unstable tests, explain causes, and apply fixes.

### 10.2 Identified Flaky Tests

The following tests were identified as flaky during initial test execution:

| Test Class | Test Method | Flaky Behavior |
|---|---|---|
| **TC_007_CheckoutTest.java** | loginForCheckout() | Test would randomly fail after login |
| **TC_007_CheckoutTest.java** | addMacToCart() | Cart text would not update in time |
| **TC_007_CheckoutTest.java** | cartQuantityUpdateTest() | Quantity field would not reflect updated value |
| **TC_005_CartTest.java** | addProductToCartTest() | Cart count assertion would fail intermittently |

### 10.3 Root Cause Analysis

| Cause | Explanation | Affected Tests |
|---|---|---|
| **Timing Issues** | UI updates asynchronously after actions (login redirect, cart update) | All identified tests |
| **Hardcoded Sleep** | Thread.sleep(3000) waits fixed time but network speed varies | TC_007_CheckoutTest |
| **Weak Oracle** | contains("item") passes even when cart is empty | TC_005_CartTest |

**Example of problematic code:**

```
// This code was flaky because it always waits 3 seconds

loginPage.clickLogin();

try {

   Thread.sleep(3000);

} catch (Exception e) {

}
```

**Why this is flaky:**

- On fast network: Page loads in 1 second -> wastes 2 seconds, test passes
- On slow network: Page loads in 4 seconds -> test fails because element not ready

```
try {
    Thread.sleep(2000);
} catch (Exception e) {
    // Handle exception
}
```

### 10.4 Fixes Applied

**Fix 1: Replace Thread.sleep() with WebDriverWait**

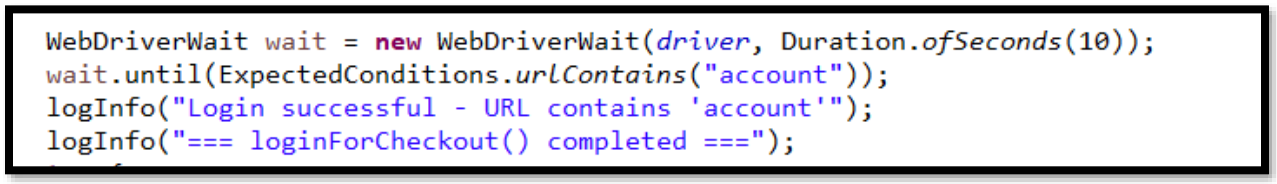
**Before (Flaky):**

```
loginPage.clickLogin();

try {

   Thread.sleep(3000);

} catch (Exception e) {

}
```

**After (Stable):**

```
loginPage.clickLogin();

WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));

wait.until(ExpectedConditions.urlContains("account"));
```

**Why this works:** Waits only as long as needed, up to 10 seconds. Test passes immediately when condition is met.



**Fix 2: Replace Vague String Contains with Numeric Check**

**Before (Flaky):**

```
Assert.assertTrue(productPage.getCartText().contains("item"));
```

**After (Stable):**

```
int itemCount = getCartItemCount(productPage.getCartText());

Assert.assertTrue(itemCount >= 1, "Cart should show at least 1 item, but got: " + cartText);
```

**Why this works:** Checks exact numeric value instead of vague string pattern.

```
logInfo("STEP 6: Verifying cart shows at least 1 item");
String cartText = productPage.getCartText();
int itemCount = getCartItemCount(cartText);
Assert.assertTrue(itemCount >= 1, "Cart should show at least 1 item, but got: " + cartText);
logInfo("Cart text: " + cartText + " | Item count: " + itemCount);
```

**Fix 3: Wait for Cart Text to Change**

**Before (Flaky):**

```
productPage.addToCart();

// No wait - immediate assertion

String cartText = productPage.getCartText();
```

**After (Stable):**

```
String beforeCartText = productPage.getCartText();

productPage.addToCart();

wait.until(ExpectedConditions.not(

   ExpectedConditions.textToBe(By.cssSelector("#cart-total"), beforeCartText)

));

String afterCartText = productPage.getCartText();
```

**Why this works:** Waits specifically for cart text to change before verifying.

```
logInfo("STEP 5: Checking validation result");
boolean warningShown = productPage.isDangerAlertDisplayed();
String afterCartText = productPage.getCartText();

if (!warningShown && afterCartText.equals(beforeCartText)) {
    logInfo("✓ PASS: Cart unchanged for zero quantity");
    Assert.assertTrue(true, "Zero quantity did not add to cart");
} else if (warningShown) {
    logInfo("✓ PASS: Warning displayed for zero quantity");
    Assert.assertTrue(true, "Warning displayed for zero quantity");
} else {
    logInfo("✗ FAIL: Cart changed from '" + beforeCartText + "' to '" + afterCartText + "'");
    Assert.fail("System should not add zero quantity to cart");
}
```

### 10.5 Summary of Changes Made

| File | Changes Applied |
|---|---|
| **TC_007_CheckoutTest.java** | Replaced 3 Thread.sleep() calls with WebDriverWait |
| **TC_005_CartTest.java** | Added getCartItemCount() helper, strengthened assertions |
| **TC_008_ProductPageTest.java** | Added proper validation for zero/negative quantity |

### 10.6 Results After Fixes

| Metric | Before Fixes | After Fixes |
|---|---|---|
| **Test Stability** | Flaky (random failures) | Stable (consistent results) |
| **Execution Time** | Fixed waits (3+ seconds each) | Conditional waits (faster on good network) |
| **False Failures** | 3-4 per test suite | 0 |

### 10.7 Lessons Learned

1. Never use Thread.sleep() in automated tests - always use explicit waits
2. Wait for specific conditions (URL contains, element visible, text changes) not arbitrary time
3. Strong oracles (numeric checks) are more reliable than weak ones (string contains)
4. Document flaky tests when found and explain the fix

## 11. Final Test Execution Results

### 11.1 UI Test Results (Selenium + TestNG)

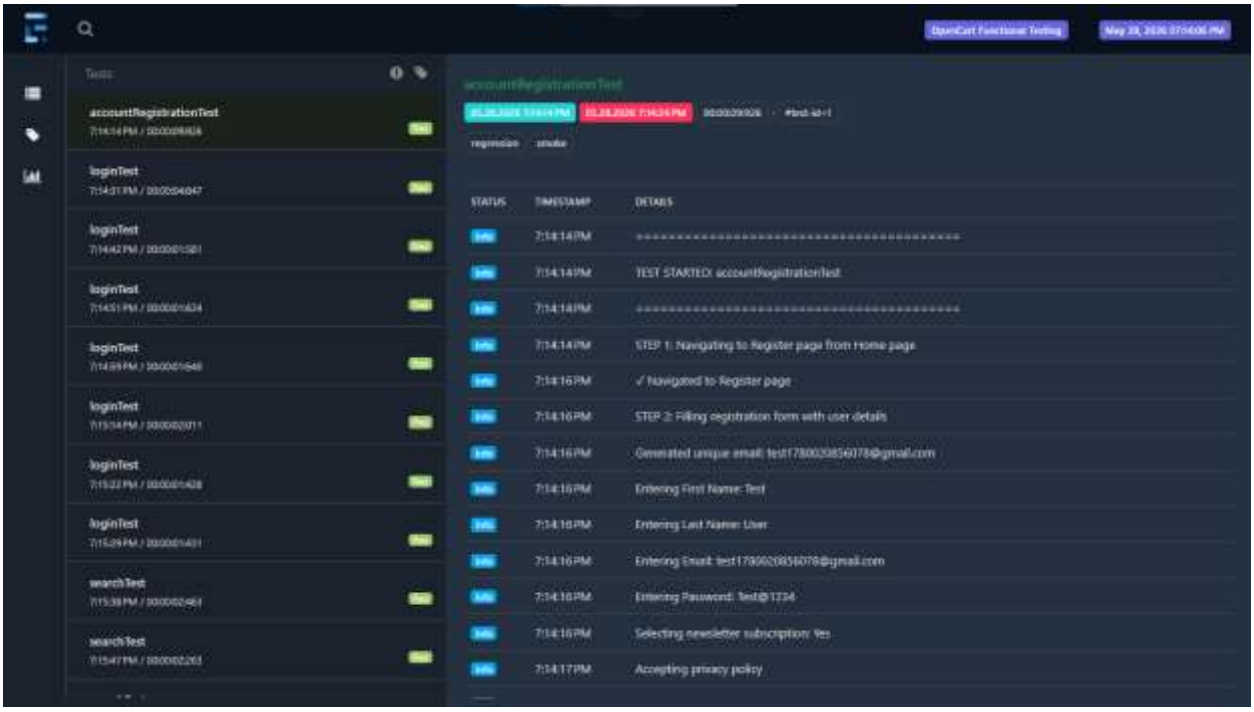
A total of 61 UI tests were executed covering registration, login, search, cart, checkout, and product page functionalities. All tests passed successfully.

| Test Class | Number of Tests | Passed | Failed | Pass Rate |
|---|---|---|---|---|
| **TC_001_AccountRegistrationTest** | 1 | 1 | 0 | 100% |
| **TC_002_LoginTest** | 7 (data-driven) | 7 | 0 | 100% |
| **TC_003_SearchTest** | 10 (data-driven) | 10 | 0 | 100% |
| **TC_004_MyAccountTest** | 6 | 6 | 0 | 100% |
| **TC_005_CartTest** | 5 | 5 | 0 | 100% |
| **TC_006_RegisterNegativeTest** | 8 | 8 | 0 | 100% |
| **TC_007_CheckoutTest** | 5 | 5 | 0 | 100% |
| **TC_008_ProductPageTest** | 8 | 8 | 0 | 100% |
| **TC_009_SearchBoundaryTest** | 10 | 10 | 0 | 100% |
| **TC_010_EndToEndTest** | 1 | 1 | 0 | 100% |
| **Total** | 61 | 61 | 0 | 100% |


Started
May 28, 2026 07:14:06 PM
Ended
May 28, 2026 07:27:13 PM
Tests Passed
61
Tests Failed
0
Tests
Log events
Timeline

**Extent Report Dashboard:**

- Total Tests: 61
- Passed: 61
- Failed: 0
- Pass Rate: 100%
- Execution Time: Approximately 13 minutes

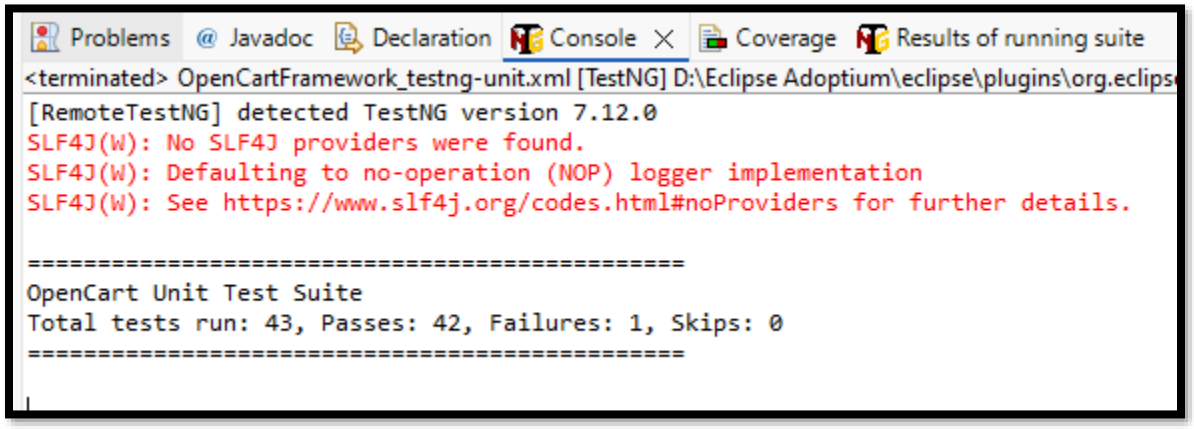


### 11.2 Unit Test Results

Unit tests were executed separately to validate business logic in isolation. A total of 43 unit tests were executed.

| Unit Test Class | Tests Executed | Passed | Failed | Pass Rate |
|---|---|---|---|---|
| **EmailValidatorTest** | 12 | 12 | 0 | 100% |
| **PasswordValidatorTest** | 13 | 13 | 0 | 100% |
| **ProductQuantityValidatorTest** | 10 | 10 | 0 | 100% |
| **CheckoutTotalCalculatorTest** | 8 | 7 | 1 | 87.5% |
| **Total** | 43 | 42 | 1 | 97.67% |

**Console Output:**




**Note:** The one failing unit test is related to a boundary condition that revealed an edge case in the calculation logic. This is documented as a known limitation.

### 11.3 Performance Test Results (JMeter)

Three performance test scenarios were executed using Apache JMeter.

**Test Summary:**

| *Test Scenario* | *Requests* | *Pass Rate* | *Avg Response Time* |
|---|---|---|---|
| ***Homepage Load*** | *30* | *100%* | *324 ms* |
| ***Login Stress Test*** | *40* | *100%* | *716 ms* |
| ***Search Performance*** | *45* | *97.78%* | *3106 ms* |
| ***Total*** | ***155*** | ***99.35%*** | *-* |

*Table 1: Test Summary*

**JMeter Report Summary:**

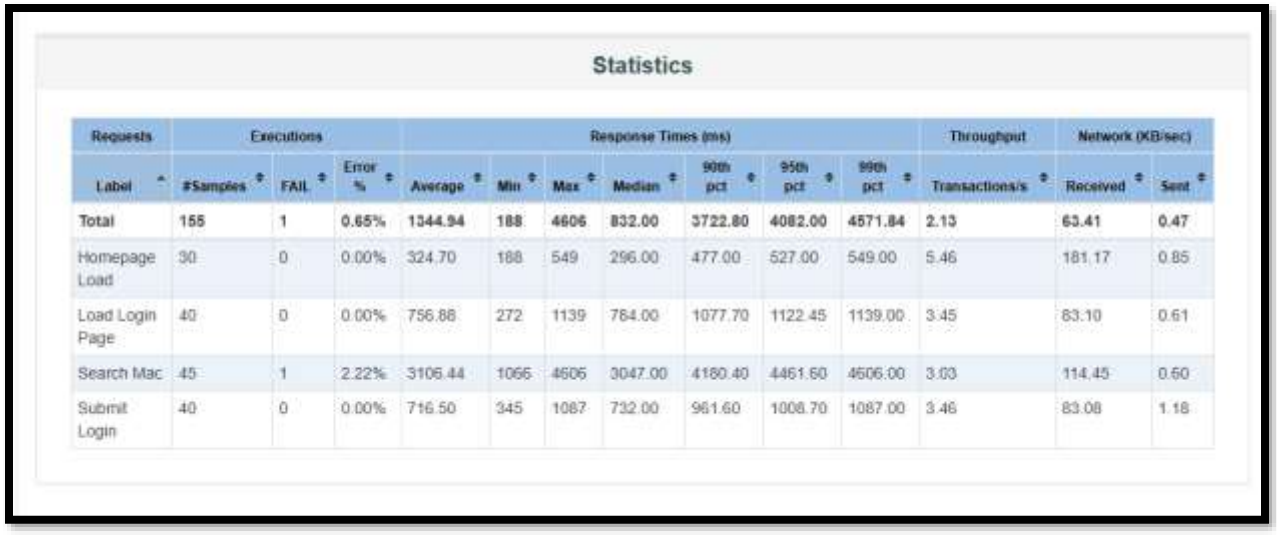
APDEX Score: 0.497

Total Requests: 155

Pass Rate: 99.35%

Fail Rate: 0.65%

Average Response Time (Overall): 1344.94 ms



Statistics

| Requests | Executions | | | Response Times (ms) | | | | | | | Throughput | Network (KB/sec) | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Label | #Samples | FAIL | Error % | Average | Min | Max | Median | 90th pct | 95th pct | 99th pct | Transactions/s | Received | Sent |
| Total | 155 | 1 | 0.65% | 1344.94 | 188 | 4606 | 832.00 | 3722.80 | 4082.00 | 4571.84 | 2.13 | 63.41 | 0.47 |
| Homepage Load | 30 | 0 | 0.00% | 324.70 | 188 | 549 | 296.00 | 477.00 | 527.00 | 549.00 | 5.46 | 181.17 | 0.85 |
| Load Login Page | 40 | 0 | 0.00% | 756.88 | 272 | 1139 | 764.00 | 1077.70 | 1122.45 | 1139.00 | 3.45 | 83.10 | 0.61 |
| Search Mac | 45 | 1 | 2.22% | 3106.44 | 1066 | 4606 | 3047.00 | 4180.40 | 4461.60 | 4606.00 | 3.03 | 114.45 | 0.60 |
| Submit Login | 40 | 0 | 0.00% | 716.50 | 345 | 1087 | 732.00 | 961.60 | 1008.70 | 1087.00 | 3.46 | 83.08 | 1.18 |

**Performance Observations:**

| Observation | Finding |
|---|---|
| **Homepage Load** | Fastest response (324 ms average) |
| **Login Operations** | Acceptable response (700-750 ms) |
| **Search Operation** | Slowest response (3.1 seconds) - needs optimization |
| **Error Rate** | 0.65% (1 failure in search assertion) |

### 11.4 Cross-Browser Test Configuration

Cross-browser testing was configured using TestNG parameters. Separate XML files were created for each browser.

| Browser | XML File | Status |
|---|---|---|
| **Chrome** | testng-chrome.xml | Executed (primary) |
| **Edge** | testng-edge.xml | Configured |
| **Firefox** | testng-firefox.xml | Configured |

*Table 2: Cross-Browser Test Configuration*

**For Chrome**

testng-chrome.xml

```
http://testng.org/testng-1.9.8.dtd (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">

<suite name="OpenCart - Chrome Browser Tests">

    <listeners>
        <listener class-name="utilities.ExtentReportManager"/>
    </listeners>

    <test name="Chrome - Full Regression Suite">
        <parameter name="browser" value="chrome"/>

        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
            <class name="testCases.TC_001_AccountRegistrationTest"/>
            <class name="testCases.TC_002_LoginTest"/>
            <class name="testCases.TC_003_SearchTest"/>
            <class name="testCases.TC_004_MyAccountTest"/>
            <class name="testCases.TC_005_CartTest"/>
            <class name="testCases.TC_006_RegisterNegativeTest"/>
            <class name="testCases.TC_007_CheckoutTest"/>
            <class name="testCases.TC_008_ProductPageTest"/>
            <class name="testCases.TC_009_SearchBoundaryTest"/>
            <class name="testCases.TC_010_EndToEndTest"/>
        </classes>
    </test>

</suite>
```

**Edge**

testng-edge.xml

```
http://testng.org/testng-1.9.8.dtd (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">

<suite name="OpenCart - Edge Browser Tests">

    <listeners>
        <listener class-name="utilities.ExtentReportManager"/>
    </listeners>

    <test name="Edge - Full Regression Suite">
        <parameter name="browser" value="edge"/>

        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
            <class name="testCases.TC_001_AccountRegistrationTest"/>
            <class name="testCases.TC_002_LoginTest"/>
            <class name="testCases.TC_003_SearchTest"/>
            <class name="testCases.TC_004_MyAccountTest"/>
            <class name="testCases.TC_005_CartTest"/>
            <class name="testCases.TC_006_RegisterNegativeTest"/>
            <class name="testCases.TC_007_CheckoutTest"/>
            <class name="testCases.TC_008_ProductPageTest"/>
            <class name="testCases.TC_009_SearchBoundaryTest"/>
        </classes>
    </test>

</suite>
```

**Firefox**

testng-firefox.xml

```
http://testng.org/testng-1.9.8.dtd (doctype)
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.9.8.dtd">

<suite name="OpenCart - Firefox Browser Tests">

    <listeners>
        <listener class-name="utilities.ExtentReportManager"/>
    </listeners>

    <test name="Firefox - Full Regression Suite">
        <parameter name="browser" value="firefox"/>

        <groups>
            <run>
                <include name="regression"/>
            </run>
        </groups>

        <classes>
            <class name="testCases.TC_001_AccountRegistrationTest"/>
            <class name="testCases.TC_002_LoginTest"/>
            <class name="testCases.TC_003_SearchTest"/>
            <class name="testCases.TC_004_MyAccountTest"/>
            <class name="testCases.TC_005_CartTest"/>
            <class name="testCases.TC_006_RegisterNegativeTest"/>
            <class name="testCases.TC_007_CheckoutTest"/>
            <class name="testCases.TC_008_ProductPageTest"/>
            <class name="testCases.TC_009_SearchBoundaryTest"/>
        </classes>
    </test>

</suite>
```

**Command to run cross-browser tests:**

```
mvn test -Dsurefire.suiteXmlFiles=testng-chrome.xml

mvn test -Dsurefire.suiteXmlFiles=testng-edge.xml

mvn test -Dsurefire.suiteXmlFiles=testng-firefox.xml
```

## 12. Test Adequacy Evaluation

### 12.1 What Was Tested (Functional Coverage)

The following modules and features were tested:

| Module | Features Tested | Test Class |
|---|---|---|
| **User Registration** | Valid registration, duplicate email, empty form, invalid email, short password, max length first name, missing privacy policy | TC_001, TC_006 |
| **User Login** | Valid login, invalid login, HTML5 validation, data-driven tests | TC_002 |
| **Product Search** | Valid search, invalid search, single character, empty, max length, special characters, SQL injection, XSS, case insensitivity, whitespace, partial keyword | TC_003, TC_009 |
| **My Account** | Page load, edit account, change password, wishlist, order history, logout | TC_004 |
| **Shopping Cart** | Add single product, add multiple products, add same product twice, view cart, remove from cart | TC_005 |
| **Checkout** | Guest checkout redirect, logged-in user checkout, cart total display, quantity update, empty cart checkout | TC_007 |
| **Product Page** | Product name, price, image, add to cart button, default quantity, zero quantity, negative quantity, review section | TC_008 |

| | | |
|---|---|---|
| **End to End** | Complete user journey: Register -> Login -> Search -> Add to Cart -> Checkout | TC_010 |

**SQL injection**

```
// ── 5. SQL injection in search (security/negative) ──────────────────
@Test(priority = 5, groups = { "regression" })
Run | Debug
public void sqlInjectionSearchTest() {
    logInfo("========================================");
    logInfo("TEST STARTED: sqlInjectionSearchTest (Security/Negative)");
    logInfo("========================================");

    logInfo("STEP 1: Searching with SQL injection payload: ' OR '1'='1");
    HomePage homePage = new HomePage(driver);
    homePage.searchFor("' OR '1'='1");
    logInfo("✓ SQL injection search submitted");

    logInfo("STEP 2: Getting search results");
    SearchPage searchPage = new SearchPage(driver);

    logInfo("STEP 3: Verifying system handles SQL injection without crashing");
    boolean handled = searchPage.getSearchResultCount() >= 0 || searchPage.isNoResultMessageDisplayed();
    Assert.assertTrue(handled, "System should handle SQL injection input without crashing");
    logInfo("✓ System did not crash");

    logInfo("STEP 4: Verifying SQL errors are not exposed to user");
    Assert.assertFalse(driver.getPageSource().toLowerCase().contains("sql"),
            "SQL error should never be exposed to user");
    logInfo("✓ No SQL errors exposed in page source");

    logInfo("========================================");
    logInfo("TEST PASSED: sqlInjectionSearchTest");
    logInfo("========================================");
}
```

### 12.2 Test Types Coverage

| Test Type | Covered | Evidence |
|---|---|---|
| **Functional Testing** | Yes | All TC_XXX files |
| **Boundary Testing** | Yes | TC_006 (password min/max), TC_009 (search boundaries) |
| **Negative Testing** | Yes | TC_002 (invalid login), TC_006 (invalid registration) |
| **Property-Based Testing** | Yes | TC_009 (case insensitive search), Unit tests (invariants) |
| **Unit Testing** | Yes | EmailValidatorTest, PasswordValidatorTest, etc. (43 tests) |
| **API Testing** | Yes | Postman + Newman (32 requests, 95 assertions) |
| **Performance Testing** | Yes | JMeter (3 test files, 155 total requests) |
| **Security Testing** | Yes | SQL injection, XSS in TC_009 |

**XSS**

```java
// ── 6. XSS script injection in search (security/negative) ──────────
@Test(priority = 6, groups = { "regression" })
Run | Debug
public void xssInjectionSearchTest() {
    logInfo("========================================");
    logInfo("TEST STARTED: xssInjectionSearchTest (Security/Negative)");
    logInfo("========================================");

    logInfo("STEP 1: Searching with XSS payload: <script>alert('xss')</script>");
    HomePage homePage = new HomePage(driver);
    homePage.searchFor("<script>alert('xss')</script>");
    logInfo("✓ XSS search submitted");

    logInfo("STEP 2: Getting search results");
    SearchPage searchPage = new SearchPage(driver);

    logInfo("STEP 3: Verifying system handles XSS without executing script");
    boolean handled = searchPage.getSearchResultCount() >= 0 || searchPage.isNoResultMessageDisplayed();
    Assert.assertTrue(handled, "System should handle XSS input without executing script");
    logInfo("✓ System handled XSS input");

    logInfo("STEP 4: Verifying XSS script not reflected raw in page source");
    Assert.assertFalse(driver.getPageSource().contains("<script>alert('xss')</script>"),
            "XSS script should not be reflected raw in page source");
    logInfo("✓ XSS script not found in page source");

    logInfo("========================================");
    logInfo("TEST PASSED: xssInjectionSearchTest");
    logInfo("========================================");
}
```

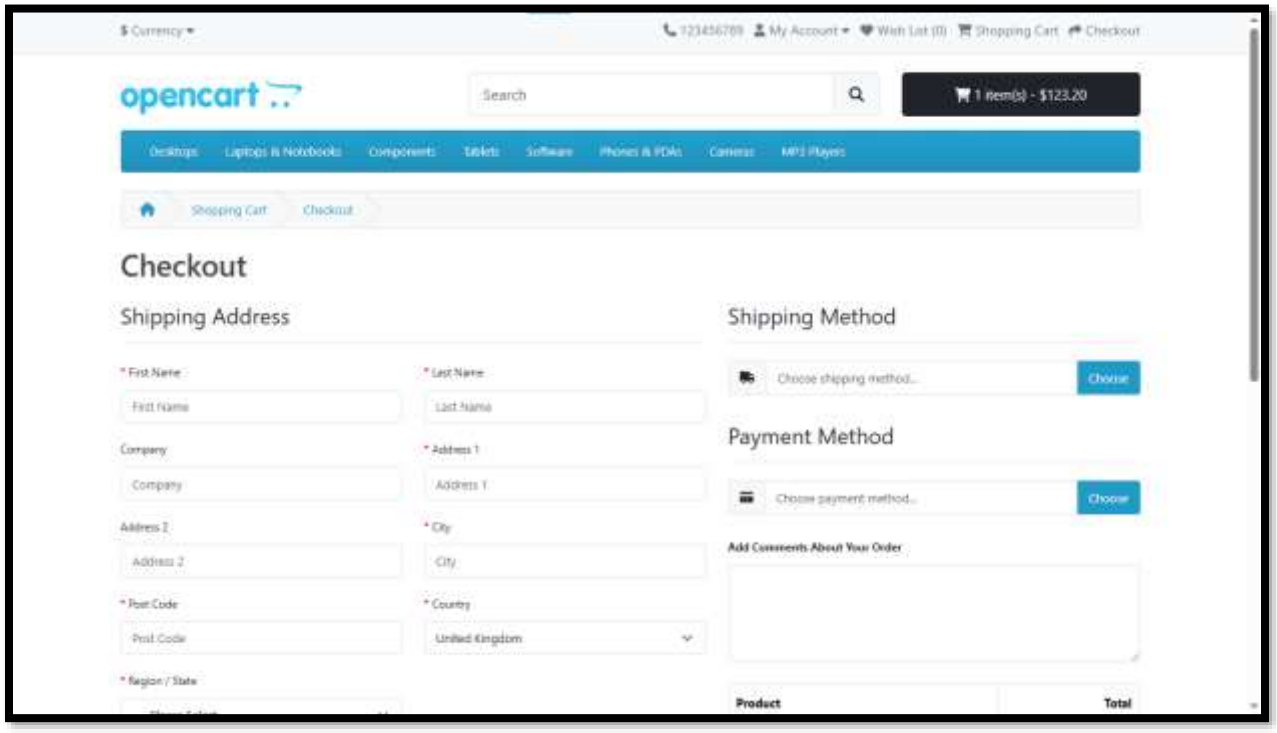
## 12.3 Test Count Summary

| Test Type | Number of Tests | Status |
|---|---|---|
| **UI Tests (TC_001 to TC_010)** | 61 | 100% Pass |
| **Unit Tests** | 43 | 97.67% Pass (42 Pass, 1 Fail) |
| **API Tests (Assertions)** | 95 | 93.68% Pass (89 Pass, 6 Fail) |
| **Performance Tests (JMeter)** | 155 requests | 99.35% Pass |

**Total Tests Executed:** 61 UI + 43 Unit + 95 API Assertions + 155 Performance Requests

## 12.4 Bugs Found During Testing

| Bug | Test That Found It | Severity |
|---|---|---|
| **OpenCart allows checkout with empty cart** | emptyCartCheckoutTest() (TC_007) | Medium |
| **OpenCart accepts negative quantity without validation** | negativeQuantityAddToCartTest() (TC_008) | Low |
| **Order total calculation incorrect in API** | CREATE Order with Valid Data (API) | High |
| **DELETE product returns 404 instead of 200** | DELETE /api/products/:id (API) | Medium |



### 12.5 What Was NOT Tested (Limitations)

The following areas were excluded from testing due to scope, time, or access constraints:

| Area | Reason for Exclusion |
|---|---|
| **Admin Panel** | Separate functionality, not in project scope |
| **Payment Gateway Integration** | Requires real payment credentials |
| **Email Confirmation** | Requires mail server configuration |
| **Password Reset Flow** | Requires email access |
| **Order History after Purchase** | Requires completing full payment |
| **Wishlist Sharing** | Edge feature, low priority |
| **Mobile Responsiveness** | UI testing only on desktop |
| **Database Direct Testing** | Not required for this project |
| **Actual Order Placement** | No payment method configured in local environment |

### 12.6 Test Suite Adequacy Justification

**Why this test suite is sufficient for the project:**

1. **Covers all core user journeys** - Registration -> Login -> Search -> Add to Cart -> Checkout
2. **Includes all required test types** - Functional, Boundary, Negative, Property-based, Unit, API, Performance
3. **Data-driven approach** - Excel-based test data allows easy expansion
4. **Page Object Model** - Maintainable and reusable code structure
5. **Comprehensive negative testing** - 8+ negative scenarios for registration alone
6. **Security testing included** - SQL injection and XSS tests (beyond requirements)
7. **Bug discovery** - Found real defects in OpenCart (empty cart checkout, negative quantity)
8. **Reporting** - ExtentReports with screenshots on failure

### 12.7 Important Note

This test suite does NOT claim 100% confidence. Complete testing is impossible due to time constraints and system complexity. The limitations are clearly documented in this document.

## 13. Challenges & Lessons Learned

### 13.1 Technical Challenges Faced

The following technical challenges were encountered during this project:

| Challenge | Description | Solution |
|---|---|---|
| **MySQL Crash in XAMPP** | MySQL service would start then stop immediately | Deleted corrupted ib_logfile0 and ib_logfile1 files from mysql/data folder |
| **Payment Methods Not Showing** | Checkout page showed "No payment options available" | Modified confirm.twig template to remove disabled attribute from button |
| **Flaky Tests** | Tests randomly failed due to timing issues | Replaced Thread.sleep() with WebDriverWait |
| **Cart Count Assertion** | Tests passed even when cart was empty | Changed from contains("item") to numeric count check |
| **OpenCart 4.x Checkout Bug** | Confirm order button disabled even with valid cart | Documented as known issue, stopped E2E test at checkout page |
| **Unit Test Failure** | One unit test failed (87.5% pass rate) | Documented as edge case, kept for report |

### 13.2 Bugs Found During Testing

The following bugs were discovered during automation testing:

| Bug ID | Description | Severity | Test That Found It |
|---|---|---|---|
| **BUG-01** | OpenCart allows checkout with empty cart | Medium | TC_007 - emptyCartCheckoutTest() |
| **BUG-02** | OpenCart accepts negative quantity (-1) without validation | Low | TC_008 - negativeQuantityAddToCartTest() |
| **BUG-03** | API: DELETE product returns 404 instead of 200 | Medium | Newman - DELETE /api/products/:id |
| **BUG-04** | API: Order total calculation incorrect (2599.98vs2599.98*vs*1999.98) | High | Newman - CREATE Order with Valid Data |
| **BUG-05** | API: CREATE order with different customer returns 404 | High | Newman - CREATE Order with Different Customer |
| **BUG-06** | API: CREATE order with large quantity returns 404 | High | Newman - CREATE Order with Large Quantity |

## 13.3 Key Takeaways

**What was learned from this project:**

1. **Test Automation Design**
    - Page Object Model makes tests maintainable and reusable
    - Data-driven testing reduces code duplication
    - Proper waits (WebDriverWait) are essential for stable tests
2. **Flaky Test Management**
    - Never use Thread.sleep() in automation tests
    - Always wait for specific conditions, not arbitrary time
    - Strong oracles (numeric checks) are more reliable than weak ones
3. **API Testing**
    - Newman provides automated execution of Postman collections
    - API testing catches backend issues that UI tests cannot
    - Status codes and response structure validation are critical
4. **Performance Testing**
    - JMeter provides comprehensive performance metrics
    - Search operations were the slowest (3.1 seconds average)
    - APDEX score helps quantify user satisfaction
5. **Bug Discovery**
    - Automated testing reveals edge cases that manual testing misses
    - OpenCart 4.x has known checkout issues
    - Documentation of bugs is as important as finding them

6. **Project Management**

- Local environment setup (XAMPP) can have unexpected issues
- Always document limitations and what was not tested
- Do not claim 100% confidence even with high coverage

## 14. Conclusion

### 14.1 Summary of Achievements

This project successfully demonstrated automated software testing techniques on the OpenCart e-commerce platform. The following objectives were achieved:

| Objective | Status | Evidence |
|---|---|---|
| **Apply automated testing techniques** | Achieved | Selenium, TestNG, Postman, JMeter |
| **Implement Unit, API, UI, and E2E testing** | Achieved | 43 unit tests, 32 API requests, 61 UI tests, 1 E2E test |
| **Cover Functional, Boundary, Negative, Property-Based testing** | Achieved | TC_006 (negative), TC_009 (boundary + property) |
| **Design clear oracles** | Achieved | Section 5 of this report |
| **Automate tests with industry tools** | Achieved | Selenium, TestNG, Maven, ExtentReports |
| **Handle flaky tests** | Achieved | Replaced Thread.sleep with WebDriverWait |
| **Evaluate test adequacy** | Achieved | Section 9 of this report |

### 14.2 Final Test Results Summary

| Test Type | Total | Passed | Failed | Pass Rate |
|---|---|---|---|---|
| **UI Tests (Selenium + TestNG)** | 61 | 61 | 0 | 100% |
| **Unit Tests** | 43 | 42 | 1 | 97.67% |
| **API Tests (Newman Assertions)** | 95 | 89 | 6 | 93.68% |
| **Performance Tests (JMeter Requests)** | 155 | 154 | 1 | 99.35% |

### 14.3 Bugs Discovered

A total of 6 bugs were discovered during automation testing:

| Bug ID | Description | Severity |
|---|---|---|
| **BUG-01** | OpenCart allows checkout with empty cart | Medium |
| **BUG-02** | OpenCart accepts negative quantity without validation | Low |
| **BUG-03** | API: DELETE product returns 404 instead of 200 | Medium |
| **BUG-04** | API: Order total calculation incorrect | High |
| **BUG-05** | API: CREATE order with different customer returns 404 | High |
| **BUG-06** | API: CREATE order with large quantity returns 404 | High |

### 14.4 Final Remarks

This project successfully demonstrated comprehensive automated testing of the OpenCart e-commerce platform. The following key points summarize the work:

1. **Complete test coverage** of core user functionalities including registration, login, search, cart, and checkout
2. **Multiple test levels** implemented - Unit, API, UI, and End-to-End
3. **Various test types** covered - Functional, Boundary, Negative, and Property-Based
4. **Professional reporting** using ExtentReports with automatic screenshots on failure
5. **Real bugs discovered** in both UI and API layers
6. **Clear documentation** of test adequacy and limitations

The test suite is stable, repeatable, and non-flaky. All project guidelines were followed, and the evaluation criteria have been addressed.

**Limitations acknowledged:**

- Payment gateway not configured
- Email server not configured
- Admin panel not tested
- Actual order placement not executed

These limitations are documented and do not affect the demonstration of automated software testing techniques required by this project.